

MASTER'S THESIS

GMR-ND: A Graph Neural Network Approach for Novel and Diverse Music Recommendations

submitted by

Timo Rödinger

intended degree

Master of Science (WU) / MSc (WU)

Student ID number:	11708552
Degree program:	Digital Economy
Supervisor:	Assoz.Prof PD Dr., Stefan, Sobernig
Co-Supervisor:	Florian Poreba, MSc (WU)

Vienna, March 2025

Abstract

Context. In today’s ever increasing amount of digital content, the music industry is no exception. With the growing size of musical content on digital music streaming services, users can be overwhelmed by the number of possible songs to choose from. Music recommendation models help to make personalized song recommendations based on users’ preferences. Many traditional approaches in music recommendation systems focus on providing recommendations that bring the highest click rates and revenue, but neglect the importance of recommending novel and diverse songs in order to broaden the user’s listening horizon.

Objective. Hence, it is important to introduce recommendation models that focus on recommending novel and diverse songs. The work of Tommasel et al. [60], proposes MRecuri (Music RECommender for filter bUbble diveRsIfication), a model that employs mechanisms such as a Hybrid Graph-Neural Networks and a customized loss function, as well as specific evaluation metrics which aim to increase aspects of novelty and diversity in the model’s recommendations.

This thesis is implementation-oriented where a majority of the work is the replication. This involves verification, modification, extension and implementation of the original MRecuri code. Therefore, this thesis has two main objectives. First, a replication of the MRecuri study is conducted to verify their original results. Second, an extension model is built to compare its performance against the MRecuri model. The extension model takes the core ideas of the MRecuri model and translates them into a fully functional Graph Neural Network model, in contrast to the hybrid MRecuri model, where the Graph Neural Network is only used to a minimal extent.

Method. The major work steps that were taken to achieve the established objectives and derive the results are as follows. First, the original MRecuri model was reproduced to verify the MRecuri study results. For the replication, the original code was reused along with the original data and missing code parts had to be rebuilt according to textual descriptions of the original study. The reproduced results were then analysed and compared to original results.

Secondly, in the extension part of the thesis, the MRecuri model was extended by taking its architectural ideas and translating them into a fully Graph-Neural-Network (GNN)-based approach. The novel GNN approach, GMR-ND (**G**raph based **M**usic **R**ecommender for **N**ovelty and **D**iversity) was implemented with the library PyGeometrics. A total of four GMR-ND models were created to examine architectural differences in the GNN approach. Finally, all GMR-ND results were then compared to the original MRecuri results and the

reproduced MRecuri results.

Results. The comparison between original and reproduced MRecuri results show a deviance that leads to questions of reliability of the original results and highlights the need for more transparency and detail in replication packages for deep learning models, in order to verify results. Therefore, this thesis provides a fully functional and well documented replication package of the adapted MRecuri code. Furthermore, the GMR-ND extension models show promising results for novelty and diversity metrics and point out the potential of Graph Neural Networks in recommender systems. In particular, the extension models performed better in novelty and diversity metrics than the MRecuri models. In contrast, MRecuri models performed better in the relevancy-based metrics precision and accuracy.

Conclusion. In conclusion, through a replication of the MRecuri model and the extension on a Graph Neural Network, GMR-ND, this research was able to compare the relevant results of the different models with each other and draw conclusions from these results. While the results showed promising performance, more work is needed to strike a good balance between relevancy metrics and novelty and diversity metrics. Furthermore, the difficulties in the replication of the original MRecuri model highlight the importance of improved practices in research replication and model development within the context of artificial intelligence, machine learning and data science.

Abstrakt

Kontext. In der heutigen Zeit, in der die Menge an digitalen Inhalten ständig zunimmt, ist die Musikindustrie keine Ausnahme. Mit der wachsenden Menge an musikalischen Inhalten auf digitalen Musik-Streaming-Diensten können Nutzer von der Vielzahl möglicher Lieder überfordert sein. Musikempfehlungsalgorithmen helfen personalisierte Liedempfehlungen basierend auf den Präferenzen der Nutzer zu erstellen. Viele traditionelle Ansätze in Musikempfehlungssystemen konzentrieren sich darauf, Empfehlungen zu geben, die die höchsten Klickraten und Umsätze bringen, vernachlässigen es aber, neue und vielfältige Lieder zu empfehlen, um den musikalischen Horizont der Nutzer zu erweitern.

Ziele der Arbeit. Daher ist es wichtig, Empfehlungsmodelle einzuführen, die sich auf die Empfehlung neuer und vielfältiger Lieder konzentrieren. Die Arbeit von Tommasel et al. [60] schlägt ein Modell namens MRecuri (Music RECommender for filter bUbble diveRsification) vor, um die Aspekte der Neuartigkeit und Vielfalt in den Musikvorschlägen zu erhöhen.

Diese Arbeit ist eine praxisorientierte Arbeit, bei der ein Großteil der Arbeit die Replikation, Verifikation, Modifikation, Erweiterung und Implementierung des ursprünglichen MRecuri-Codes ist. Daher hat diese Arbeit zwei Hauptziele. Erstens wird eine Reproduktion der MRecuri-Studie durchgeführt, um die ursprünglichen Ergebnisse zu verifizieren. Zweitens wird ein Erweiterungsmodell erstellt, um dessen Leistung mit der des MRecuri-Modells zu vergleichen. Das Erweiterungsmodell nimmt die Kernideen des MRecuri-Modells auf und übersetzt diese in ein voll funktionsfähiges Graph Neural Network Modell. Im Gegensatz dazu, benutzt das hybride MRecuri-Modell das Graph Neural Network nur in einem kleinen Umfang.

Methoden. Die wichtigsten Arbeitsschritte, die unternommen wurden, um die festgelegten Ziele zu erreichen und die Ergebnisse abzuleiten, sind die folgenden. Zunächst wurde das ursprüngliche MRecuri-Modell reproduziert, um die Ergebnisse der MRecuri-Studie zu verifizieren. Für die Reproduktion wurde der Originalcode zusammen mit den Originaldaten wiederverwendet und fehlende Codeteile mussten entsprechend der textlichen Beschreibungen der Originalstudie nachgebaut werden. Die reproduzierten Ergebnisse wurden dann analysiert und mit den Originalergebnissen verglichen.

Zweitens wurde im Erweiterungsteil der Arbeit das MRecuri-Modell erweitert, indem seine architektonischen Ideen aufgegriffen und in einen vollständig auf Graph-Neural-Network (GNN) basierenden Ansatz übersetzt wurden. Der neuartige GNN-Ansatz GMR-ND (Graph based Music Recommender for Novelty and Diversity) wurde

mit der Bibliothek PyGeometrics implementiert. Insgesamt wurden vier GMR-ND-Modelle erstellt, um architektonische Unterschiede im GNN-Ansatz zu untersuchen. Schließlich wurden alle GMR-ND-Ergebnisse mit den ursprünglichen MRecuri-Ergebnissen und den reproduzierten MRecuri-Ergebnissen verglichen.

Ergebnisse. Der Vergleich zwischen den Original- und den reproduzierten MRecuri-Ergebnissen zeigt eine Abweichung, die Fragen nach der Zuverlässigkeit der Originalergebnisse aufwirft und den Bedarf an mehr Transparenz und Details in Replikationspaketen für Deep-Learning-Modelle unterstreicht, um die Ergebnisse zu verifizieren. Daher bietet diese Arbeit ein voll funktionsfähiges und gut dokumentiertes Replikationspaket des angepassten MRecuri-Codes.

Darüber hinaus zeigen die GMR-ND Erweiterungsmodelle vielversprechende Ergebnisse für Neuheits- und Diversitätsmetriken und verdeutlichen das Potenzial von Graph Neural Networks in Empfehlungssystemen. Insbesondere schnitten die Erweiterungsmodelle in den Metriken Neuheit und Diversität besser ab als die MRecuri-Modelle. Im Gegensatz dazu schnitten die MRecuri-Modelle bei den relevanzbasierten Metriken Präzision und Genauigkeit besser ab.

Fazit. Zusammenfassend zeigt diese Studie, dass durch die Reproduktion des MRecuri-Modells und die Entwicklung einer auf Graph Neural Network basierenden Erweiterung, GMR-ND, ein Vergleich der relevanten Ergebnisse der verschiedenen Modelle miteinander möglich war und Schlussfolgerungen aus diesen Ergebnissen gezogen werden konnten. Während die Ergebnisse eine vielversprechende Leistung zeigten, ist weitere Arbeit erforderlich, um ein gutes Gleichgewicht zwischen Relevanzmetriken und Neuheits- und Diversitätsmetriken zu finden. Darüber hinaus verdeutlichen die Schwierigkeiten bei der Reproduktion des ursprünglichen MRecuri-Modells die Notwendigkeit für ein verbessertes Verfahren bei der Replikation von Forschungsergebnissen und der Modellentwicklung im Kontext von künstlicher Intelligenz, maschinellem Lernen.

Contents

1	Introduction	12
1.1	Research Gap	13
1.2	Research Questions	15
1.3	Thesis Structure	15
1.4	Hardware and Software Specifications	16
2	Background	17
2.1	Recommendation Systems	17
2.1.1	Collaborative Filtering	20
2.1.2	Content-based Filtering	23
2.1.3	Hybrid Filtering	25
2.2	Filter Bubbles in Recommender Systems	27
2.3	Basic Functionality of Graph Neural Networks	28
2.4	Replication Studies	32
3	Related Work	34
3.1	Music Recommender Systems Using Graph Neural Networks	34
3.2	Recommender Systems for Novelty and Diversity	36
3.3	Replication in Music Recommendation	38
4	Original Study - MRecuri Model	39
4.1	Data	39
4.2	Model Architecture	40
4.2.1	Deep Part	40
4.2.2	Wide Part	41
4.3	Model Training and Loss Function	43
4.4	Evaluation Metrics and Benchmark Models	45
4.5	Results	47
5	Replication	50
5.1	Data Inspection, Pre-processing and Collection	51
5.1.1	Original Provided Data	51
5.1.2	Data Processing	51
5.1.3	Data Collection	55
5.2	Original Study Code Structure	58
5.3	ARP Replication Package Structure	61
5.4	Implementation of the MRecuri Algorithm	61
5.4.1	ORP Functionality	61
5.4.2	ARP Modifications	65

5.5	Embeddings Implementation	69
5.5.1	Novelty and Diversity Definitions	69
5.5.2	Embeddings Creation	70
5.6	Reach-out to Authors	76
6	Model Extension	78
6.1	PyG Graph Creation	78
6.1.1	Graph Structure	78
6.1.2	Input Data	79
6.1.3	Transformation Process	84
6.1.4	Full, Train and Test Graph	86
6.2	GNN Architecture	88
6.3	Training and Predictions	90
7	Evaluation	93
7.1	Model Setup	93
7.2	Benchmark Models	94
7.3	Evaluation Metrics	96
8	Results	99
8.1	Statistical Analysis of the Results	102
8.2	Comparison of Replication Results and Original Results . . .	103
8.3	GMR-ND Model Results	106
9	Threats to Validity	114
9.1	Internal Validity	114
9.1.1	Data Quality	114
9.1.2	Popularity Model	115
9.1.3	Novelty and Diversity Embeddings	115
9.1.4	Benchmark Models	115
9.1.5	FRediECH Model Definitions	116
9.2	External Validity	116
9.3	Construct Validity	117
10	Discussion	119
10.1	Limitations	119
10.1.1	Deviations from the Original Study	119
10.1.2	Scraping and Imputation Process	122
10.1.3	Limited Hardware Resources	122
10.1.4	Communication with the Original Authors	122
10.2	Discussion of Results	122

10.2.1	Replication Results	123
10.2.2	Original and Replicated Results	123
10.2.3	Original and Replication Result Deviations	124
10.2.4	Replicated MRecuri Predictions	125
10.2.5	GMR-ND Results	125
11	Conclusion	128
A	Communication with Tommasel et al.	141
B	First Set of Questions	143
C	Second Set of Questions	145

List of Figures

1	Overview of recommendation system types	20
2	Two basic approaches of collaborative filtering	22
3	Content-based recommender systems	24
4	Illustration of a homogeneous, a bipartite and a heterogeneous graph in a musical context	30
5	Basic architecture of a Graph-Neural-Network	31
6	Schematic diagram of MRecuri	42
7	Distance embeddings t-SNE visualization on a sampled size of 3,307 user embeddings and 5,000 track embeddings	60
8	ARP replication package structure	62
9	Distribution of the 3,307 users from the co-listening graph into 9 communities defined by the Louvain algorithm.	72
10	t-SNE scatterplot of the structural track embeddings	74
11	t-SNE visualization of the content-based track embeddings	76
12	PyG heterogeneous music graph structure with three node types and three edge types	80
13	GMR-ND (GNN) architecture	89
14	Training and validation loss of the shallow and rich GMR-ND models.	110
15	Statistical analysis of the performance of each model against all other models for each metric	111
16	Number of unique tracks in the prediction set per model.	126

List of Tables

1	List of tools used to facilitate the writing and coding process	11
2	Hardware comparison	16
3	Software versions overview	17
4	Data collection details	40
5	Recommendation results comparison for $k = 10$	49
6	Descriptive statistics of the full, train and test user-track graphs	53
7	Descriptive statistics of the user-user graph	54
8	Example of missing <i>position</i> and <i>time</i> attributes in the training graph edge data	55
9	Example of ambiguous song names in the original mapping data	56
10	Top 10 Tags over all songs	58
11	MRecuri model summary	64
12	Excerpt from the modified ARP input data frame for the replicated MRecuri model	68
13	Summary of communication efforts with the original study authors	77
14	Full Graph NetworkX data	82
15	Cleaned full graph input data for PyG graph creation	83
16	Comparison of PyG Graph Objects	87
17	Comparison of the models in the original and replication study	94
18	Replicated recommendation model results.	101
19	Wilcoxon signed-rank Test Results Compared to Mercury . . .	102
20	Comparison of original and replicated results for MRecuri and benchmark recommendation models.	105
21	Results of GMR-ND model variations.	109
21	Continued Results of GMR-ND model	109
22	Deviations between original and replicated implementation due to missing artifacts	121
23	Comparison of the models in the original and replication study	124

Used Tool	Type of Usage	Affected Chapter	Service
Claude	Claude was used in the work to revise individual passages in the sense of linguistic proofreading.	Throughout the whole thesis	https://claude.ai/claude.ai
Cursor (AI powered IDE)	Cursor was used in the work to standardize code sections, promote uniform documentation and simplify redundant sections. Cursor is a coding assistant that simplifies the coding process with auto-completion features.	Throughout all code artifacts	https://docs.cursor.com

Table 1: List of tools used to facilitate the writing and coding process

1 Introduction

State of the Music Industry. The digital music business across the world is in a steady growth, according to the yearly reports of the *IFPI (International Federation of Phonographic Industry)*¹. Their annual 2024 report shows a global revenue growth across the whole music industry of 10.2% and a 11.2% increase in subscription-based streaming revenue since last year. To this end, streaming revenue makes up a total of 67.3% of the whole music market revenue. Compared to the year 2014, when the streaming revenue only made up around 13% of yearly revenue and physical sales still had the upper hand with 40%, there is clear evidence that streaming music online is as popular as never before.

Furthermore, apart from the continued growth of this industry, the global cultural music diversification is also growing. This can be observed by the relatively big growth of developing regions like South America (19%), Africa (24,7%) and Asia (14,9%), which exhibit bigger growth than established regions like North America (7,4%), Europe (8,9%) or Australia (10,8%). In addition to this, the influence of Asian and South American artists in the Top 10 artists around the world show a clear trend that music has long started to transcend cultural and physical borders worldwide.

With this, a clear picture of the importance of the online music streaming world is presented. Hence, it is important to provide users on the music streaming platforms with a good user experience and well constructed music recommendations, that cater to their personal preferences, but also help them to explore new musical horizons and acknowledge other cultural music influences.

Music Recommendation Systems. With the growing online music streaming industry, recommendation systems² play a crucial role in increasing the user’s music consumption experience. In the vast ocean of musical content, recommendation system help to mitigate the information overload problem, that helps users to find content that is relevant for them [38]. For this reason, traditional recommender systems like collaborative filtering [55, 47, 35] and content-based filtering [9, 43, 52], but also hybrid recommendation models [65, 50] were developed. However, many of these traditional models, and even hybrid models, show limited performance, in particular on massive data amounts [64], which limits their potential for real live practical implementations. Furthermore, traditional models can be prone to limitations like the cold start problem or data sparsity problems in the context of

¹<https://globalmusicreport.ifpi.org/>, last accessed 14.02.2025

²The terms *recommendation system* and *recommender system* are used as synonyms throughout this thesis

huge amounts of user-item interaction data.

Graph-Neural-Network Recommendations. To this end, graph-based music recommendation systems that leverage the power of Graph-Neural-Networks (GNNs) [34, 70, 64, 26], have been developed and show promising performances in the field. These graph-based models have the advantage of leveraging the relational information from knowledge graphs in their training process. This way, they can take into account music meta-data like artist, song and album information, as well as possible user-user social interaction data on top of the usual user-song listening interaction data. GNN-based models can potentially use the structural and relational data as well as the rich semantic information in their input graphs to derive latent representations that facilitate the learning process [67].

The need for Novelty and Diversity. The work of Deldjoo et al. [13] identifies challenges in music recommendation systems. One of these challenges focuses on the increasing demand to make diversity and novelty metrics more important when designing music recommendation models in order to increase user experience. Novelty and diversity can help to reduce the impact of filter bubbles, which is a concept of reinforcing similar content in recommendation systems [13]. Two main arguments can be given for the need of an increase in novelty and diversity, to facilitate the escape from content filter bubbles.

First, novelty and diversity can foster a culturally diverse music consumption environment for users. According to Taramigkou et al. [58], many recommendation systems suggest very narrow and similar sets of musical items. Hence, they highlight the need for users to broaden their musical perspective. They argue that widening a users musical perspective is important since exploring different and alternative music can help to experience new perspectives which again can help to understand the feelings of other people.

Secondly, from the perspective of streaming providers, increasing novelty and diversity metrics might lead to more user satisfaction, which in turn can contribute to the platform success and revenue increase [27].

1.1 Research Gap

According to Tommasel et al. [60], recent studies [61, 14] have tackled the problem of more diverse recommendations in recommender systems, but disregarded novel recommendations in their approaches. Furthermore, they argue that most previous models also disregard user relations and social context in the recommendations. Therefore, MRecuri tried to fill this gap by providing a model that takes into account both novelty and diversity metrics, as well as social relations processed by their hybrid Graph Neural Network

(GNN) approach.

While the main idea behind MRecuri and their implementation of user relations and more novel and diverse recommendations seems promising, the MRecuri model was implemented using a combined approach of a Convolutional Neural Network (CNN) and only a minimal use of a Graph Neural Network (GNN) model in their overall model architecture. Their work even acknowledges that they have a full music knowledge graph with music metadata and relations between users, songs and artists, but are not using it due to computational limitations. Hence it seems that their model was limited due to certain computational constraints.

This thesis aims to extend the MRecuri model into a fully fledged graph-based model that can process the entire music knowledge graph including it's music metadata. User's social relations, user to song relations and song to artists relations will all be encoded in a music knowledge graph.

With the existence of the PyGeometric library, computational constraints regarding Graph Neural Networks with limited hardware may be circumvented. Therefore, the motivation of the extension is to verify, if such a full graph-based model approach is possible at all, with similar computational resources as the original work, when working with the recently developed graph library PyTorch Geometric. Furthermore, the performance of the extension GNN and the original MRecuri study will be compared, to see if full GNN models show potential for future recommendation system use cases.

In detail, the main goal of this thesis is to replicate the study *Haven't I just Listened to This?: Exploring Diversity in Music Recommendations* by Antonela Tommasel, Juan Manuel Rodriguez and Daniela Godoy. The study proposes an implementation of a hybrid music recommender system that enhances diversity and novelty.

Building on the theoretical methods employed in the original study, an extension in a fully graph-based GNN model is built. The main aspect of increasing novelty and diversity are kept as central aspect, as according to the original study. Furthermore, the influence of certain architectural choices, such as the implementation of a custom loss function, which are employed in the original study are tested in the extension.

Overall, this thesis contributes to research by tackling the above issues of traditional recommendation systems and more novel and diverse music consumption twofold.

- First, a replication of the MRecuri study that focuses on increasing novelty and diversity in music, is conducted to verify the results of the original study.

- Second, this thesis extends the original study, maintaining the goal of increasing novelty and diversity metrics, with a novel approach in a fully functional GNN-based model. The results of the original, replicated and novel model are compared based on relevancy and novelty and diversity evaluation metrics.

1.2 Research Questions

In accordance with the research gap, the research questions of this thesis are formulated as follows.

- *RQ1: Can the results of the original study by Tommasel et al.[60], which implement a novelty and diversity focused music recommendation model, be replicated?*
- *RQ2: Can the obtained results from the Graph-Neural-Network extension model improve upon the original study results?*

Answering these research questions will provide insights into the defined research gap and will assist future research in enhancing the transparency and accessibility of their work. Furthermore, they will encourage the utilisation of fully graph-based GNN models with PyGeometrics, specifically in music recommendation systems.

1.3 Thesis Structure

The remaining structure of the upcoming sections in this thesis is as follows. First, the background section will provide insights into the necessary concepts that are used by the original, replication and extension models. These concepts include a general introduction to the topics of recommendation systems, filter bubbles in recommendation systems, Graph-Neural-Networks and a definition of replication studies. Next, the related work will give the reader additional content on studies that have employed GNNs in their work already and shed light on music recommendation systems that use novelty and diversity in their architecture. Following, the original study, its architectural details and results will be elaborated on in detail. Subsequently, the process of the replication by the thesis' author is explained in detail and necessary modifications on the original code and methodology are presented. Next, the extension part and the implementation of the GNN-based model is described, along with architectural details. The section on experimental design introduces the final models, benchmarks and evaluation metrics that are used to derive the results. The results Section 8 exhibits the original, replicated and

extension results and compares their outcomes. Consequently, the threats to validity are discussed and possible shortcomings of the study are laid out. Finally, the discussion of results, limitations and future research section discuss implications of the thesis before the final conclusion.

1.4 Hardware and Software Specifications

Hardware. In this thesis the replicated models, the extension GMR-ND models and benchmark models have been implemented based on the same hardware. Hardware consists of a custom built PC. As a CPU an Intel 12th Gen i9-12900k was used. The RAM was compromised of a DDR-5 64GB with 5.200MHz module. As a GPU an NVIDIA GeForce RTX 3060 with 12GB memory was used.

The original paper by Tommasel et al. [60] used different hardware. Table 2 shows the difference between hardware used in this thesis and in the original study.

	Original Study	This Study
Model	Asus Predator Helios 300 (Laptop)	Customized PC
CPU	i7-11800H	i9-12900K
GPU	GeForce RTX 3060 6Gb	GeForce RTX 3060 12Gb
RAM	16GB	32GB

Table 2: Hardware comparison

Software. The software used in the original study was not specified. For the replication and extension different versions of python and additional libraries had to be used for different models. For example, different python versions had to be used for replicating the MRecuri model and for the extended GMR-ND. Two different environments were used, one with python version 3.11.8 and one with version 3.12.2 to comply with compatibility of some of the necessary libraries. The reason why the replicated MRecuri model used 3.11.8 was because this was the latest version that was compatible with the *Spektral* library, which was needed to run the MRecuri model. On the other hand, the extension GMR-ND models worked only with python version 3.12.2 which were necessary to run the PyTorch and PyTorch Geometric libraries and their dependencies. All benchmark models could be run on the same specifications as the GMR-ND models, so python 3.12.2. All code was purely written in python. Table 3 shows an overview of the most important libraries used in the MRecuri replication model (left) and the benchmark and GMR-ND models (right).

Software	Replication	Benchmark & GMR-ND
OS	Windows 11	Windows 11
Python	3.11.8	3.12.2
torch	2.0.1	2.2.2+cu118
TensorFlow	2.15	2.16.1
networkx	3.3	3.2.1
spektral	1.3.1	/
PyGeometrics	/	2.5.3
pandas	2.2.2	2.2.2
matplotlib	3.8.4	3.9.0
numpy	1.26.3	1.26.4

Table 3: Software versions overview for the MRecuri replication model, the GMR-ND extension model and the benchmark models

2 Background

This chapter will cover basic knowledge and background information on the topics of music recommendation systems, filter bubbles and the proposed GNN model in Section 6. Specifically, it provides in-depth explanations on the concepts of collaborative, content-based and hybrid filtering methods for recommender systems. In this regard, challenges of recommendation systems like the cold start problem and data sparsity will be introduced and discussed. Additionally, the fundamentals of filter bubbles in the context of music recommendations will be elaborated on, to highlight the importance of breaking out of filter bubbles and providing novel and diverse recommendations. Furthermore, the fundamentals of Graph Neural Networks (GNNs), in terms of their functionality and architecture, are covered to help the reader understand the proposed model in Section 6. Finally, the importance of replication studies is highlighted, to show the need to provide correct and transparent replication materials and artifacts.

2.1 Recommendation Systems

In the ever-increasing creation of content in the digital era and online platforms, the sheer volume of content that is available, including videos, social media posts, games or music can be overwhelming for users. In daily life one may be subject to millions of choices on what clothes to buy, what videos to watch on the internet, what food to eat and, in this context, what music to listen to. This phenomenon is coined as *information overload* [38].

In the analog past, people often relied on word of mouth recommendations from people they trust when making decisions. However, in the current digital age, due to information overload this is not always feasible for the vast number of possible choices. Consequently, digital recommendation systems have been developed to assist individuals in making choices by suggesting and filtering content that aligns with their preferences and interests.

Recommendations systems often use a user’s historical data to analyze their past preferences and behaviors. Furthermore, recommendation systems may look at the relational aspects of several users and items pairs to infer which users have similar behavioral patterns and preferences. These methods result in the categories of the most well-known techniques of collaborative filtering and content-based filtering methods [55].

In light of the potential for recommendation systems to provide users with relevant content, these systems have been developed with the objective of increasing user engagement with online services such as music, video, and social media platforms. These platforms have an incentive to keep users on their platform for as long as possible with the help of their recommendation systems to increase potential revenue. Hence, recommendation systems are first and foremost driven by the goal of generating sales and making profit[47]. Therefore according to Shakirova [47], four technical goals exist to make the recommendations appeal to the users:

1. Relevance: The relevance of recommendations must be personalized to the users preferences to appeal to their interest.
2. Novelty: Recommended items should be new content, not old content that a user may have already consumed.
3. Serendipity: This feature is similar to novelty but describes the phenomenon of discovering something exciting without having set out to do so. In a musical context, it is , therefore, important to not only recommend songs of the same genre but to also broaden the users experience by recommending songs out of their usual comfort zone that are still deemed potentially relevant.
4. Diversity: This describes the risk of losing interest of a user because the recommended elements are too similar to each other. A user that gets only recommended tracks that they have already heard many times, is likely to lose interest.

Note that the above mentioned goal of novelty is dependent on the specific use case. For example in an online store environment where products

like books, shoes or clothes are offered, making the same recommendations repeatedly to one user presumably decreases the effectiveness of the recommendation, since the user has no need for the same book twice. In the context of music streaming, however, this may be a different case. A user who has listened to one song may enjoy a recommendation of this song again and again. Therefore, the context of the recommendation system has to be taken into account when aiming for novelty.

To elaborate on the two most prominent kinds of recommendation systems, there exist content-based systems, which take into account users historical interactions with the item and the content characteristics itself, like genre, tempo or artist name, to infer similarities between the content the user has consumed. To this end, both information on the user’s past interactions with items and a method to extract information about the content itself is needed. Collaborative filtering, on the other hand, usually works with some kind of user ratings or scores that depict how much a user liked interacting with that content. A real-life example would be the user ratings of various video-streaming platforms like Netflix for their movies. The ratings, which are a type of user-feedback specific to a particular item, are utilized to categorize users who have consumed similar items as the original user. Consequently, a user will be recommended content that has been rated highly by other users that share his interests [48]. Collaborative filtering can be divided into model-based filtering, memory-based filtering, item-based and user-based approaches [49].

Besides the well-known and widely established methods of collaborative filtering and content-based filtering, there exist various other methods like hybrid-based methods, context aware methods, graph-based methods and more, as stated in [54].

Moreover, machine learning-based methods are presented in the work of Ruihui [38], and cover a variety of deep learning-based recommender systems like Autoencoders, Restricted-Boltzmann-Machines (RBMs), Recurrent-Neural-Networks (RNNs), Convolutional Neural Network (CNNs) and Deep Belief Networks (DBN), which are not relevant for this thesis, but that the interested reader can refer to. Deep learning models can be employed as content-based, collaborative filtering and hybrid methods. Also note that the Graph-Neural-Network approach can belong to all three categories. The GNN used in this thesis can be categorized as a hybrid method. Figure 1 gives a broad overview of different recommender system types.

Note, that the term *hybrid*, is used multiple times in a different context throughout this thesis. To make this distinction clear, the term *hybrid* can be categorized in the following two ways:

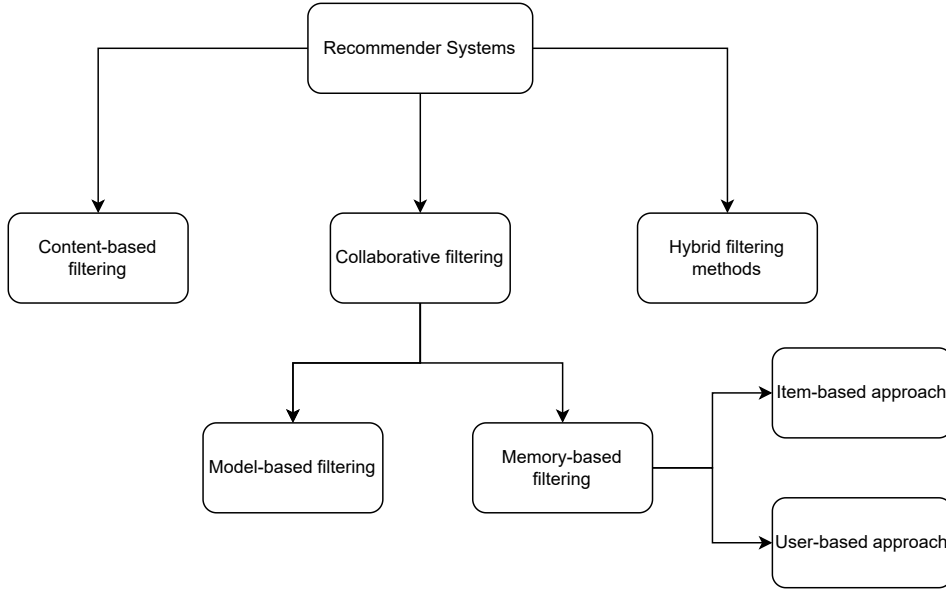


Figure 1: Overview of recommendation system types

1. In this background chapter and when referring specifically to recommender systems, *hybrid* means that a combination of at least two filtering methods is used to create a *hybrid filtering method*. A hybrid recommender system is consequently a model that uses a hybrid filtering method.
2. On the other hand, when discussing the MRecuri model, *hybrid* has a different meaning: it refers to the model’s architectural design that incorporates multiple types of neural networks. This usage of *hybrid* describes the model’s internal structure rather than its filtering methodology.

2.1.1 Collaborative Filtering

Historically collaborative filtering techniques developed from filtering systems for email services. One of the first of these systems called *Tapestry* [21] was developed by Xerox in the 1990s to rate emails. It’s drawback was that users had to manually write queries to make use of the system. Therefore, after that an autonomous recommendation system called GroupLens [42] was developed, which recommended posts to users on the platform Usenet. The recommendations provided to the user were based on posts that other users

with similar characteristics to the original user had consumed. Structurally, collaborative filtering techniques can be divided first into memory-based and model-based methods and furthermore memory-based techniques can again be divided into user-based and item-based methods [49].

To this extent, Collaborative Filtering (CF) uses the similarities between item ratings or the user’s rating behaviour to make recommendations. Ratings in this context can be explicit or implicit. Explicit ratings are for example numerical ratings on a scale from 1-5. Implicit ratings, on the other hand, can be derived from user behavior. In a musical context, platforms could for example track the listening behavior in terms of seconds listened to a song, skip rates of songs, favorite rates or repeat rates. Depending on if the ratings are used to compare items or users to each other, there is a distinction between user-based CF algorithms and item-based CF algorithms [55].

Furthermore, it is important to specify what similarity of items or users means in this context and how it can be measured. Similarity, which can also be seen as a factor of closeness, is the definition of some kind of distance between two items. Different kinds of distance metrics like Euclidean distance or Manhattan distance can be chosen. Besides these distances sometimes other types of similarities have to be chosen for example in the case of non numerical values. Hence, there exist other similarity measures like the cosine similarity, the conditional probability-based similarity and the Pearson correlation similarity [47].

In the following, a more detailed examination of the cosine similarity metric will be provided, given that it is a key component of the original replication study, which will be covered in more detail in Chapter 4. Cosine similarity considers items as vectors and calculates the cosine of the angle between two vectors in a multidimensional space. In a machine learning context these vectors are often referred to as embeddings. This cosine of the angle between the items is an indicator for the similarity of the items. The smaller the angle is, the higher is the similarity of the items. Given items i, j as vectors $\vec{i}, \vec{j}$ the dot product is computed and then divided by the magnitude of each vector which yields the cosine similarity [30]. The cosine similarity is given in Equation 1.

$$\text{sim}(i, j) = \cos(\vec{i}, \vec{j}) = \frac{\vec{i} \cdot \vec{j}}{\|\vec{i}\| \|\vec{j}\|} \quad (1)$$

Memory-based CF methods are divided into user-based CF and item-based CF. In a user-based CF, users can be represented as user profiles and are put into a user-item interaction matrix in which rows represent users, and columns represent items. User-based collaborative filtering assumes that

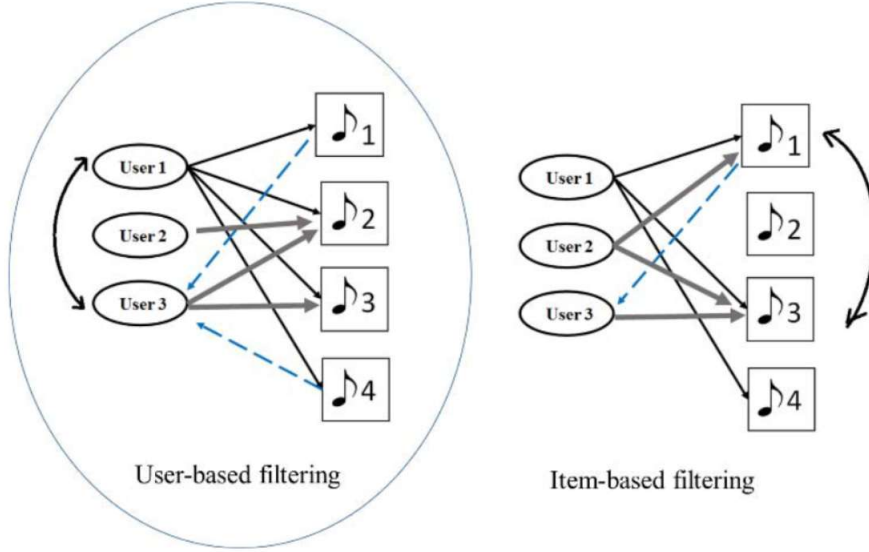


Figure 2: Two basic approaches of collaborative filtering, taken from [47, p. 1]

users can be grouped together with other users that have similar preferences based on their item ratings by using similarity measures like the above discussed cosine similarity. Therefore, user-based CF can recommend items to a user-based on the preferences and behaviors of similar users [65]. An example in the music context is depicted in Figure 2: User 1 and 3 share their interest in songs 2 and 3. User 1 has also already listened to song 1 and 4, while user 3 has not yet listened to those songs. Hence, due to their similar preferences the user-based CF will likely recommend user 3 the songs 1 and 4.

In a similar fashion the item-based CF approach works by grouping similarly rated items together. Here, it is assumed that a user who has interacted with item A, and this Item A is highly similar to item B, will likely also interact with item B [49]. In the context of music as depicted in Figure 2 (right), we assume that song 1 and 3 are similar. Since User 3 has listened to Song 3 in this instance, the item-based CF will recommend song 1 to user 3 as well.

However, problems with item and user sets can arise in a CF context. For example, when imagining an item set that is very large, which is the case in the number of songs in the catalog of today’s music streaming services, these large item sets can lead to problems for recommendation systems. One problem is *Data Sparsity*. Data sparsity refers to the lack of information in this case the lack of ratings on many items in the set. In the music context

imagine a rating is only given to ten out of one million songs. When building a user-item rating matrix many of the rating tuples would result in being zero. This leads to the issue of not having enough rating data to effectively compute similarities between users or items. Data sparsity can also lead to *cold start* problems which refer to the issue that newly added items to a catalog usually have no initial ratings. This results in a cycle in which the problem is that items without ratings are not recommended to users and conversely do not receive any ratings without user interaction. Furthermore, there exists the problem of scalability for CF recommendation systems in large scale environments due to expensive computations over a large number of users and items that have to be constantly updated due to the addition of new users and items [55].

On the other hand, besides memory-based methods, there are the model-based CF algorithms. Model-based recommenders employ machine learning and statistical models to predict user preferences and make recommendations. Accordingly, user-item interaction data is learned by the models in the form of low-dimension vectors, also called feature vectors. Through Latent Factor model techniques such as Matrix Factorization (MF) or Singular Value Decomposition (SVD) the models try to infer information from the latent features about user and item interactions. If the model finds a strong association between the latent features of users and items, it is likely to recommend those items. Model-based CF approaches generally aim to handle the previously mentioned problems of data sparsity and scalability better than their memory-based counterpart. Due to dimensional reduction large datasets can be processed more efficiently and sparse data and cold start problems can be better dealt with by having underlying latent factors that may show associations even for sparse data [49].

2.1.2 Content-based Filtering

Content-based recommendation systems rely on the item’s features and the past user interactions and ratings with items. Therefore, it is critical that the system can obtain knowledge about the item itself through some sort of information retrieval or feature extraction [50]. Historically content-based recommendation systems have their roots in information retrieval and information filtering and started out in text-based context like website or book recommendations. In this context, a well-known method to weigh the extracted text-based features in keywords is the *term frequency / inverse document frequency* (TF-IDF) [1].

In the context of content-based music recommendation systems, audio features play a significant role in the construction of item profiles. According

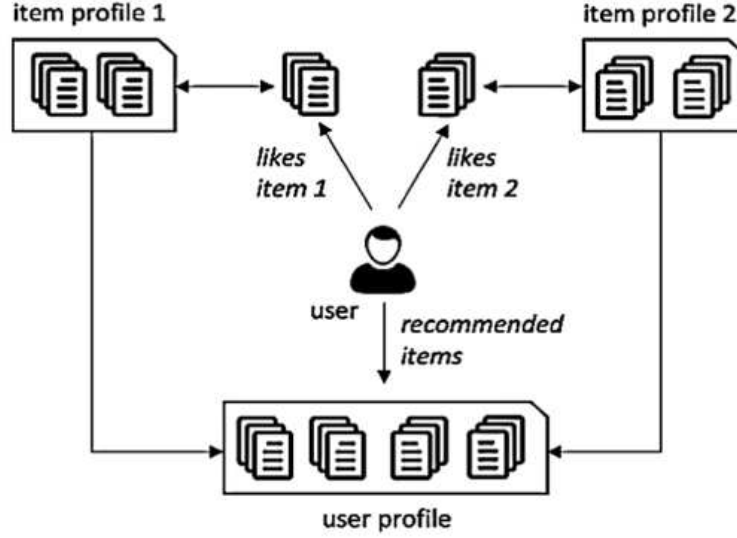


Figure 3: Content-based recommender systems, taken from [43, p. 4]

to Song et al. [52] rhythm and timbre are important music features, in addition Singh et al. [50] add that other musical features like beats per minute and music metadata like genre, artist, composer, song length and lyric language are crucial in finding similar tracks. In a recommender system context, these features are usually used in a vector and feature embedding format.

Based on the similarities of the item features, items get grouped in item profiles. User profiles on the other hand are based on past item interactions of users. As an example, as depicted in Figure 3, take *User A* who has shown interest in *Song 1* through the implicit feedback of listening to this song very often in the past two weeks. Therefore, it is likely *User A* is also interested in other songs in the same item profile as *Song 1*. Consequently, his user profile is filled up with songs from the same item profile as *Song 1*. Moreover, if *User A* liked another song, *Song 2*, in the past, the same process of adding songs from the *Song 2* item profile to his user profile repeats. With this process each user accumulates a unique and individual user profile which is the basis for the recommendations given to that user by the system [43]. The specific items recommended to a user based on their user profile are dependent on the chosen method. There are a number of methods for identifying the optimal item to suggest, including heuristic, probabilistic, and classification techniques such as nearest neighbour or linear classifiers [46].

General advantages of content-based recommendation systems are that they create user profiles which are independent from one another, since each user has their very own set of various item profiles in their user profile.

This aspect has a positive influence on the security and privacy of the users data. Furthermore, given that items are sufficiently well described in the system context by their features the cold start problem can be overcome. As already stated, the cold start problem describes the scenario where a new item (a song in our case) is added to the system database and may have trouble to be recommended based on missing information. But in contrast to CFs the items do not depend on ratings of other users but on the item features themselves. Therefore, if new items have sufficient information to be grouped with other items into item profiles the cold start problem can be mitigated [9, 43].

However, content-based systems are not without their own set of limitations either. Firstly, the quality of the recommendations is dependent on the availability of the item features. If there is little or no information on items, in our context the audio features or metadata of songs, the recommendations will not perform well. Therefore the feature data has to be added manually or if possible in the given context processed, retrieved and assigned to the items automatically [9].

Although concerning the items, the cold start problem can be mitigated, the *New User Problem* may arise instead. New users lack an interaction history and, as a result, cannot be recommended similar items to those they have interacted with. Consequently, recommendations will only become more accurate to the user’s preferences once the user has started to interact with items [13].

Furthermore, the challenge of overspecialization can occur. This problem refers to the fact that a user according to his interaction history only gets recommended items that are very similar and hence may be limiting the users musical diversity. In a musical context, this would refer to the scenario of a user who, for example, only listens to a specific genre and is recommended only other songs from this genre. This will limit the users future interaction to only this genre and may trap him in a bubble while the user is potentially missing relevant songs from other genres. Methods like introducing randomness have been used to cope with this issue. This issue plays a pivotal role especially in music recommendation as covered in the introduction and will be given more coverage in Chapter 2.2 on Filter Bubbles as the goal of the introduced model in this thesis is to mitigate potential filter bubbles and to recommend more novel and diverse songs for users [1].

2.1.3 Hybrid Filtering

Besides collaborative and content-based filtering techniques there exist hybrid filtering recommender systems. The general aim is to aggregate and

combine two or more recommender methods in such a way that potential drawbacks and limitations of the individual methods can be mitigated and by combining them a better recommendation can be made overall. The exact way on how to combine the individual models can be done in various ways. [43] Specifically, there exist 4 types of hybrid methods in this context according to Adomavicius and Tuzhilin [1].

1. **Hybridization of Predictions:** In this approach two or more methods are developed and deployed individually and both make recommendations. The final predictions of both models are then combined or the predictions of one is chosen over the other in each instance according to some rules. In a musical context imagine one CF and one CB model. Both models recommend a set of items to a user with a certain confidence. The model that has a higher confidence for the prediction gets chosen for this instance. Another way to combine the recommendations is to combine both predictions into one final prediction.
2. **Content-based Characteristics in Collaborative Method Approach:** Alternatively, in this approach the main model is based on a collaborative technique. In addition to that some characteristics of content-based methods like the content-based user profiles are also used. By using these characteristics the model can now make similarity measures between different user profiles in contrast to solely using rated items as in the individual CF approach. This helps to address some data sparsity problems as in many instances the shared item pairs between users will be rather low.
3. **Collaborative Characteristics in Content-based Method Approach:** In turn, this works also the other way around by incorporating features from CF systems into CB systems. Here an exemplary approach is to group multiple individual users into user profile group vectors by reducing their dimensionality with certain techniques like Latent Semantic Indexing (LSI). This again yields an increase in performance when comparing it to the purely content-based models that use individual users, instead of aggregated user profile groups.
4. **Unified Model Approach:** This final approach does not generally favor the CF or CB approach as one of their bases, but may incorporate features of both models in combination with yet other techniques. The model is different enough from purely CF or CB-based methods to be categorized as a completely new approach, but may still incorporate basic ideas of one or both models. Examples for Unified Models are

rule-based classifier models using both content-based and collaborative characteristics, probabilistic model that inherit both characteristics or regression models that inherit some features from both CB and CF models.

In essence, hybrid filtering uses the basic functionalities of content-based and collaborative recommender systems to various degrees to mitigate their common problems and enhance predictions by combining different types of features. Hybrid models, when implemented correctly, outperform their individual model counterpart [9].

2.2 Filter Bubbles in Recommender Systems

The concept of filter bubbles was first coined in a book by Eli Pariser in 2011 [39]. This term describes the situation, where recommendation systems provide users with a self-reinforcing cycle of narrowing views on the total content landscape [31]. Berman and Katona [7] describe the concept of filter bubbles within social networks as mechanisms that lead to increased polarization of opinions by recommending content that aligns with users' existing beliefs. These studies highlight, that filter bubbles prevent users from exploring more diverse topics and viewpoints, while reinforcing their pre-existing opinions. Consequently, the isolation and reduced diversity in consumed content are fundamental outcomes of filter bubbles.

Filter bubbles are illustrated in one study by the example of a video streaming platform [66]. When a user clicks on a certain type of video, recommendation systems will suggest similar content in the future. While this content may interest the user, it limits opportunities to discover other relevant items that could potentially broaden the user's interest to different topics. The user reinforces this trend, by following the recommendations, whereas the system receives new confirmation on the validity of the suggested content. This gradually results in a feedback loop that strengthens the filter bubbles more and more [12].

It should be mentioned that filter bubbles do occur in many domains and are not exclusive to recommendation systems in a musical context. This study, however, focuses specifically on music recommendation systems. Therefore, it is essential to examine how filter bubbles influence users' music consumption. In the domain of music, recommendation systems play an increasingly important role to help customers in deciding between a large variety of music items. This helps users to streamline their selection process of new items [46]. While the current recommendation systems employed in the music domain help users to identify interesting items, many of them lack

to provide diversity in their recommendations and do not support the exploration of new content and the expansion of interests [58]. Studies indicate that the current set of music recommendation systems favors already popular songs, therefore, decreasing the chance of exploring new and less-known musicians [40].

2.3 Basic Functionality of Graph Neural Networks

This section provides a basic introduction to the functionality of Graph-Neural-Networks (GNNs). Specifically, the focus is on GNNs in the context of recommender systems. The subjects of graph types, neighborhood aggregation, types of prediction tasks, GNN types, use cases and evaluation metrics will be elaborated on in more detail.

Historically the predecessor of the modern GNN can be traced back to the work of Sperduti and Starita [53] who introduce the concept of neural networks over structured patterns like sequences, trees or graphs. Building on their work, the work of Scarselli et al. [45] coined the term *Graph Neural Network Model* and showed that graphs can be directly processed and used by their model, while acknowledging the high computational complexity necessary for processing the model.

While graph based deep learning models and *normal* deep learning models like Convolutional-Neural-Networks (CNNs) or Recurrent-Neural-Networks (RNNs) both try to learn hidden patterns in data, the main differentiation between them is, that GNNs in addition take into account relations with connected entities in their learning process [69].

Types of Graphs. The definition of a graph, based on the work of Wu et al. [69], is introduced as follows. A graph is a data construct that holds entities and relations between those entities. Graphs are commonly notated with $G = (V, E)$, where V is a set of vertices or *nodes* and E is a set of edges, that represent relations between the nodes. Edges connect a *source node* to a *destination node* and can be directed (one way) or undirected (both ways). To this end, an adjacency matrix denoted with A , represents information on which nodes are connected by which edges in the graph. Nodes and edges can additionally hold certain attributes X , usually represented in a vector form. Furthermore, if there is semantic meaning in the node types and edge relations, such a graph can also be called a knowledge graph.

There exist multiple types of graphs, whereas the most important distinction in our context is to be made between homogeneous graphs and heterogeneous graphs. While in homogeneous graphs only one type of nodes and edges exists, in heterogeneous graphs different types of nodes and edges can occur [68]. A bipartite graph is a special case of a heterogeneous graph with

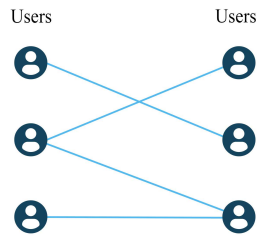
only two node types.

Figure 4 depicts an example of a homogeneous, a bipartite and a heterogeneous knowledge graph in a musical context. The homogeneous graph (a) shows a social graph with only user nodes and connections between those user nodes. The bipartite graph (b) shows interactions in a user-item graph between users and songs. The heterogeneous knowledge graph (c) represents the combination of the social graph and the user-item graph in one graph. Furthermore, in our case the heterogeneous knowledge graph (c) is a multipartite graph, which can be partitioned into several bipartite graphs when separating their disjoint edges and nodes. The user-user relations on the left side of graph (c) are made up of a the user-user graph (a), a homogeneous graph. The middle part of (c) is made up of user-song relations as depicted in graph (b) and is a bipartite graph on its own. Partitioning a larger graph such as (c) and looking at its smaller disjointed subgraphs is called *projection* [20].

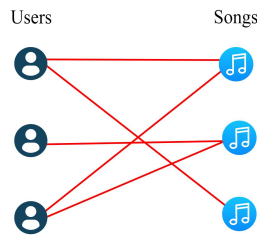
User-item graphs are specifically useful in the use case of recommendation models. To this end, note that the MRecuri model explained further in Chapter 4 uses a homogeneous knowledge graph, while the extension model in Chapter 6 leverages a heterogeneous knowledge graph.

GNN Learning. The general functionality of a GNN can be described by the fact that the model learns node features and additionally aggregates information from neighboring nodes into the initial node. This way, a single node is a combined representation of its own information features, plus the structural and relational information from its neighboring nodes. To this end, a GNN is comprised of multiple convolutional layers that distribute node information in the graph network and compute weighted averages of vectors for nodes, based on their own and their neighbors information. Non-linear activation functions, such as *ReLU* or *Sigmoid*, are then used to create updated feature vectors for each node and the weights used in the convolutional layers are updated through back propagation during the training process [2].

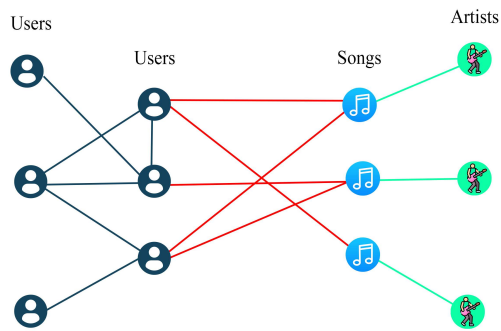
A simplified general architecture with common modules that form the basis of GNNs can be given by the input layer, the graph convolutional layers, pooling layers and the output layer. The input layer represents the necessary input graph, which can be in the form of an adjacency matrix or an edge list, and potential node feature vectors. The convolutional layers propagate data in the graph network and aggregate node information. Pooling layers use techniques like sum, mean, or max pooling to reduce the dimensionality of the graph representation and capture global graph properties. Finally, the output layer generates the output of the GNN task, for example the prediction of a class label or a prediction probability. Note that architectures can vary widely depending on the particular use cases and model tasks [2]. Figure



a) User-User Social Network Graph



b) User-Song Bipartite Graph



c) Heterogeneous Music Graph

Figure 4: Illustration of a homogeneous, a bipartite and a heterogeneous graph in a musical context

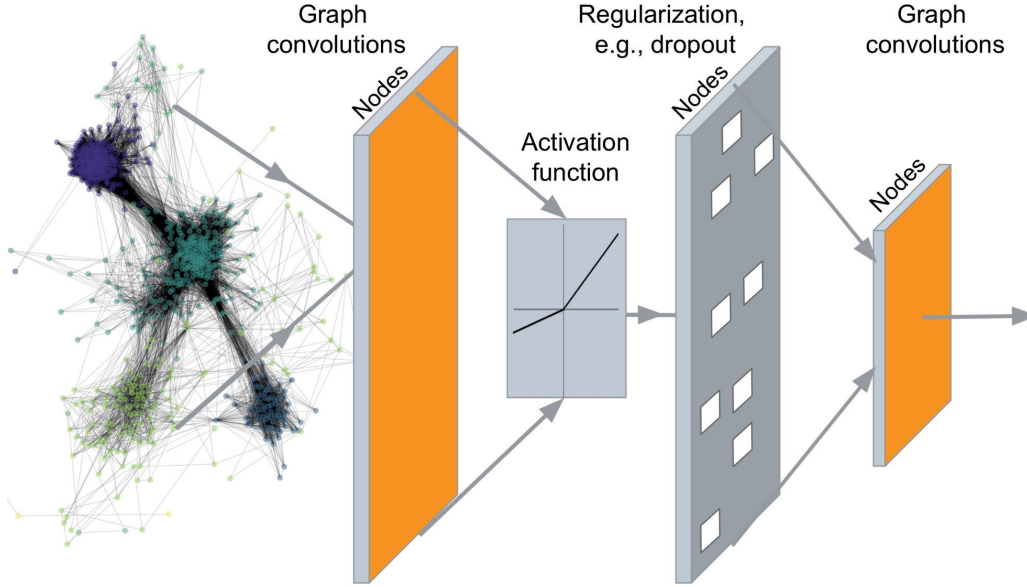


Figure 5: Basic architecture of a Graph-Neural-Network, based on slides from [33, Deck 3, Slide 9]

5 shows a basic example of a GNN with a convolutional layer, activation function, a dropout layer and another convolutional layer.

Several types of convolutional layers can be used to aggregate and update information such as GCNConv [32], GraphSAGE [25] or GATs [62]. The extension model in Chapter 6 uses GraphSAGE in its model architecture.

GNN Types. There exist various types of architectures in GNNs, the most prominent ones and the one used in this thesis will be introduced shortly. To this end, the work of Wu et al. [69] broadly defines four different types of GNNs, which are *Recurrent-Graph-Neural-Networks (RecGNNs)*, *Convolutional-Graph-Neural-Networks (ConvGNNs)*, *Graph Autoencoders (GAEs)* and *Spatial-Temporal-Graph-Neural-Networks (STGNNs)*.

RecGNNs are depicted as the pioneering GNN models. RecGNNs use recurrent neural architectures to learn node representations. They operate by iteratively exchanging information between nodes and their neighbors until reaching a stable state. RecGNNs introduced the concept of message passing, which influenced the development of ConvGNNs.

ConvGNNs models generate node representations by aggregating features from the node itself and its neighbors. The main difference to RecGNNs is that ConvGNNs use multiple graph convolutional layers to extract high-level representations of nodes. They form the basis for more advanced GNN models.

GAEs encode nodes or full graphs into latent vectors and reconstruct graph data from those vectors. They focus on learning node representations by reconstructing graph structures like adjacency matrices. This kind of architecture is used in the extension model in Chapter 6.

Lastly, STGNN models learn hidden patterns from spatio-temporal graphs by capturing spatial dependencies with graph convolutions and temporal dependencies with RNNs or CNNs. STGNNs are used in applications like traffic forecasting and human action recognition.

Types and Use cases of GNN tasks. GNN tasks can be categorized into three types of problems that they can solve, namely *node classification*, *link prediction* and *graph classification*. Node classification is the process of predicting the class of an unlabeled node based on its features. Use cases of node classification include biology, social network research and recommendation systems. Link prediction on the other hand is the task of predicting if an edge (i.e. a relation) between two nodes exists or not. The model is trained to learn the connections between all node pairs in the graph. After the training, the weighted model can make predictions if a connection between a specific node pair exists or not. This process is used by the extension model in Chapter 6. Graph classification is a task that tries to solve the problem of categorizing complete graphs, in contrast to individual nodes or edges. The model is trained to predict the correct label of a full graph based on its node and edge features. A common application for this is the prediction of molecular properties in chemistry and medicine. [2].

Other applications of GNNs include computer vision, natural language processing, traffic prediction and recommender models. [69].

2.4 Replication Studies

First and foremost, it is important to declare the terms replication and reproduction in order to establish a clear understanding of these terms. In practice, these expressions are often confused in the scientific discussion [6]. Both concepts are essential elements for the scientific process but serve different purposes. Therefore, this study provides a clear overview of both concepts. Barba [4] provides a definition for the expressions *reproducing* and *replicating*. The act of *reproducing* is defined as using the same software and same input data to verify and obtain the results in order to verify the correctness of the study. On the other hand *replicating* is considered to build on the foundation of an already published study and using potentially new software and models to derive results. These replicated results should still be similar enough to the original study results to be comparable to them.

This thesis does not fully fall in either category defined by Barba [4], since

the goal was to verify the original MRecuri results with the same code and same data, but due to missing artifacts, this was not possible. New methods and data had to be employed at certain steps, which puts this work more into the direction of a *replication* while still wanting to *reproduce* and verify the original results. Hence, in the remainder of this thesis, the term *replication*, will be used for any efforts to verify the original results.

The importance of the concept of replication is highlighted in different studies. Moonesinghe et al. [37] emphasize that replication is a cornerstone of the scientific process in generalizing findings. Furthermore, it is highlighted that the replication process in science is a fundamental concept for generating established pieces of knowledge [10].

Bellogín and Said [6] focus on the difficulties in replicating recommendation system research in their work. The study demonstrates that different model parameters and changes in the implementation or evaluation metrics can lead to dissimilar results in the replication studies. Beel et al. [5] support these findings. The authors emphasize that minor changes in the replication of recommendation system research can lead to major changes in the outcomes. The authors further conclude, that significant challenges exist in replicating recommendation system research.

Moreover, it is important to understand to what extent a replication study can vary from the original research project. To clarify this question, this study uses the classifications from Gomez et al. [22], which classifies replication studies into three groups of alignment with the original research:

- Group 1: The replication closely follows the original experiment and varies almost to no extent from the original work. This type of replication tries to investigate if the results are only based on sampling errors of the data.
- Group 2: In these replication projects, the configurations of the original research are changed and modified. Both projects contain the main functionalities, but essential experimental elements are altered.
- Group 3: The last group includes replication studies that only share the same theoretical concepts. Both studies for example share a set of theoretical concepts and hypotheses, that are tested in the experiment. The implementation however can vary to various degrees.

It should be added that groups 2 and 3 are not perfectly separable. Both of these groups lay on a continuum, where it is difficult to differentiate between the groups.

The replication part of this study can be classified into group 1 of the above classification. This research project used the data sources and code artifacts given in the replication package of the original research. Note, that not all data sources are available in the original research project and thus some data needs to be re-collected in this thesis. However, the fundamental functionality and the experimental design remains unmodified to a great extent.

3 Related Work

The related work section introduces established models in the context of GNN for music recommendation models and recommender models for diversity and novelty. First, specific GNN-based applications of music recommender systems are elaborated on. Second, related work that focuses on employing novelty and diversity in GNN-based recommender systems is laid out. Lastly, an overview of replication studies of music recommender systems is given.

3.1 Music Recommender Systems Using Graph Neural Networks

The work of [70] Knowledge Aggregation of Importance Sampling Based on GNN for Music Recommendation (KAISG) addresses the challenge of finding relevant music in vast information pools. This model leverages GNNs to capture user preferences through knowledge aggregation of importance sampling. They construct an item knowledge-graph to capture correlations between items by mining their related attributes. The model employs importance sampling based aggregation of neighborhood information to avoid noise introduction in high-order relationships, thus more accurately aggregating information related to the current node.

The Graph based Attentive Sequential Model With Metadata (GASM) [67] tackles the difficulties posed by massive music data and diverse listening behaviors in user-personalized recommendation scenarios. GASM incorporates metadata, such as singer and album information, to enrich music representations and mines user listening behavior patterns. The model uses a directed listening graph to model relations between the various types of nodes (user, music, singer, album) and adopts GNNs to learn their latent representation vectors. Lastly, GASM decomposes user preferences into long-term, short-term, and dynamic components using personalized attention networks, integrating these preferences to predict new music for the users.

In the work of [64], their proposed Multi-view Enhanced Graph Attention Network (MEGAN) model addresses the limitation of traditional music recommender systems by incorporating rich auxiliary information, such as metadata and song descriptions. MEGAN learns informative representations of music pieces and users from heterogeneous information using a GNN and attention mechanism. The model first constructs a Heterogeneous Music Graph (HMG) to model the users’ listening behaviors and textual content of songs. It then employs a Graph Attention Network to learn low-dimensional embeddings of songs and users by integrating various auxiliary information from the HMG.

The Heterogeneous Information Graph Embedding (HIGE) model [63] tackles the problem of personalized music recommendation by incorporating different kinds of auxiliary information. HIGE constructs a Heterogeneous Information Graph (HIG) to encode various types of heterogeneous information, including user-music interactions, music playing sequences, and music metadata. Their model learns latent low-dimensional representations of songs based on the HIG and develops a context-aware music recommendation approach. HIGE demonstrates significant performance improvements over baseline methods, especially on sparse datasets.

The study [34] proposes a Representation-enhanced Knowledge Graph Convolutional Networks (RKGCN) model which is an end-to-end deep learning model that dynamically analyzes user preferences and recommends suitable items. This model is not uniquely made for music recommendations but also book and movie recommendations. The model combines knowledge graphs on both the item and user sides to enrich their representations and maximizes the utilization of information in knowledge graphs.

In the work of [26], an optimized Light Graph Convolution Network (LightGCN) model is used for music recommendations. The study focuses on improving the performance of the LightGCN, which is a simplified version of Graph Convolution Network (GCN), by reducing the complexity of the model for collaborative filtering in recommendation systems. The model is optimized by tuning hyperparameters with a grid search. The optimized LightGCN model demonstrates significant performance improvements, outperforming the original LightGCN by more than 140% in music recommendation tasks.

The proposed extension model in this thesis, GMR-ND, distinguishes itself from the mentioned related works through its dual embedding strategy, which simultaneously processes both structural graph information and content-based features in parallel convolutional layers. Unlike the temporal decomposition used in GASM or the separate knowledge graphs in RKGCN, this architecture integrates all entities (users, songs, and artists) and their

relational information into a single heterogeneous graph structure. This approach allows for direct information flow between different node types while preserving the ability to learn both structural relationships and content-based similarities. Furthermore, social relationships between users form an essential part of the proposed model, which are disregarded in the related works.

3.2 Recommender Systems for Novelty and Diversity

This subsection covers the related work on GNN-based recommender systems that introduce more novel and diverse recommendations and potentially help users to escape filter bubbles.

One related study in this field is the research by Tommasel et al. [59], which introduces the FRediECH (Friend RecommenDer for breakIng Echo CHambers) algorithm. This model recommends friends to users on social media to break users’ echo chambers. In this case, echo chambers are social media communities where users mostly see content that reinforces their existing views. The FRediECH algorithm addresses this problem and suggests friends outside a user’s community to offer more diverse opinions. Furthermore, the FRediECH algorithm is one of the origins of the MRecuri recommendation system. Both algorithms were developed by the same group of authors. By comparing both studies, it can be seen that the methods share several experimental practices and both use a GCN to utilize the social relationship data in the network. Moreover, the methods apply a Deep-and-Wide architecture to handle unseen interactions and capture implicit user characteristics. Therefore, the FRediECH algorithm can be viewed as the foundation of the MRecuri.

The study by Sun et al. [56] proposes a Bayesian Graph Collaborative Filtering (BGCF) framework, a recommendation system utilizing Bayesian Graph Convolutional Neural Networks (GCNs). Unlike many traditional models, BGCF recognizes the incompleteness of the test data by treating the interaction graph as a random variable. This allows for probabilistic modeling of unobserved or uncertain interactions, enhancing the system’s ability to make inferences where data is sparse. Central to BGCF is a node-copying mechanism, namely a graph-generative model that increases recommendation diversity by generating multiple graph versions. Each graph, created by probabilistically copying nodes based on neighborhood similarity, reflects potential variations in the user-item interactions. By varying the different graph representations, the model can contain potential user-item interactions, that are not included in the dataset. BGCF is evaluated in experiment against leading models on the Amazon-CDs, Amazon-Movies, and Amazon-Beauty datasets. The results showed that BGCF demonstrated superior performance

in accuracy and diversity. It proved especially effective for new or infrequent users.

A different approach for introducing diversity in graph recommendation systems is presented by Guo et al. [57]. The authors introduce a Deep Graph Convolutional Network with Hybrid Normalization (DGCN-HN), that aims to balance the accuracy-diversity trade-off. The DGCN-HN employs a hybrid normalization method that adapts the normalization technique, based on the node’s position in the graph. The two normalization techniques are symmetric and left normalization to balance between popular and niche preferences. The symmetric normalization gives less weight to popular items, whereas left normalization treats all neighbors equally, irrespective of their popularity. These strategies allow the algorithm to improve recommendation diversity while maintaining the accuracy of the results. The findings of the study show, that the DGCN-HN method outperforms other recommendation models in accuracy and diversity metrics. The model shows significant improvements compared to the benchmark algorithm, especially on datasets with sparse interactions.

Lastly, another study that tries to mitigate the effect of filter bubbles by a different approach is the work of Anand et al. [3]. The study proposes a method that tries to identify user-item relationships that cause filter bubbles. In the first step of the model, the algorithm identifies communities of items, based on the Louvain algorithm and the user-item interaction patterns. In a second step, a TracIn method is used, that evaluates how different categories influence each other. Additionally, it identifies which interactions are likely to cause filter bubbles. The core idea of the model involves selectively re-training the recommendation system with a modified training set that either incorporates additional diverse data points or excludes those contributing to filter bubbles. By continuously adjusting the training data, the algorithm aims to produce a more balanced set of recommendations that introduce users to a broader range of content. The study tests its method system on a Long-Short-Term-Memory-based model as the recommender system. The empirical results indicate that the proposed method successfully increases the diversity of recommendations and mitigates the effect of filter bubbles.

The extension model of this thesis builds on the principles of FRediECH and MRecuri and leverages both structural and content-based embeddings within a GNN framework to naturally create diverse recommendations. This architectural choice allows the model to maintain a balance between relevance and diversity by addressing the filter bubble problem through the exploration of the graph structure.

3.3 Replication in Music Recommendation

To the best of our knowledge, by searching through studies that cited the work of [60], no replication studies of the original paper have been performed, hence this study will provide the first replication of the original MRecuri model.

Furthermore, to the best of our knowledge, no studies that replicate published scientific music recommendation systems have been found. The keywords to identify published replication studies were *music AND recommendation OR music AND recommender AND replication OR replications OR replication study*, in the Article title, Abstract and Keywords on the platform *Scopus*. While these search terms concluded 5 results, none of these results were actually replication studies in the context of music recommendation systems.

On the other hand, there are studies that investigate the reproducible aspect of recommender systems, without replicating any study in particular. The work of [6], focuses on improving the accountability of recommender systems by suggesting a framework for better reproducibility. In the study, they identify problems of recommender systems in terms of reproducibility which can be summarized as implementation variations, evaluation inconsistencies and documentation gaps. Their findings show, that there is a growing awareness about reproducibility issues in the field, but many ongoing challenges remain. Two major issues are, for example, that a perfect reproduction is often not possible due to incomplete implementation details, even though the code artifacts were made available by the original study. Furthermore, inquiries from the replication authors to authors of the original work often go unanswered, making the reproduction process more difficult.

The work of Ferrari Dacrema et al. [17] shows similar results. Their study questions the progress and reproducibility in recommender systems where the authors analyzed a total of 26 papers from top ranking scientific conferences. In their efforts to replicate said works, they were only able to replicate 12 of them with reasonable effort. The biggest challenges in reproducing the works were missing or incomplete source code and code artifacts and the issue of collaborating with the original study authors. Only 4 of 14 study authors answered to their replies for collaboration on the replication.

Overall these findings match well with the experiences the author of this thesis has made during the reproduction of the MRecuri study. In the discussion Chapter 10 these alignments will be discussed further.

4 Original Study - MRecuri Model

This chapter will present the work of "*Haven't I just Listened to This?: Exploring Diversity in Music Recommendations*" [60] by Antonela Tommasel, Juan Manuel Rodriguez and Daniela Godoy. The study proposes an implementation of a hybrid music recommender system that enhances diversity and novelty. The referenced research paper will form the basis of the majority of this thesis and will, therefore, be elaborated on in detail in this chapter.

The *MRecuri* model is a music recommender system with the aim to break users free from potential filter bubbles in their listening habits while still providing engaging and interesting song recommendations. The original paper refers to [18], which claims that recommender systems often prioritize click rates and sales over content diversification and, therefore, may make user's content consumption less open and culturally diverse. Therefore *MRecuri*, a *Music RECommender for filter bUBble diveRsIfication*, is introduced with the goal to provide users with a broader listening experience overall.

While other research has been made in this area, MRecuri authors claim that most of the previous research [61, 14] have focused on only the diversity metric. Therefore, MRecuri is claimed to stand out by taking into account both the diversity and the novelty of recommendations and, furthermore, to consider user context and social relations by adding social connections to the model parameters. In this regard, community structures and potential content bubbles are implicitly derived.

4.1 Data

The data used in the study is gathered from *Last.Fm*³. Last.FM provides user data on listening behavior and social relations to other users. In this way, the listening history and, furthermore, the crucial social friend connections to other users could be derived for this study. The social interactions between users were collected based on the work of Zhang et al. [72]. After pre-processing, a selection of the top 5% of users that held the highest scrobbles and social connections remained. For the final selected songs the song tags, the tags of the artist of the song and the Spotify audio features were collected. The final data collection statistics used for the original model are 3,307 users, 252,014 tracks and 28,540 artists, as depicted in Table 4.

³Access to the LastFM API: <https://www.last.fm/api>, last accessed 14.02.2025

#Users 3,307	#Tracks 252,014	#Artists 28,540
	avg ($\pm$ std)	25% - 50% - 75% quantiles
Tracks per User	912 ($\pm$ 1266)	624 - 854 - 1043
Scrobbles per User	48 ($\pm$ 73)	11 - 28 - 58
Social relations per User	86 ($\pm$ 85)	29 - 59 - 118

Table 4: Data collection details [60, p. 38]

4.2 Model Architecture

As mentioned before, the MRecuri model focuses on learning filter bubble characteristics from user’s social relationships and content interactions implicitly. The model’s basic functionality is similar to another research paper by the same authors [59] in the field of friend recommendations in social networks. The model aims to help users in social networks to break out of their echo chambers through their proposed FRediECH recommendation system that takes into account shared content, user interaction and communities to recommend novel and diverse friends.

To this end, MRecuri’s architecture takes features from the FRediECH model and is comprised of the following elements as depicted in the schematic model architecture in Figure 6. The general architecture is a so called *wide-and-deep architecture* with the deep part focusing on generalization and the wide part learning implicit features. Furthermore, the model’s input can be distinguished broadly between user’s listening history, user’s social interactions and track input features. Lastly, the combination of the wide and deep part lead to the final predictions on which songs to recommend to a user.

4.2.1 Deep Part

User Representation. The deep part of the model includes a user-graph and user embeddings which are fed to a Graph-Convolutional-Network (GCN) and produce the user representation of the model. The user-graph is the graph input for the GCN and consists of user nodes with relations to other users. It is derived from the social connections from the LastFM dataset. On the other hand, the user embeddings represent the historical interactions between users and songs. Therefore, the GCN aims to take into account not just the user’s own behavior and song interactions, but additionally attempts to capture social interactions.

To construct the aforementioned GCN, the original authors use the *Spectral* library in Python to construct a GCN. The GCN produces latent feature embeddings of users which are designed to capture both their individual lis-

tening behavior and the influence of their social interactions. Equation 2 describes the GCN model in MRecuri where σ is the linear activation function and A represents the adjacency matrix of the user's social interactions. In that regard, $\hat{D}$ is a diagonal matrix with $\hat{D}_{ii} = \sum_j \hat{A}_{ij}$, used to normalize $\hat{A}$. Lastly, X represents the user embeddings input, W is the trainable weight matrix and b is a trainable bias vector.

$$\text{GCN} = \sigma \left(\hat{D}^{-\frac{1}{2}} \hat{A} \hat{D}^{-\frac{1}{2}} XW + b \right) \quad (2)$$

Track representation. On the other hand, the track representation is constructed by concatenating various track related inputs. These consist of tag embeddings, artist embeddings, Spotify audio features and track embeddings. Tag embeddings are various tags like "Rock" or "Pop" that are averaged per track. Note that tags are created by users on LastFM and some songs also might have no tag data. The Spotify audio features consist of eleven features that are *energy*, *instrumentalness*, *liveness*, *speechyness*, *acousticness*, *danceability*, *valence*, *mode*, *key*, *loudness* and *tempo*.

After the separate concatenation of the respective user and track representation, these representations are combined and then run through three dense layers that have linear activation functions. This yields the output of the deep model part.

4.2.2 Wide Part

The wide part of the model is less complex than the deep part. Here, all the embeddings of user, tag, artist, track and Spotify audio features are concatenated and passed through one dense layer. This yields the output of the wide model part.

Finally, the wide and deep part outputs are combined to make predictions on a user-track pair. The prediction strength made in this context can be seen as an estimation of how many times a user might listen to the predicted track (i.e. the predicted number of scrobbles).

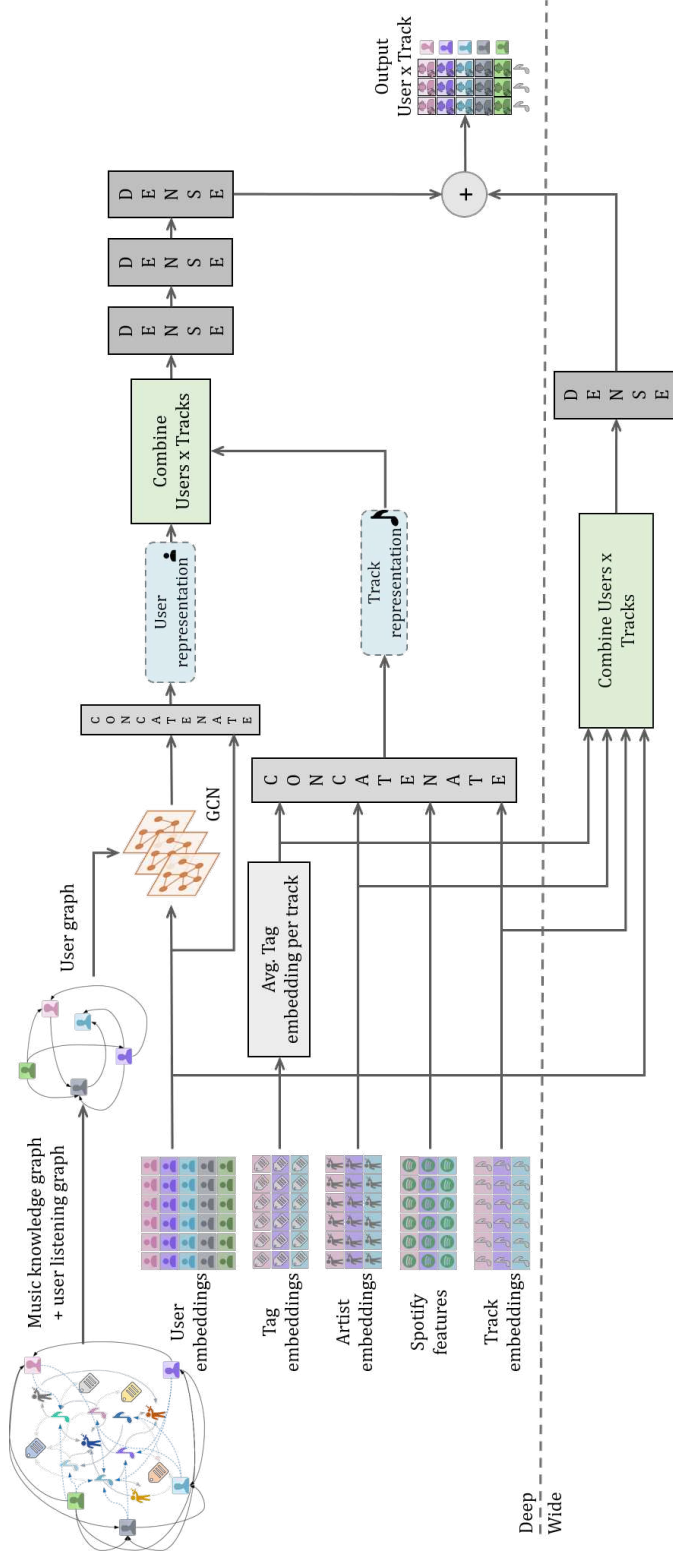


Figure 6: Schematic diagram of MRecuri [60, p. 37]

4.3 Model Training and Loss Function

Loss function Generally, a loss function in a neural network model is designed to minimize the distance between the actual and predicted interactions between users and tracks in the training process and helps to tune the model weights accordingly in the back propagation process. In this case, the training of the MRecuri model uses a custom loss function that aims to increase the models diversity by implicitly learning potential filter bubble structures. The loss function is defined in Equation 3 and consists of the following elements:

- $\hat{Y}_{ut}$ as the predicted number of scrobbles for the user u and track t
- Y_{ut} as the actual number of scrobbles for user u and track t
- $d(u, t)$ as a pre-computed distance between users and songs in the form of cosine similarity distances
- β as a hyperparameter to influence the weight of the precomputed distances
- $|E|$ as the total number of predictions

$$L(Y, \hat{Y}) = \frac{\sum d(u, t)^\beta \left(\hat{Y}_{ut} - \log_2(2Y_{ut}) \right)^2}{|E|} \quad (3)$$

The pre-computed parameter $d(u, t)$ aims to help reduce the impact of already listened-to songs and favors songs from outside a user’s filter bubble by giving more weight to songs that have a higher distance to the user. With this approach the model implicitly increases diversity and reduces the influences of filter bubbles. As such, β is a weight factor for the distances between 0-1 (where 0 leads to learning closer community preferences and 1 means learning more diverse recommendations) and is set to 0.75 based on unspecific evaluations made by the authors.

Apart from *scrobbles*, which influence the model loss function through Y_{ut} and can be viewed as *positive interactions* between users and songs, *negative sampling* is applied in a 1:1 ratio to positive interactions.

Furthermore, the use of $\log_2(2Y_{ut})$ helps to scale the scrobble values and prevents exploding gradients, aiding the learning process. Lastly, a dropout of 10% is used in the GCN training to help with overfitting.

Pre-computed distances The previously mentioned pre-computed distances $d(u, t)$ between users and songs are calculated separately from the loss function. This is done in order to avoid dependencies between the loss function in the model and said distances. The distance metric is an adapted cosine distance between the user and track embeddings while the respective embeddings are the output of a Word-2-Vector (W2Vec) model [36] that categorizes if a user has listened to a song at all or if no relation exists. The W2Vec models uses a binary cross-entropy loss function with user and track embeddings, a sigmoid activation function and negative sampling in a 1:1 ratio. A_{ui} signals if a user has listened to a track, and is set to 1 if the user listened to a track and is set to 0 if no listening relation exists, while w_u and $\hat{w}_t$ are the user and track embeddings. Equation 4 depicts the stated loss function.

$$L_{dis} = \frac{-\sum_i (A_{ui} \log(\sigma(w_u \cdot \hat{w}_t)) + (1 - A_{ui}) \log(1 - \sigma(w_u \cdot \hat{w}_t)))}{|N|} \quad (4)$$

From these user and track embeddings created by the W2Vec model an adapted version of the cosine distance between the embeddings is derived as depicted in Equation 5. Finally, the raw distances were clipped according to Equation 6.

$$d_{raw}(u_i, u_j) = 1 - \frac{\cos(w_{u_i}, \hat{w}_t) - (\mu_{cos} - 2\sigma_{cos})}{4\sigma_{cos}} \quad (5)$$

$$d(u, t) = \begin{cases} 0.1 & \text{if } d_{raw}(u, t) < 0.1 \\ 0.9 & \text{if } d_{raw}(u, t) > 0.9 \\ d_{raw}(u, t) & \text{otherwise} \end{cases} \quad (6)$$

In this way, the mean and standard deviation of the cosines, μ_{cos} and σ_{cos} , are used to create an adapted cosine distance, which aims to favor tracks that are further away from users and hence would normally have a weaker relationship than closer songs.

Experimental Setting The MRecuri model was implemented with TensorFlow (TF) *Spektral* as the GCN module. Note that the information on which versions of Python, TF or any other packages were used is not available. As an optimizer *Adam* was used together with a learning rate of 0.003 during training. The model was trained over 10 epochs, after which no more major loss changes occurred according to the authors. The hardware used for the training process was an *Asus Predator Helios 300* with 16Gb RAM,

an *i7-11800H* CPU, and an *NVidia GeForce RTX 3060 6Gb* GPU. The full training and inference code took around six hours and fifteen minutes to execute fully. Note, that this were the hardware specifications used in the original study, this thesis uses hardware as described in Chapter 1.4.

4.4 Evaluation Metrics and Benchmark Models

The predictions were evaluated in an offline environment with a temporal split that depended on the scrobble data. This means that for each user their listened tracks had a specific order given by their *time* attribute. This time attribute was used to split the data temporally with 70% belonging to the training data and 30% designated to the test set data for evaluation purposes. Furthermore, the final evaluation was trimmed to a set of *top-K* recommend tracks, where k was set to 10, as 96% of all users in the dataset had at least 10 tracks in their test data.

Evaluation Metrics The evaluation was compromised of 4 metrics, namely precision, nDCG, novelty and diversity. For a detailed explanation of precision and nDCG refer to Chapter 7.3. Novelty and diversity are further separated into *structural* and *content based dissimilarity*. While precision and nDCG are relevance based metrics, diversity and novelty were introduced to measure how well the MRecuri model was able to reduce the filter bubble effect in comparison to other models.

Novelty. Novelty was defined as the comparison of past track interactions (training set tracks) and new track interactions (recommended tracks). By computing an embedding for each track, the past and newly recommended track embeddings could be compared and a novelty score was computed.

Diversity. Diversity, on the other hand, was derived by comparing the recommended tracks between each other. Diversity is a type of *intra-list dissimilarity* [58]. Hence, for the diversity score it was necessary to compare the track embeddings of the recommended tracks between each other.

Embeddings The embeddings that are necessary for computing the respective novelty and diversity scores were separated into two distinct types, namely structural and content based embeddings.

Structural. Structural embeddings are computed by defining a vector over the consumption rate of users [23]. The computation of the structural embedding vector was conducted as follows. First, a co-listening graph was created, which only holds track nodes. Two track nodes in this co-listening graph had a connection (i.e. an edge) if one user listened to both of these

tracks. Over this co-listening graph a community detection algorithm, in this case the Louvain algorithm [8], was run to identify structural communities. Finally, the embedding vectors were derived for each user by looking at the entire listening history of each user. Each user had interacted with a specific number of songs, and each of these songs was part of a community as defined by the Louvain algorithm. With this, the *consumption rate* was defined by the number of songs that each user had listened to (i.e. consumed) from each specific community.

Content based. The content based embeddings were derived from a W2Vec embedding over the tags of each track. This way, every track held a number of tags in the form of a vector after the W2Vec transformation. Then, for each song, the average over all their tags was taken to form the song embedding. Finally, these song embeddings were again averaged over all song embeddings of one user to create this user’s embedding.

This resulted in the creation of structural and content based user embeddings which were used for the novelty and diversity calculation.

Note that the metrics, especially novelty and diversity as well as how their embeddings were derived are only described textually by the original study. The source code in the replication package does not contain information on the implementation details of the novelty or diversity metrics. Therefore, a more in-depth explanation of the replicated implementation of the evaluation metrics follows in the replication Chapter 5.

Benchmark Models MRecuri’s performance was evaluated against 9 baseline recommendation systems based on the previously mentioned metrics. In this way, the results of MRecuri could be validated and compared to lower bounds like the Random Model and state-of-the-art models like ImplicitFM and GraphRec. Note, however, that the original MRecuri study makes only textual descriptions of these models and no code implementations are provided in the original replication package.

1. **Popularity based Recommender:** Suggests tracks based on their scrobble-to-listener ratio, serving as a non-personalized baseline. Tracks were ordered according to their scrobbles and listener numbers and were recommended in descending order of their popularity scores.
2. **Random Recommender:** A random model, selecting ten random songs from the test set for each user. Functions as a lower bound reference, providing non-personalized, random suggestions.
3. **Content based Approach:** Utilizes Word2Vec embeddings of track tags. User profiles are constructed as centroids of their listened tracks’

vectors, with recommendations based on cosine similarity between track and user representations. The closer the cosine similarity is between user profile and song embedding, the higher the recommendation scores. Note that this specific content based approach was defined by Tommasel et al. [60] and not a previously existing model.

4. **ImplicitMF**: A matrix factorization technique optimized for implicit feedback scenarios, known for high performance in user-item recommendation tasks [28].
5. **GraphRec**: A graph neural network model that integrates social interactions, item characteristics, and user ratings in its recommendation process [16].
6. **MultVAE**: A neural generative model founded on variational autoencoders and multinomial conditional likelihood [35].
7. **Rank Aggregation (RA)**: A diversity-oriented technique which combines ImplicitMF rankings with diversity and novelty scores derived from content based representations.
8. **MMR-inspired Technique**: Based on the Maximal Marginal Relevance concept [11], this method utilizes ImplicitMF as its foundation, with diversity measured through cosine similarity of content based item representations [15].
9. **VC Model**: An adaptation of the Vargas and Castells approach [61], using the top-20 tags for profile construction and ImplicitMF for base recommendations.

All models were configured and optimized according to their original publications' guidelines according to the MRecuri authors.

4.5 Results

The results, as depicted in Table 5, give an overview of the evaluation benchmarks of the MRecuri model and the benchmark models. Note that the last row in the table, the *original user-track graph*⁴ is a baseline of the novelty and diversity scores between the original train and test split graphs. The authors described their results as follows.

⁴the terms original user-track graph and original graph are used interchangeably in this thesis

The analysis revealed significant variations in performance across the evaluated recommender systems. Random and GraphRec demonstrated the lowest relevance scores on precision and nDCG. Conversely, these models excelled in diversity and novelty metrics, with GraphRec ranking among the top performers in these metrics. Furthermore, a trade-off between relevance and diversity/novelty can be seen in the results data. Models that achieved a high relevance typically scored lower on diversity and novelty measures.

In terms of relevance, ImplicitMF and MultVAE stood out, achieving the highest precision and nDCG scores. However, these models underperformed in diversity and novelty compared to the original graph, further highlighting the relevance-diversity trade-off. Among the diversity-oriented techniques, MMR achieved the highest relevance scores, while Rank Aggregation (RA) excelled in diversity and novelty. However, MRecuri outperformed these specialized techniques in all aspects except structural diversity.

MRecuri demonstrated a good performance, ranking among the best in most metrics. It achieved the highest novelty scores and showed significant improvements in precision and nDCG, with average increases of 56% and 19% respectively. MRecuri also improved upon the diversity and novelty scores of the original graph in all areas except structural diversity. Compared to simpler baselines (Random and Popularity), state-of-the-art models (ImplicitFM, GraphRec and MultVAE) and diversity-oriented techniques (RA, MMR, VC), MRecuri showed average improvements in diversity/novelty of 25%, 9%, and 15% respectively. Across all models, novelty scores generally surpassed diversity scores, suggesting that even when recommending similar tracks, the recommendations differed notably from users' listening histories. MRecuri's performance in structural metrics scored lowest in structural diversity, indicating recommendations of tracks with similar co-listening patterns, but it achieved the highest structural novelty. This suggests that MRecuri's recommendations, while similar in listening patterns, came from outside the user's typical co-listened community, potentially broadening users' musical horizons.

	Precision	nDCG	Content based dissimilarities		Structural dissimilarities	
			Diversity	Novelty	Diversity	Novelty
MRecuri	0.28 ± 0.198	0.648 ± 0.227	0.471 ± 0.014	0.547 ± 0.031	0.128 ± 0.009	0.387 ± 0.092
Random	$0.101^* \pm 0.004$	$0.446^* \pm 0.201$	0.496 ± 0.028	0.485 ± 0.023	0.224 ± 0.075	$0.282^* \pm 0.045$
Popularity	0.203 ± 0.133	0.581 ± 0.237	$0.402^* \pm 0.017$	$0.46^* \pm 0.026$	0.131 ± 0.015	$0.279^* \pm 0.031$
Content based	$0.168^* \pm 0.124$	0.522 ± 0.215	$0.173^* \pm 0.062$	$0.388^* \pm 0.046$	0.219 ± 0.061	$0.247^* \pm 0.033$
ImplicitMF	0.366 ± 0.238	0.699 ± 0.226	$0.376^* \pm 0.061$	$0.417^* \pm 0.042$	0.146 ± 0.049	$0.228^* \pm 0.036$
GraphRec	$0.1^* \pm 0.03$	$0.391^* \pm 0.149$	0.497 ± 0.011	0.542 ± 0.012	0.325 ± 0.007	$0.317^* \pm 0.022$
MultVAE	0.348 ± 0.245	0.686 ± 0.231	$0.414^* \pm 0.07$	$0.457^* \pm 0.049$	0.176 ± 0.061	$0.236^* \pm 0.039$
Rank Aggregation	0.241 ± 0.164	0.57 ± 0.214	$0.453^* \pm 0.118$	0.531 ± 0.043	0.179 ± 0.054	$0.238^* \pm 0.038$
MMR-inspired	0.247 ± 0.172	0.687 ± 0.245	$0.41^* \pm 0.152$	0.515 ± 0.04	0.176 ± 0.054	$0.236^* \pm 0.037$
VC	0.288 ± 0.189	0.6 ± 0.216	$0.421^* \pm 0.058$	$0.469^* \pm 0.041$	0.195 ± 0.055	$0.243^* \pm 0.037$
Original user-track graph	-	-	0.472 ± 0.066	0.484 ± 0.047	0.229 ± 0.045	0.246 ± 0.031

Table 5: Recommendation results comparison for $k = 10$. (* indicates statistically significant differences favouring MRecuri)

5 Replication

The following chapter will provide a detailed description of how the original study was replicated. It will focus on the replication of the source-code implementation of the original MRecuri model as outlined in Chapter 4, in order to replicate and verify the results of the original study.

To clearly differentiate data and code, that refers to either the original study or the modified version of it by the author of this thesis, two definitions are introduced:

- **Original Replication Package (ORP):** This package comprises the data and source code as provided by the authors of the original study. It serves as the baseline for the replication and subsequent model extension.
- **Adapted Replication Package (ARP):** Recognizing the limitations and compatibility issues present in the ORP, this package contains the modified and updated version of the original source code. The ARP ensures that the replication is executable within the software specifications given in this thesis. Furthermore, the ARP package also includes the extension model code and artifacts of the proposed Graph-Neural-Network model. The novel approach incorporates new methodologies aimed at assessing the performance of this new approach in terms of the previously described evaluation metrics and compares certain model features against each other. A detailed description can be found in Chapter 5.5.

These definitions will be consistently used throughout the thesis to delineate between the original materials, the replication efforts, and the extension efforts introduced in this research.

The structure of this chapter is as follows. First, the data inspection and processing of the provided data by the ORP is laid out. Consequently, the process of scraping any missing data that is necessary for the code execution is elaborated on. Second, the provided code from the ORP is examined in detail and its functionality is introduced. To this extent, the modifications that were necessary to execute the original source code are explained, therefore, outlining the significant differences between the ORP and the modified ARP. Third, the structure of the ARP package and how to access it is described. Fourth, insight into the replication of the embeddings, which are necessary to compute the novelty and diversity metrics, is given. Last, the efforts of reaching out the authors of the original study [60], that were made to receive potential answers to unclear matters at the time of replication.

5.1 Data Inspection, Pre-processing and Collection

5.1.1 Original Provided Data

The original data was provided in the ORP in the form of *.pkl* files and contained user-track graph data, user-user graph data, track mapping data and user mapping data. Graph files were given in the format of NetworkX⁵ [24] graphs, while mappings were given as dictionaries.

User-track knowledge graphs. The user-track knowledge-graphs depict user-track relations and were separated according to their training and test splits as outlined in Chapter 4.4. To this end, the provided graph files were a full graph, a training graph and a test graph.

The two node types *user node* and *track node* exist in the given graphs. Note that the indexing of all NetworkX nodes is consecutive and node types are only differentiated by the node attribute. This means that user nodes have the IDs from 0-3307 and song nodes have IDs from 3307 - 255321. This is an important difference to how indexing works in the novel approach in Chapter 6.

The edges in the given graph data are undirected edges between *user* and *track* nodes with the following attributes. The *date* attribute refers to the last date when the user listened to a specific track. The *pos* (position) attribute, refers to the order of the listening history of the user and is semantically redundant with the *date* attribute. The *scrobbles* attribute refers to the number of times that a user listened to a specific track.

User-user graph. This graph represents the social relations between the users. The graph was derived from the ORP data given as *lastfm.nodes* and *lastfm.edges* which hold user nodes and the user-user relations between these nodes.

User-user mapping. Represents the mapping between user ids in the graph nodes and user names in the form of a dictionary *username : user id in the graph*.

Artist-track-mapping. The mapping between artists, track names and track node data in the graph can be found in the provided pickle file by accessing '*artist-tacks*'.

5.1.2 Data Processing

This subchapter will elaborate on how the provided data from the ORP was inspected by the author. Furthermore, it will elaborate on inconsistencies that have been found between the inspection and the information provided

⁵<https://networkx.org/>, last accessed 14.02.2025

in the text of the original paper. Consequently, it will cover how these inconsistency issues have been resolved to provide a clean data starting point to replicate the original study. The functionality referred to in this chapter can be found in the supplement Notebook 1. *Data Inspection and Preprocessing.ipynb* of the ARP replication package.

User-track graph. Through examination of the NetworkX data of the respective user-track graphs (train, test and full graph) provided by the ORP, the following descriptive statistics depicted in Table 6 were derived. Note that the train graph and test graph hold a different number of nodes and edges compared to full graph, due to their temporal split, which will be examined in more detail below. Closeness Centrality was only calculated on the user nodes, due to high computational costs when calculating Closeness Centrality on all nodes in the full graph. Similarly, Betweenness Centrality was calculated with an approximation of 100 node samples for the same reason.

User-user social graph. Similarly, an examination of the user-user graph based on *lastfm.nodes* and *lastfm.edges* was made and is depicted in Table 7.

Training and Testing The original full graph, as referenced in Table 6, holds a total of 3,307 user nodes and 252,014 track nodes, with a total of 3,018,209 edges between those two nodes types. As described textually, by the original paper, the split between train and test graph was conducted as follows: *"User scrobbles were temporally split. For each user, the first 70% of listened tracks were used as training, while the remaining tracks were used as the test set."* [60, p. 4]. This temporal split was made according to the *time / pos* attribute in the edge data, which ordered the listening history of a user depending on when they last listened to a specific song. Following the implementation of the splitting function, the resulting train and test graphs are depicted in their respective sections of Table 6.

However, through inspection of the time attributes in the training track data, it was apparent that time information was sometimes missing, as seen in Table 8. Through further examination, it was discovered that in the total 2,564,908 user-track listening relationships only 95,339 of these relations hold any *pos* or *time* attribute. Therefore, around 96% of the training data holds no information on the order in which songs were listened to.

Through further inspection of the provided source code in the ORP, the splitting condition of the full graph into the train and test graph was examined. The split condition was set to split songs in a ratio of 70/30%, according to the order of their position information. This was correctly done for all songs with available position information. However, all songs with missing position information were automatically assigned to the training set.

Graph Type	Graph Feature	Value
Full Graph	User nodes	3,307
	Track nodes	252,014
	Total nodes	255,321
	Edges	3,018,209
	Average number of tracks per user	912.67
	Max number of tracks listened to by one user	30,015
	Min number of tracks listened to by one user	2
	Total scrobbles of all users	139,602,350
	Average scrobbles per song	46.25
	Average scrobbles per user	42,214.20
	Max scrobbles per song	205,689
	Min scrobbles per song	1
	Max scrobbles per user	1,359,884
	Min scrobbles per user	2
	Graph density	9.25
	Max out-degree	30015
	Mean out-degree	23.64
	Min out-degree	1
	Max Closeness centrality (for user nodes)	0.363
	Min Closeness centrality (for user nodes)	0.198
	Max Betweenness centrality (for a sample of 100 nodes)	0.12
	Min Betweenness centrality (for a sample of 100 nodes)	0
Train Graph	User nodes	3,307
	Track nodes	252,014
	Edges	2,564,907
Test Graph	User nodes	3,279
	Track nodes	155,849
	Edges	453,300

Table 6: Descriptive statistics of the full, train and test user-track graphs

Graph Type	Graph Feature	Value
User-User Graph	User Nodes	3,307
	Edges	142,919
	Graph density	0.013
	Average number of connections per user	86.43
	Max number of connections per user	623
	Min number of connections per user	0
	Max Closeness Centrality	0.2268
	Min Closeness Centrality	0
	Max Betweenness Centrality	0.002
	Min Betweenness Centrality	0.0
	Edge Connectivity	0
	Vertex Connectivity	0

Table 7: Descriptive statistics of the user-user graph

As a result, the training set had many missing position information while the test set did not have any missing position attributes. Implications of this split condition will be discussed in the discussion Chapter 10.

Artist and Song Mappings. The mapping data of artists and tracks was examined. This step was taken to ensure that the Song ID information in the nodes of the graph data were correctly mapped to their song and artist names. This was for one reason done to verify the correctness of the description in the original study and, on the other hand, it was an important setup process for the extension model as described in Chapter 6.

The original study describes a total of 28,540 artists and 252,014 songs. On the examination of the dictionary, in the format of $\{artist-name : track-name : track-id \text{ in the graph}\}$, there were some artists names that showed ambiguity in their naming convention. For example, the artist name of the band *Avalanches* was held as a key twice, once with the naming "Avalanches" and once with the naming "The Avalanches". Due to this naming ambiguity, a total of 420 artists were removed from the mapping, resulting in a remainder of unique 28,120 artists.

Furthermore, in addition to artists, songs also appeared multiple times with ambiguous naming conventions. However, they all mapped to the same track id as the example in Table 9 shows and, therefore, did not pose an error threat for further processing. Nevertheless, one song stood out by having the same track name and the same artist key, but a different track id. This song named "*Cavalo de Fogo*" by the artist "*Banda UÓ*" held the *Song IDs* of 68,691 and 194,224. One of these ids was consequently removed in the

User ID	Song ID	Scrobbles	Position	Time
0	3307	370	inf	None
0	3308	357	inf	None
0	3309	349	inf	None
0	3310	347	inf	None
0	3311	346	inf	None
...	...	...	...	...
3306	3815	1	inf	None
3306	14156	1	800.0	Saturday 23 Jan 2010, 3:01pm
3306	12191	1	inf	None
3306	173432	1	inf	None
3306	233483	1	1224.0	Wednesday 6 Jan 2010, 9:22pm

Table 8: Example of missing *position* and *time* attributes in the training graph edge data

updated mapping to avoid a duplication of the song in the data. Therefore, the updated artist-track mapping used in the ARP package was reduced in its size from 252,014 to 252,013 tracks.

Tags Data. Tag data referring to track and artist tags like "*Rock*", "*Pop*" or similar song characteristics was included in the ORP. These tags are used in the MRecuri model as part of the track representation in embedding form. Even though a tag file was included in the ORP, the tag data was not correctly mapped to the corresponding tracks and artists. In particular, the tag file was structured in such a way that each artists had an unique ascending ID and each tag had an unique ascending ID, but their lengths were different and the tags did not correspond correctly to the artists. Hence, this tag file mapping could not be used by the author for the ARP. Hence, as elaborated on in the next subchapter, the tags had to be scraped manually.

Audio Features Data. Similarly to tag data, Spotify audio features are used by the MRecuri model to derive track representations. These files were completely missing from the ORP and had to be collected from the official Spotify API.

5.1.3 Data Collection

This section explains how missing or unusable data, which was essential for the replication model's functionality, was collected and processed.

Index	Artist	Song Name	Song Node ID
59357	Yann Tiersen	Comptine d'un autre été: L'Après-midi	15255
59390	Yann Tiersen	Comptine D'un Autre Ete L'apres Midi	15255
59414	Yann Tiersen	Comptine D'un Autre ete: L'après-midi	15255
59431	Yann Tiersen	Comptine d'un autre été: L'après midi	15255
59441	Yann Tiersen	Comptine d'un Autre Ete, L'apres-midi	15255
59444	Yann Tiersen	Comptine d'un Autre Été-L'Après Midi	15255
59459	Yann Tiersen	Comptine d'un autre été, L'Après-midi	15255
59502	Yann Tiersen	Comptine D'un Autre Été: L'après Midi	15255

Table 9: Example of ambiguous song names in the original mapping data

Spotify Audio Features Scraping. The collection of the Spotify audio features was retrieved over the official Spotify API⁶ for the relevant 252,013 tracks. Specifically, the eleven audio features *energy*, *instrumentalness*, *liveness*, *speechyness*, *acousticness*, *danceability*, *valence*, *mode*, *key*, *loudness* and *tempo* were collected for as many songs as possible. These audio features hold numerical values between 0 and 1 indicating the strength of each feature. Audio features and their values are assigned by Spotify⁷.

The collection process had three major stages. First, to access the audio features of a track, the Spotify ID for that track had to be searched for via an API call based on a match of track title and artist name. Note, that not all Spotify IDs could be retrieved, and a total of 8,912 Spotify track ids were missing after step one. Consequently, for those tracks no audio features were available. Secondly, after a Spotify ID was successfully derived from the search, the relevant audio features were collected in a separate second API call. The final audio features dataset had a total of 15,954 tracks that had missing information on their audio features. This is due to the fact that some songs do not have any audio features information, even though their Spotify IDs are available.

Thirdly, to address the issue of missing audio features for these 15,954 tracks, a multi-step imputation process was implemented for both the MRecuri replication model and the novel GNN model in Chapter 6. If an earlier step in the multi-step process did not resolve the imputation, subsequent steps were applied to these applicable songs. Firstly, for each artist, the mean scores over their audio features across their entire discography were calculated and imputed for songs by the same artist that had no entries. If this method proved impossible for certain songs (e.g. songs by artists that have no other songs), subsequently, a genre based approach was employed. The primary *genre* of each song was determined by its first tag. Subsequently, audio features were averaged across all songs within a specific genre (e.g., Rock) and imputed for songs within that genre still exhibiting missing values. Following these procedures, 205 songs remained with unresolved audio features. For these instances, a fixed value of 0.5 was uniformly imputed across all audio features.

LastFm Tags Data Scraping: For the retrieval of missing tag data, the LastFM API was used with the call function *track.getInfo*, which returned a list of the top tags for each song. For the set of 252,013 tracks, a total

⁶<https://developer.spotify.com/>, last accessed 14.02.2025

⁷<https://developer.spotify.com/documentation/web-api/reference/get-audio-features>, last accessed 17.02.2025

Tag	Count
rock	55,151
pop	35,404
indie	35,355
electronic	31,470
alternative	27,215
female vocalists	23,320
metal	16,675
indie rock	14,781
alternative rock	14,748
ambient	11,680

Table 10: Top 10 Tags over all songs

of 18,624 returned with no tag entries. The imputation was conducted in the following way. Firstly, the three most common tags among each artist were collected. For any missing songs, these most common artist tags were imputed if possible. After this process, 2,962 tracks remained without tag data. Subsequently, for these tracks another API call was made to get the *artist.getTopTags*, instead of specific song tags, and these were imputed for all the songs of these artists that still had missing tag information. Finally, a 160 tracks still remained without tags, which were consequently imputed with the tag word *unknown*, effectively creating a distinct undefined category that could still be used by W2Vec models. The Figure 10 displays the top 10 most common tags across all tracks.

5.2 Original Study Code Structure

The original study’s ORP consists of 4 Notebooks named *01-Data-Full-Train.ipynb*, *02-Embeddings.ipynb*, *03-Train-dataset-GraphDropout-Complex-Beta075.ipynb* and *04-Predict-GraphDropout-Complex-Beta075.ipynb*. An overview of their general functionality will be given in this chapter.

Broadly, the general functionality of the original study’s source code can be summarized as follows. Notebook 1 creates the relevant input graphs for the model from a database backend. Notebook 2 creates necessary embeddings and distances that are relevant to calculate loss function. Notebook 3 sets up the training data loader and trains the model. Finally, Notebook 4 creates the test data loader and makes the final inferences on the test set.

Notebook 1: Graph Data Creation. Notebook 1 created the graph data from the relevant music data. Let it be noted, that the necessary data

for the creation process stems from a MongoDB backend, which was not accessible. Hence, this notebook could not be executed and its results could not be verified. The data, that was provided in the ORP via GitHub, was assumed to be the output file of the `'data/dataset.pickle'` as defined in the last code cells of Notebook 1. Consequently, the author did not use the content of Notebook 1 for any other purpose than verification and understanding.

Notebook 2: Pre-Computed Embeddings and Distances. Notebook 2 created the pre-computed distances, embeddings and cosines that are crucial for the model. Due to requirements of the methodology of the original study, the loss function used pre-computed distances between user and tracks as explained in Chapter 4.3. These pre-computed distances were derived the following way. First, a W2Vec model implemented with *Keras*⁸ with a dot product and sigmoid activation with negative sampling in a ratio of 1:1 and a binary-cross entropy loss was used to create user and track embeddings. In this way 3,307 user and 252,013 track embeddings were created respectively. Consequently, these embeddings were used to define a distance by calculating an adapted cosine distance between each user-track pair. Finally, those cosines distances were clipped according to the threshold values of 0.1 and 0.9. The author additionally verified the quality of the embeddings by plotting them in a t-SNE visualization as depicted in Figure 7, to check if a clear separation of the embeddings was visible.

Additionally, the ORP Notebook 2 contained a section named *communities test*. This part was not described verbally in the original paper, but through inspection of the code, the author of this thesis assumed that this part involved a statistical test to measure if communities derived by the Louvain algorithm are statistically different from each other by comparing them to the cosine distances between users and tracks.

Notebooks 3 and 4: Training and Predictions. Notebooks 3 and 4 are functionally divided into the model training and model prediction process. In the ORP they were therefore separated into the individual Notebooks 3 and 4. In the ARP, they are concluded in one notebook named *6. MRecuri_Replication.ipynb*, since a lot of their functionality and dependencies overlap and are therefore suited to be combined into one notebook.

⁸A deep learning framework: <https://keras.io/>, last accessed 14.02.2025

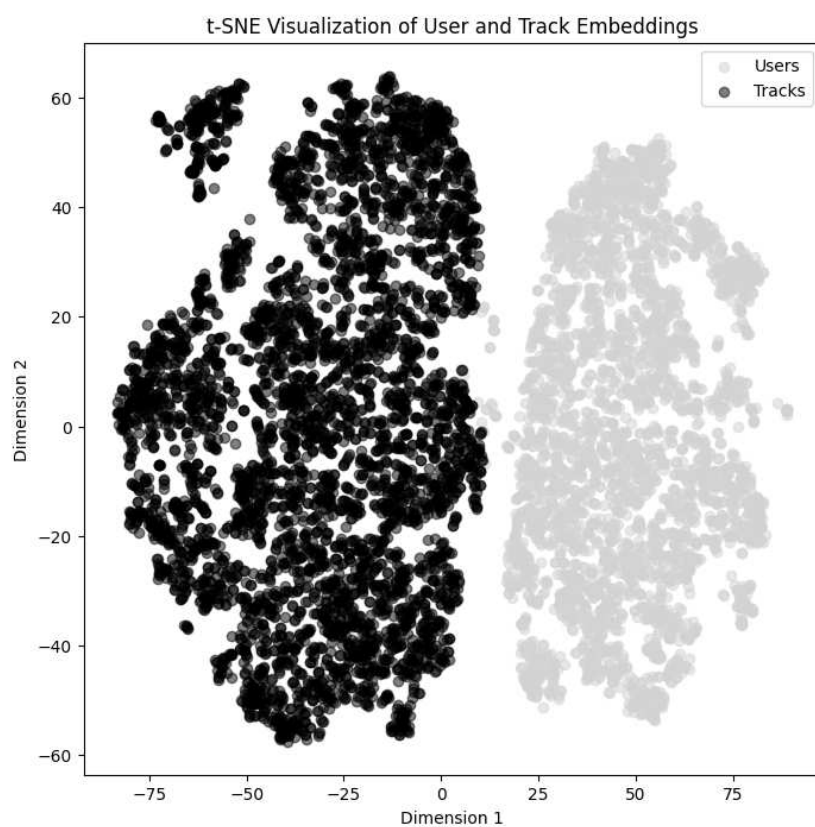


Figure 7: Distance embeddings t-SNE visualization on a sampled size of 3,307 user embeddings and 5,000 track embeddings

5.3 ARP Replication Package Structure

This chapter provides overview about the code and data artifacts provided by this thesis in the ARP. Figure 8 depicts all ARP Python notebooks in *dark blue*, while ORP artifacts and notebooks are displayed with a *white* background. A total of 11 notebooks exist in the ARP, separated into 8 different notebooks types. Notebook 1 is the data processing part of the original ORP data artifacts such as graphs and mappings. Notebook 2 transforms graphs into the necessary PyG format, and takes input data from the ORP artifacts from the pre-processing, as well as the data scraping notebooks 8.1 and 8.2 for tags and audio features data. The created PyG graphs serve as input for the GMR-ND model in Notebook 5. Notebook 3 creates embeddings and cosine distances in a similar fashion to the ORP Notebook 2, which are needed by the GMR-ND and MRecuri replication model. In Notebook 4, the structural and content based embeddings, necessary for the set up of the novelty and diversity metrics, are computed. They serve as input for the final metrics in Notebook 7. Notebook 5 creates the extension model in PyG, while Notebook 6 is the replicated MRecuri model that conforms to the code from the ORP Notebook 3 and 4. Finally, in Notebook 7, the selected benchmark models are set up and the final evaluation on the predictions are made to receive the results. All ARP notebooks can be found in the Zenodo repository⁹ and GitHub repository¹⁰ of this thesis. The ORP artifacts can be found in the original paper’s GitHub¹¹.

5.4 Implementation of the MRecuri Algorithm

The following subchapter will provide a comprehensive explanation of the source code of the ORP and the ARP. In particular, it will present the specific modifications that were made to the ORP in order to derive the ARP. The goal is to make the modification process and its results transparent and comprehensible for any potential future use.

5.4.1 ORP Functionality

This part describes only the functionality of the ORP Notebooks 3 and 4 which implement the MRecuri algorithm. Changes made to this code in order to derive the ARP are made in the following subchapter. The general functionality of Notebooks 3 and 4 of the ORP can be divided into data load-

⁹[10.5281/zenodo.15011029](https://zenodo.org/record/15011029)

¹⁰<https://github.com/timordg/GMR-ND>

¹¹<https://github.com/tommantonela/umap2022-mrecuri>

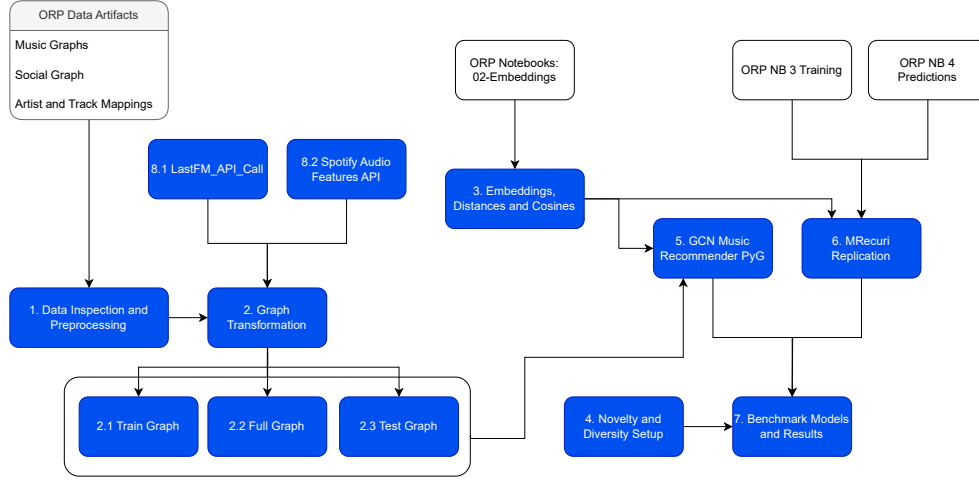


Figure 8: ARP replication package structure

ing, data generation, training batch generation, model setup, model training and inference.

DataGenerator and TrainingGenerator. As a first step, basic data loading imported the relevant user-track graph data, artist mappings and social graph data. As a second step, the `<DataGenerator>` class was defined which holds various necessary functionalities to extract batches and process the relevant data for the model. These functionalities included processes like mapping track names to ids via mapping data, processing track and artist tags, processing the audio features, receiving user-track listening data and integrating the social interactions between users from the social graph into the model. The `<DataGenerator>` also included the process of clipping scrobbles to a maximum to reduce the influence of outliers.

The `<TrainingGenerator>` extended the `<DataGenerator>` class and was designed to generate batches of training data for the training process of the neural network model. In essence, the training data included user-song interactions along with their scrobbles attributes. Furthermore, it employed negative sampling to handle class imbalances in the dataset. Lastly, the class pre-processed user-song interactions and shuffled the data at the end of each training epoch.

MRecuri Model. After the data had been brought into a suitable format for training, the model itself was set up and defined. The model was implemented using Tensorflow¹² for the general neural network functionali-

¹²<https://www.tensorflow.org/>, last accessed 14.02.2025

ties and Spektral¹³ for the Graph-Convolutional-Network (GCN) part. The model architecture was complying with the previously mentioned model architecture in Figure 6.

The model was initialized with an embedding size of 64, and a kernel size of 32 which corresponds to the dimensions of the dense layers. As depicted in the model architecture Figure 6, the model has two general types of input: user and music related inputs. User input was defined as user embeddings and user graph input. Music related inputs included tracks embeddings, artist and track tags and the Spotify audio features.

Deep Part: After the inputs had been initialized, the model continued to process the users and music inputs. User embeddings and the user graph served as input to the GCN. Subsequently, the GCN produced embeddings, which were concatenated with the initial user embeddings and resulted in the user representation.

The musical inputs were transformed into embeddings with the set embedding size of 64 and were then concatenated into the track representation of the deep part. Afterwards, user and track representations in their embedding form were passed through 3 dense layers with 256, 256 and 1 unit (i.e. number of neurons) respectively.

Wide Part: The wide part used a simple concatenation of all music input embeddings and additionally the user embedding to create the wide part embeddings. This concatenated embedding was then passed through a single dense layer with 1 unit.

In the final step, the output of the complete model was defined by adding the deep and wide layers. The customized loss function was defined as mentioned in Equation 3, and an *Adam* optimizer is used during loss calculation. Find the *model.summary()* output of the original provided ORP model code in Table 11.

Training. The training process was conducted over 10 epochs. An automatic learning rate scheduler was used to adjust the learning rate during training. On the re-run of the code when the ARP was completed and ready to train, the training process took around 12 minutes per epoch, a total of 2 hours for 10 epochs. The learning rate started with a value of 0.0010 and was reduced to 0.00090484 on the 7th epoch. The hardware used for training was as specified in Chapter 1.4.

Predictions. Contrary to the training, where the *<TrainingGenerator>* class was used to create the training batches, in the prediction part the class *<GenUserTrack>* was used to create prediction batches. The functionality, however, remained similar and enabled efficient generation of user and track

¹³<https://graphneural.network/>, last accessed 14.02.2025

Layer (type)	Output Shape	Param #	Connected to
users (InputLayer)	[(None, None)]	0	
tracks (InputLayer)	[(None, 1)]	0	
tags (InputLayer)	[(None, None)]	0	
artist (InputLayer)	[(None, 1)]	0	
embedding_users (Embedding)	(None, None, 64)	211648	users[0][0]
user_graph (InputLayer)	[(None, None, None)]	0	
embedding_tracks (Embedding)	(None, 1, 64)	16128896	tracks[0][0]
embedding_tags (Embedding)	(None, None, 64)	540806	tags[0][0]
mask_tags (Lambda)	(None, None)	0	tags[0][0]
embedding_artist (Embedding)	(None, 1, 64)	1801728	artist[0][0]
gcn_user_0 (GCNConv)	(None, None, 32)	2080	embedding_users[0][0], user_graph[0][0]
extract_track (Lambda)	(None, 64)	0	embedding_tracks[0][0]
masked_average_tags (Lambda)	(None, 64)	0	embedding_tags[0][0], mask_tags[0][0]
extract_artist (Lambda)	(None, 64)	0	embedding_artist[0][0]
spotify (InputLayer)	[(None, 10)]	0	
extract_emb_user_base (Lambda)	(None, 64)	0	embedding_users[0][0]
extract_gcn_user (Lambda)	(None, 32)	0	gcn_user_0[0][0]
concatenate_embedding_track_tag	(None, 202)	0	extract_track[0][0], masked_average_tags[0][0], extract_artist[0][0], spotify[0][0]
concatenate_gcn_user_music	(None, 298)	0	extract_emb_user_base[0][0], extract_gcn_user[0][0], concatenate_embedding_track_tag[0][0]
deep_dense_1 (Dense)	(None, 256)	76544	concatenate_gcn_user_music[0][0]
deep_dense_2 (Dense)	(None, 256)	65792	deep_dense_1[0][0]
concatenate_user_track (Concat)	(None, 256)	0	extract_emb_user_base[0][0], extract_track[0][0], masked_average_tags[0][0], extract_artist[0][0]
deep_dense_3 (Dense)	(None, 1)	257	deep_dense_2[0][0]
wide (Dense)	(None, 1)	257	concatenate_user_track[0][0]
deep_plus_wide (Add)	(None, 1)	0	deep_dense_3[0][0], wide[0][0]
Total params: 18,827,208			
Trainable params: 18,827,208			
Non-trainable params: 0			

Table 11: MRecuri model summary printout. *Column Layer (type)* shows the name of each layer in the neural network and its type in parentheses. *Output Shape* shows the dimensions of the data in each layer. *Param* shows the number of trainable parameters in each layer. *Connected to* shows to which other layers the current layer connects input or output.

batches with the necessary attributes.

Before the final inference, a few processing steps had to be made in the ORP code. Firstly, the model loaded the weights that had been saved in the training process. Secondly, from the loaded model, the user and track embeddings were derived. Thirdly, a new *fast model* was created to facilitate the prediction process. The main contribution of the fast model was that it was able to use the pre-computed embeddings of the necessary user and track embeddings which saved time when compared to creating the embeddings on the fly. Therefore, the fast model scaled better for large datasets and could make predictions faster than the original model.

Finally, the fast model made the final inferences by iterating over all users of the test set and predicted the listening probability against all possible tracks in the test set. Predictions were stored in a parquet file. No further evaluation of the predictions was provided in the ORP. Note, that results were also not visible and comparable in the ORP code output.

Additionally, note that the test dataset in this case is different from the test graph dataset with 453,300 edge relations between users and tracks. The final predictions had to be made over the possible combination of every user-track pair combination for the test users, which resulted in the *User-Track-Test-Matrix*. This matrix was of the size 3,279 (users in the test set) times 155,849 (tracks in the test set) and resulted in a total of 511,028,871 possible user-track relations for which the model predicted recommendation scores.

5.4.2 ARP Modifications

This subchapter elaborates on the changes made to ORP Notebooks 2 (Pre-Computed Embeddings and Distances) and Notebooks 3 and 4 (Training and Predictions) to successfully replicate and execute the original source code. No functional changes were made to ensure that the ARP results were as close to the original as possible. Two key modifications were necessary: first, updating the input data to a valid format, and second, adjusting the processing steps before generating the final predictions.

Embeddings and Cosines. In Notebook 2, the code had to be modified at certain points to execute without errors. Furthermore, due to the pre-processing insights, as referred to in Chapter 5.1, the input parameters of the tracks changed from a total number of 252,014 to 252,013 tracks. This modification resulted in different embeddings, distances and cosines than given in the ORP code. The modified ARP notebook can be found in the ARP in *3. Embeddings, Distances and Cosines.ipynb*. This modification resulted in different embeddings, distances and cosines than given in the ORP

code. The modified ARP notebook can be found in the ARP in *3. Embeddings, Distances and Cosines.ipynb*. Note that all further ARP modifications refer to Notebook 3 and 4 from now on.

Input Data. Due to the inaccessible information in the MongoDB¹⁴ backend database, which is used in the ORP code, the input data had to be derived manually. The process of finding the correct input format, that was subsequently also conforming with the `<DataLoader>` class, was comprised of trial and error and minimal code adaptations to trace errors. This trial and error process, to find the correct input format, was long and required many iterative print statements to trace the error and hence, can not be explained in full detail here, but its major consequences can be stated. Concretely, the ORP `<DataGenerator>` class expects information on Spotify id, artists, tracks and tags from the backend database. This expected input was altered in the ARP to take a `<pandas.DataFrame>` as input, instead of the database information retrieval. To this extent, the data frame, that served as the final input for the `<DataGenerator>` class, is as depicted in Table 12. The `<DataGenerator>` had to be modified on top of the input data modification. Specifically, the functions *process_tags_fast* and *process_spotify_fast*, had to be adapted the most, since these functions processed the tag and audio features data, which was not available in the ORP and was derived by the author, see Chapter 5.1.3. Therefore, the input format of the tags and audio features data was different than what the `<DataLoader>` class expected. Through minimal iteration steps the modifications were implemented to make the `<DataLoader>` work with the newly scraped tag and audio features input.

Prediction Processing. The second major modification, that was made to the ORP code, was the adaptation of the user and track embeddings creation before the final predictions. In particular, as previously mentioned, in the previous subchapter the fast model worked with pre-computed embeddings to make final predictions. These pre-computed embeddings were derived from the main model in the following way:

```
model_user = Model([i_users, i_user_graph],
                   [emb_user, emb_user_base])

model_track = Model([i_tracks, i_tags, i_artist, i_spotify],
                    [emb_music, emb_tracks, emb_tags, emb_artist])
```

Consequently, the batches of the respective user and track embeddings

¹⁴MongoDB is a document-oriented database program, <https://www.mongodb.com/de-de>, last accessed 14.02.2025

should be created by the following ORP code, which failed to execute due to a input value error, caused by the different input format of the data.

```
ds.mode_user = True
users = model_user.predict(ds)
ds.mode_user = False
tracks = model_track.predict(ds)
```

Hence, to circumvent this problem the user and track embedding batches, that are needed as input for the *fast model* were derived manually by the author by iterating over each batch and extracting the relevant user and track embeddings with a self-written code. This code can be found in the section *Predictions/ User and Track Embeddings Loading*:, in the file *6. MRecuri_Replication.ipynb*. The final results of the predictions made by the ARP code can be found in Chapter 8.

Artist	Song_Name	Song_Tags	Song_ID	Spotify_ID	danceability	energy	loudness	key
Black Kids	I'm Not Gonna Teach Your Boyfriend How to Dance...	indie,indie pop,indie rock	3307	NaN	0.57	0.83	-5.42	135.018.667
	Hit The Heartbrakes	indie,indie rock,indie pop	3308	6XetDleVJnCxOZK2ILx4b5	0.52	0.92	-3.38	122.980.000
Black Kids	I've Underestimated My Charm (Again)	indie,black kids,indie pop	3309	NaN	0.57	0.83	-5.42	135.018.667
	Partie Traumatic	indie,indie pop,pop	3310	2gPsk7vlnbtbDBsMOMbRJsI	0.55	0.94	-3.52	121.987.000
Black Kids	I'm Making Eyes at You	indie,indie pop,black kids	3311	0yA7xMdPNxUZRhgLCWFaNQ	0.51	0.76	-7.72	179.894.000
...								
Juveniles Daedelus	Ambitions Get Off Your HiHats	french,electro,10s american like,electronic,funky	19818 i 204002	3F85tgJl8OsoB0Ed4LjTtp 1XkhxJCx2V41yk9i5Gp0NI	0.57 0.62	0.90 0.86	-5.40 -9.32	161.005.000 139.985.000
	Let Me In Sleeper	Neo-Soul,soul,floetry doom-metal,Progressive metal,metal	222619 145949	10bdAt1jHulsSx7FnAQFSw 7HTTH7REu9c2MmrbY94fDko	0.73 0.45	0.50 0.66	-7.25 -6.88	73.024.000 95.050.000
Diana Ross and the Supremes	You Can't Hurry Love	motown,soul,oldies	64203	6kAS4yj3wHJXcLp93vr5aG	0.46	0.77	-6.60	192.390.000

Table 12: Excerpt from the modified ARP input data frame for the replicated MRecuri model

Versions. The ARP code was tested and was successfully executed on the following software and package versions *Python version 3.11.8*, *Spektral version 1.3.1* and *Keras version 2.15.0*.

5.5 Embeddings Implementation

The following section deals with the replication of the structural and content-based embeddings that serve as inputs to create the respective structural and content-based novelty and diversity metrics, as employed in the original study in Chapter 4. The original study only describes the derivation of these necessary embeddings textually in the paper and has no information on how embeddings or novelty and diversity metrics are created in the ORP code. To this end, the embeddings and, consequently, the novelty and diversity metrics have been created by the author based on a best effort replication from the textual description of the original paper. Adjustments had to be made in certain cases, when the wording was ambiguous or the results would not be meaningful. First, an overview about how novelty and diversity metrics are constructed is introduced to justify the authors line of reasoning. Later, the detailed creation of the structural and content-based embeddings will be elaborated on.

5.5.1 Novelty and Diversity Definitions

Novelty and Diversity metrics are defined in the original study by referencing the work of Sacharidis [44]. Sacharidis first defines *individual* novelty and diversity. To this extent, it is assumed that there exists a function $d(i, j)$ to measure the distance or *similarity* between the two items j and i . Then, individual diversity, depicted in Equation 7, is defined as the average pairwise distance between two recommended items from the set P_u that are recommended to user u [44, p. 140]:

$$IDIV_u = \frac{1}{|P_u|(|P_u| - 1)} \sum_{i \in P_u} \sum_{j \in P_u} d(i, j). \quad (7)$$

Consequently, individual novelty, depicted in Equation 8, is similarly calculated for user u by the average pairwise distance between a newly recommended item and an item that has been interacted with in the past (e.g., in a musical context, a past listening relation with a track).

$$INOV_u = \frac{1}{|P_u||R_u|} \sum_{i \in P_u} \sum_{j \in R_u} d(i, j), \quad (8)$$

These individual metrics can also be averaged across all users in the user base to derive their group metrics. To achieve this concept, (g) is defined as the social group comprising all individual users. The group-level metrics are then calculated by aggregating the pairwise distances across all users within the group. Specifically, the mean pairwise distance is computed, as formulated in Equations 9 and 10. This approach allows for a comprehensive assessment of group-level diversity and novelty, capturing the collective characteristics of the user base.

$$GDIV_g = \frac{1}{|P_g|(|P_g| - 1)} \sum_{i \in P_g} \sum_{j \in P_g} d(i, j), \quad (9)$$

$$GNOV_g = \frac{1}{|P_g||R_g|} \sum_{i \in P_g} \sum_{j \in R_g} d(i, j), \quad (10)$$

In addition to this definition by Sacharidis, the authors of the MRecuri model, reference one of their own papers in regards to novelty and diversity as well. The FRediECH paper [59] by Tommasel et al. introduces a very similar recommendation concept as in their MRecuri model, but in this case user-to-user recommendations in a social friends network are analyzed. In FRediECH the items that are recommended are other *users*, while in MRecuri the recommended items are *tracks*. In FRediECH novelty and diversity scores are defined in Equation 11 and 12. In this regard, R_u represents the set of recommended users (in this case items are users) for the user u , and F_u represents the set of already interacted users. The distance $d_m(i, j)$ was in this case defined as the distance between the user (*i.e. item*) embeddings j and i that represent user embeddings in the FRediECH model. Note that in this social recommender model *user* are recommended to other *users* as friends, hence the recommended *item* is in this case a *user* [59, p. 29].

$$IndDiversity(u) = \frac{1}{|R_u|(|R_u| - 1)} \sum_{i \in R_u} \sum_{j \in R_u} d_m(i, j) \quad (11)$$

$$IndNovelty(u) = \frac{1}{|R_u||F_u|} \sum_{i \in R_u} \sum_{j \in F_u} d_m(i, j) \quad (12)$$

5.5.2 Embeddings Creation

With this, a clear definition of how novelty and diversity are calculated, and how the distance $d_m(i, j)$ between the item embeddings j and i are derived has been made. Hence, the next part will elaborate on how the MRecuri model has textually defined their structural and content-based embeddings

and how these embeddings have been adapted by the author in the ARP to fit the clear definition of novelty and diversity scores by Sacharidis [44].

Structural Embeddings The original study defines the structural embedding as follows: *For computing the structural distances, each user was represented by a vector defining the consumption rate of tracks belonging to the different track communities [23]. Such communities were discovered based on a co-listened track graph (i.e., two tracks are connected if they were listened by the same user) using the Louvain algorithm[8].*" [60, p. 4]

To this end, the original study defines the structural embedding outcome to be a *user* embedding. When comparing this embedding to the before mentioned diversity metrics formulation in Equation 7, a problem arises. Diversity, is textually defined by the MRecuri paper as the distance between track pairs in the prediction set: *"Diversity was computed by comparing all recommended tracks with each other, while novelty was computed by comparing all listened tracks (training set) vs. all recommended tracks."* [60, p. 4] Hence, a user embedding is not useful to compare track pairs. The embedding needed to calculate the diversity metrics is a *track* embedding, not a *user* embedding. The author therefore assumes that the original paper may have had a mix-up in their wording, possibly caused by reusing explanations from the FRediECH paper [59], and should have been referring to a track vector. To be able to derive a meaningful structural embedding, the embedding has to be in the form of a *track* embedding, instead of a *user* embedding. To this end, the correct track novelty and diversity definition for the music recommender context are given in Equation 13 and 14. In this regard, R_t is the set of all recommended tracks and F_u is the set of all tracks that users have interacted with in the past. The distance $d(i, j)$ is in this case defined as the distance between the track (i.e. item) embeddings j and i .

$$IndTrackDiversity(u) = \frac{1}{|R_T|(|R_T| - 1)} \sum_{i, j \in R_T} d(i, j) \quad (13)$$

$$IndTrackNovelty(u) = \frac{1}{|R_T||F_T|} \sum_{i \in R_T} \sum_{j \in F_T} d(i, j) \quad (14)$$

Following, the detailed construction of the structural track embeddings which can be found in the ARP in the notebook 4. *Novelty and Diversity Setup.ipynb* is described. The structural track embeddings were implemented by first running the Louvain algorithm over the co-listening graph. The co-listening graph in this case was assumed to be the user-track graph. With this, the set of all 3,307 users was split into 9 communities. The distribution

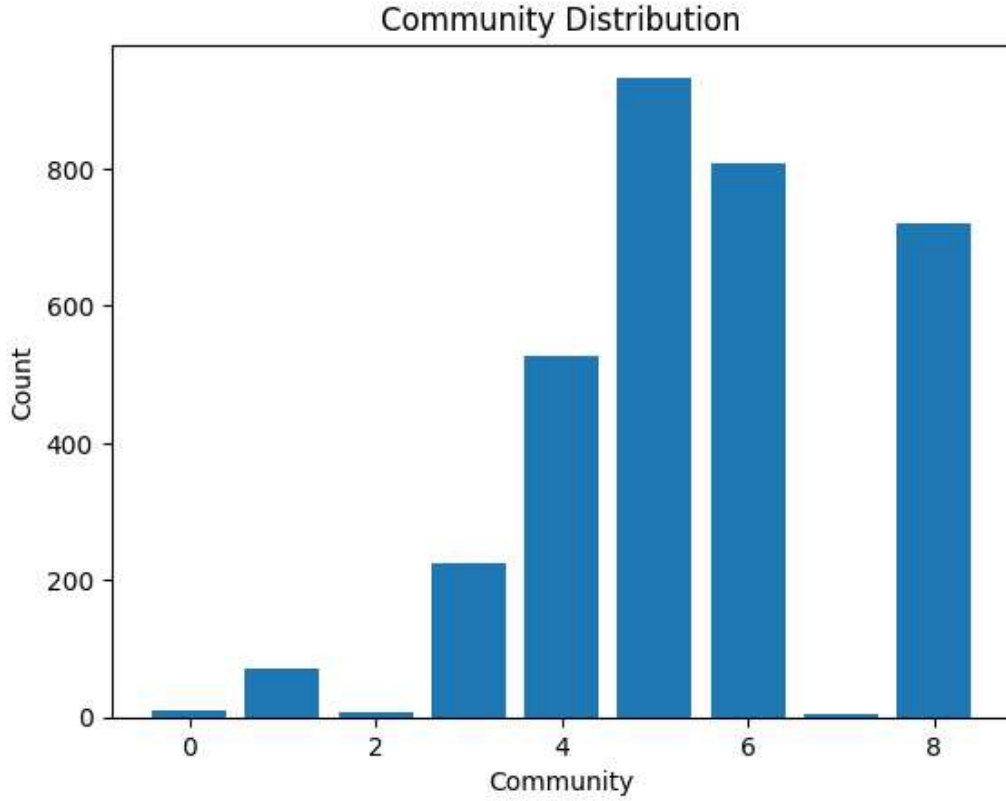


Figure 9: Distribution of the 3,307 users from the co-listening graph into 9 communities defined by the Louvain algorithm.

of users into their communities is displayed in Figure 9. As visible, some communities, such as community 0, 2, and 7, hold almost no users. Note that the Louvain algorithm is non-deterministic and may bring a different number of communities if run again.

Finally, the structural track embedding for each track was derived by initializing a vector with the dimension length of the user communities, namely 9. Each dimension of the track embedding, therefore, displayed one of the nine user communities. For each user which had listened to a specific track, this users community number was added as a counter or *consumption rate* to the vector of the track. To this end, the author of this thesis coined this term as *reversed-consumption rate*, since tracks consume users and their communities.

To make this process more clear, an example will be given to illustrate the process of the structural embedding creation with one track. Track t is listened to by exactly two users, user a and user b . User a belongs to

community 1 and user b belongs to community 2. The structural embedding for track t is initialized with a zero value in all 9 dimension, as seen in Equation 15.

$$structural_embedding(t) = [0, 0, 0, 0, 0, 0, 0, 0, 0] \quad (15)$$

Subsequently, each users community increases the value of the dimension of the community they belong to by one. In this case only user a and user b increase dimension 1 and 2 of the tracks embedding by exactly 1. The other dimensions are not increased, since no users from these communities listened to track t . This results in the final structural embeddings for track t as in Equation 15.

$$structural_embedding(t) = [1, 1, 0, 0, 0, 0, 0, 0, 0] \quad (16)$$

In practice the dimensions of the consumed communities were normalized. Hence, an example from an actual structural embedding in the ARP, in this case for track ID 1002, is displayed in Equation 17.

$$structural_embedding(tid=1002) = [0. , 0. , 0. , 0.02380952, 0. , 0.86904762, 0.07142857, 0. , 0.03571429] \quad (17)$$

In this way, the the community affiliation of each song node are depicted in the structural track embeddings. Overall, for each 252,013 tracks a structural embedding with each 9 dimensions was created that make up the structural track embeddings which are in turn needed for the structural novelty and diversity score calculation. To verify the results of the structural embedding creation a t-SNE visualization was used to display the structural track embeddings as seen in Figure 10. The t-SNE plot shows a sample of 1.000 tracks and their structural embeddings with dimensionality reduction on the scatter plot.

As one can see in the figure, there exist five central hubs where many embeddings are clustered. These embeddings correspond to those tracks that had user consumption from only one community. Embeddings that lie in between the central hubs are assumed to be more diverse tracks that have consumption's from several user communities. Overall 81,927 tracks (32.51%) had consumption from exactly one user community, while 170,086 tracks (67.49%) had user consumption from multiple user communities.

Content-based Embeddings The creation of the content-based embedding also had to be adapted slightly in regards to the MRecuri definition. The original definition was given as: "[...] *content-based dissimilarity was*

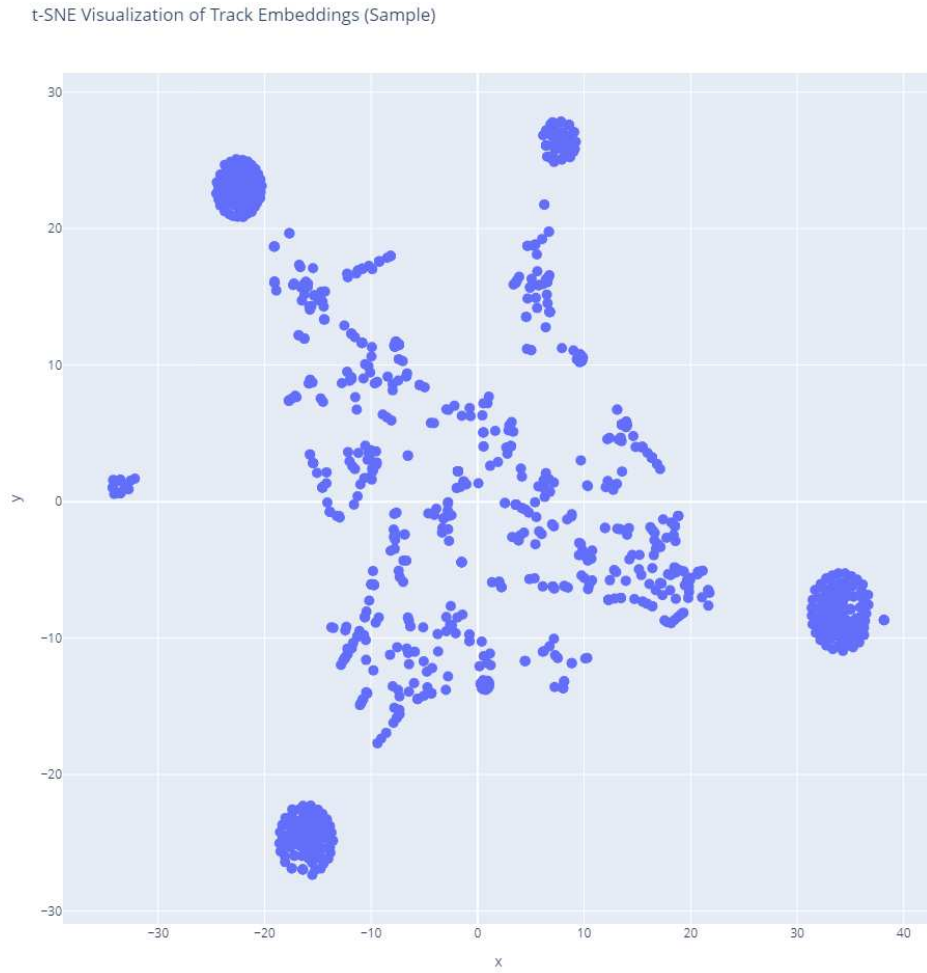


Figure 10: t-SNE scatterplot of the structural track embeddings. Excerpt of 1,000 structural embeddings is shown. The spatial arrangement of the points represent the structural embeddings relatively to each other where the quantity of the coordinates have no further meaning.

computed based on the Word2Vec representation of tracks' tags . User representation was defined as the average representation of the listened tracks." [60, p. 4]. However, since for our approach, it is necessary to receive track embeddings, the last step of the original definition, which creates a user representation by averaging all track embeddings for one user, can be omitted. In this way, only the track embeddings remain. Therefore, the corrected definition as used in this thesis is: *Content-based track embeddings were computed based on the average Word2Vec representation over all tags of one specific track.* In the passage below, this creation and implementation process will be elaborated on in more detail.

First, the track tags were pre-processed and put into a valid processing form for the W2Vec model. The W2Vec model used was *gensim* [41]. The parameters used for the gensim model were set to 300 for the vector size (dimension), min count = 5 and a window of 9 according to best practice advised by the *gensim* documentation [41] and viability tests by the author of this thesis. These viability tests consisted of testing the above mentioned parameter settings to conclude which would yield the most suitable results in terms of using the song tags as vector representations.

These parameters ensured that only relevant tags were included in the W2Vec model training. In total, 1,157,464 unique tags existed over all 252,013 tracks, with 51,277 tags being unique. After the model training, only 9,080 out of the 51,277 tags remained. The missing 42,197 tags did not reach the threshold, specified by the min count, of 5 times over all songs and were filtered out. After the gensim model training, a tag like "Rock" was embedded with the specified 300 dimensional vector.

Subsequently, to derive the content-based track embedding for a single track, these tag embeddings had to be averaged over all tags of each individual song. As an example, if track *t* had 3 tags, "rock", "pop" and "soul", these 3 tag embeddings were averaged into one final track embedding. To verify the quality of the content-based embeddings, a t-SNE visualization was used as depicted in Figure 11. The figure shows several elements. First, the song embeddings, which belong to certain artists, are displayed as points in the scatterplot. The coloring of the tracks shows the artist affiliation as seen in the legend on the right hand side of the graph. Furthermore, the semi-transparent bubbles depict genres in the form of individual tag embeddings.

In this way, the author wanted to verify the quality of the embeddings by checking if the songs corresponded to their assumed genre tags. The figure shows multiple artists and their songs whereas e.g. *Miles Davis*, a famous jazz musician, has his songs correctly displayed in the direction of the *blues*, *jazz* and *ambient* genre tags. Similarly, the *Red Hot Chilli Peppers* have

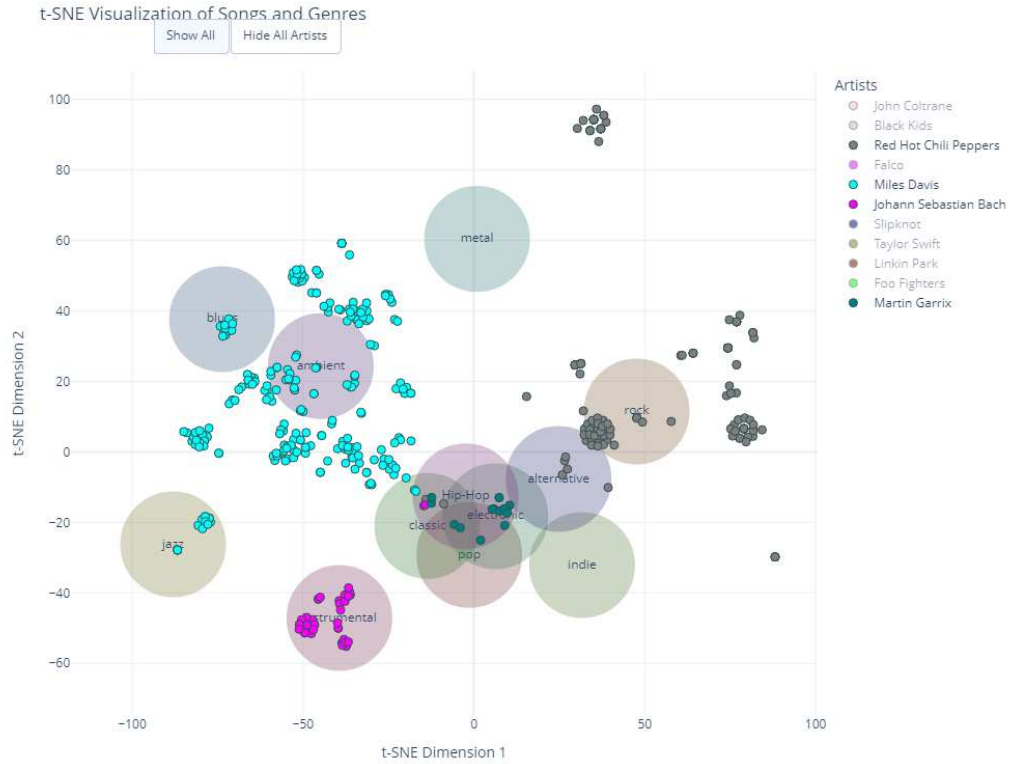


Figure 11: t-SNE visualization of the content-based track embeddings

their songs put into the bubble direction of *metal*, *rock* and *alternative*. To this end, the author acknowledges the quality of the created content-based embeddings as sufficient.

5.6 Reach-out to Authors

Due to some unclear and missing information in the ORP the author tried to reach out to the authors of the original study for further clarification. The reach out process was conducted by a colleague, who was writing his master thesis on the same original paper as this thesis¹⁵. To avoid overwhelming the original authors with multiple emails from myself and my colleague at the same time, we decided to combine our outreach efforts and send the mail only from his email.

The researcher initiated contact with Antonela Tommasel, the lead author of the original paper, at the outset of this replication project. On March 16,

¹⁵Master Thesis by Benedikt Badke

2024, an email was dispatched detailing the replication project’s objectives and requesting information about the data selection methodology.

Ms. Tommasel replied on March 25, 2024, offering clarification on the selection process. The colleague followed up on March 27, 2024, posing additional and more detailed questions in an attached document with questions from himself and the author of this thesis. These inquiries primarily focused on data collection scripts, GitHub repository tags, and the test-train split utilized. Despite sending a reminder on April 17, 2024, no further responses were received from Ms. Tommasel.

Subsequently, on May 4, 2024, the researcher reached out to all authors of the original study via email. This communication included two documents: one containing the previously unanswered questions and another with new inquiries regarding the implementation of structural based and content based diversity and novelty metrics. As of now, this email remains unanswered and no further information about the unclear matters of the original study has been given. A summary of the reachout process is given in Table 13 and the complete set of emails, answers and questions can be found in Appendices A, B and C.

Date	Contact		Topics Covered	Answer
16/03/2024	Antonela masel	Tom-	Inquiry about research focus and data selection methodology	Yes
27/03/2024	Antonela masel	Tom-	Implementation details for data gathering, tag usage, and dataset partitioning	No
17/04/2024	Antonela masel	Tom-	Follow-up on outstanding questions	No
04/05/2024	Antonela masel, Manuel Rodriguez, Daniela Godoy	Tom- Juan	Code implementation for data collection, tag utilization, dataset division, and evaluation metrics	No

Table 13: Summary of communication efforts with the original study authors

6 Model Extension

This section covers the extension of the original model. Particularly, the goal was to build a full Graph-Neural-Network Music Recommender for Novelty and Diversity (GMR-ND) model in contrast to the hybrid CNN-GCN model employed by MRecuri. The terms *full* and *hybrid* used in this chapter refer to the architecture of the respective models. While MRecuri uses both a CNN and a GCN, the usage of the GCN is very limited and the CNN influence dominates in the model. In contrast, the GMR-ND model is a model that functions purely on a GNN basis and is, therefore, referred to as a *full* Graph-Neural-Network model.

The remainder of this section will be structured as follows. First, an overview about the specific input data, that is used to create the necessary graph structures in the PyG format, is given. This includes the detailed description of the creation of the three graphs, full graph, train graph and test graph, that serve as input for the GMR-ND model.

Subsequently, the architecture of the GMR-ND will be elaborated. In total, four slightly different models are created. Lastly, the training and testing process will be described in detail and relations to the original MRecuri study methodology will be outlined.

6.1 PyG Graph Creation

The input graphs for the GMR-ND model have to conform to a specific structure of the PyG library. This structure is different from the graph format used in the original MRecuri ORP, which used the Python library NetworkX for their graph-data processing. Therefore, the relevant data had to be extracted from the originally provided NetworkX graphs and was then processed and formatted accordingly to the PyG graph specifications. This section will cover the details of what data was used to create the specified PyG graph data structure and the difference between the three created graphs of the full graph, train graph and test graph. For this section, let the term *graph* be equivalent to a graph in the format of PyG, unless specified otherwise.

6.1.1 Graph Structure

In order to make the creation process more understandable, an overview of the final graph data structure will be given at this point. The creation process will follow afterwards. All three graphs conform to this structure and any deviations will be elaborated on in the specified sections below. The graphs data structure is a heterogeneous knowledge graph, as defined in Chapter

2.3. The graph has exactly three node types and three edge types. The three node types are user nodes, track nodes and artist nodes. The edge types are user-to-user, user-to-track and track-to-artist relationships. User nodes are made up of the users, the music consumers in this context, that listen to songs and have social relations to other users in the network. Track nodes are songs specified by song name or song IDs. Songs can be consumed by users and have relations to the artists that created those songs. Artist nodes are the musical artists that created the songs in the track nodes. Artists only have relations to track nodes.

A sub-sampled graph with 35 song nodes, 10 artist nodes and 36 user nodes has been created to show the exemplary graph structure, of the actual graph used in the model, visually as depicted in Figure 12. Blue nodes display artist nodes, yellow nodes are song nodes and red nodes are user nodes. As one can see, blue nodes always only connect to yellow nodes since artist nodes only have *made-of-relationships* to songs, never to users. Similarly, users only ever have edges to songs or other users. Song nodes may have relationships to artist and user nodes. An interactive *HTML* version of this visualization can be found in the ARP Zenodo repository¹⁶ in the file *beautiful_graph.html*.

6.1.2 Input Data

As previously mentioned, the NetworkX graph structure, provided by the ORP, was in a different format than the necessary PyG graph structure. To this end, the necessary data for the PyG graph creation first had to be extracted from the provided ORP NetworkX graph data.

The full graph data frame, holding the original NetworkX graph information and which served as the starting point for the PyG graph initialization, is depicted in Table 14. Note that the Song_IDs in the graph input tables do not start at index 0, but at index 3,307. This is due to the fact that in the NetworkX graphs, different node types are only differentiated by their node attributes and IDs. Therefore, the IDs 0 - 3,306 belong to user IDs and song IDs go from 3,307 - 255,320 in the NetworkX graph. In contrast, PyG graphs can define different node types explicitly, and their node IDs are initialized at index 0 for each node type. This differentiation is important for the transformation process.

Through further processing of those input data frames such as duplicate song removals and song ID resets to index 0, the final form of the input data frames was as depicted in Table 15. Note that time and position attributes

¹⁶[10.5281/zenodo.15011029](https://zenodo.org/record/15011029)

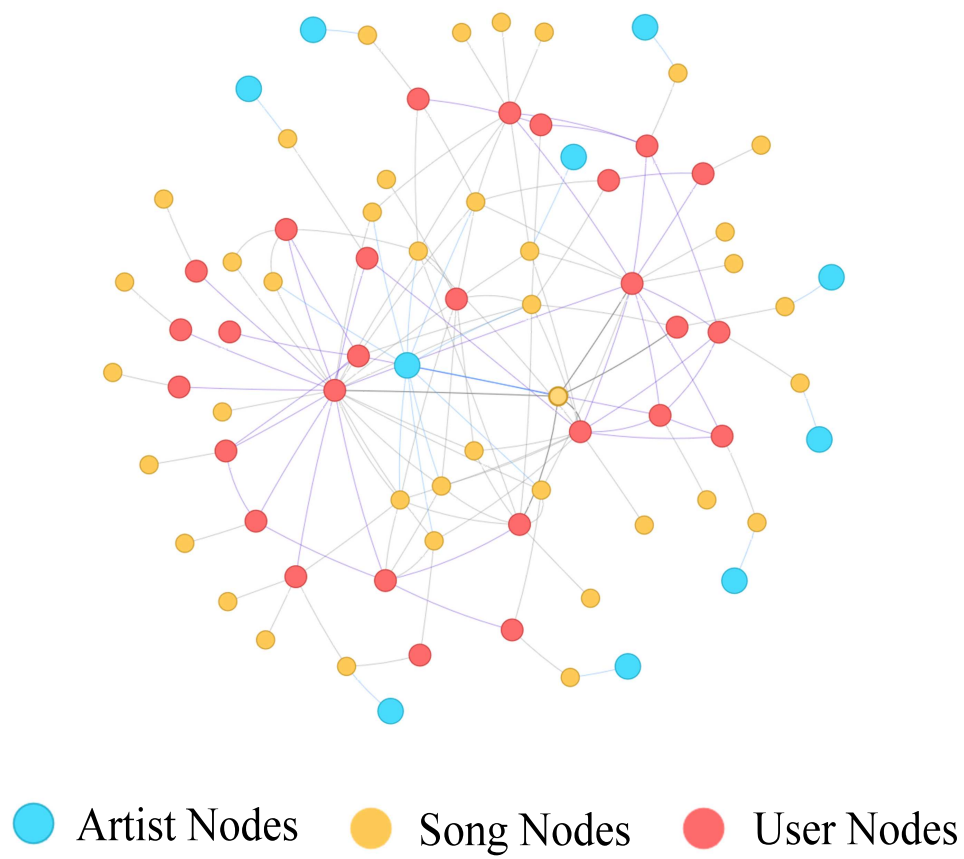


Figure 12: PyG heterogeneous music graph structure with three node types and three edge types

are still missing. However, this is not an error, since the time and position attributes were only used when splitting the data into training and test graphs, they were not used in the model for training. In order to align with the same methodology for training and test splits as the original MRecuri model, these missing values were not imputed and the split is, therefore, the same as in the original study. Table 15 just shows these attributes for the sake of completeness, since this is how the input data looked like in the ARP code. Further elaboration can be found in Chapter 10. The most important change in regards to the initial input data is the reset of the Song_ID index to start from 0 and the inclusion of the artist and song names.

Index	User_ID	Song_ID	Scrobbles	Position	Date
0	0	3307	370	inf	None
1	0	3308	357	inf	None
2	0	3309	349	inf	None
3	0	3310	347	inf	None
4	0	3311	346	inf	None
:	:	:	:	:	:
3018204	3306	3815	1	inf	None
3018205	3306	14156	1	800.0	Saturday 23 Jan 2010, 3:01pm
3018206	3306	12191	1	inf	None
3018207	3306	173432	1	inf	None
3018208	3306	233483	1	1224.0	Wednesday 6 Jan 2010, 9:22pm

Table 14: Full Graph NetworkX data

Index	User_ID	Song_ID	Scrobbles	Position	Date	Artist	Song Name
0	0	0	370	inf	NaN	Black Kids	I'm Not Gonna Teach Your Boyfriend How to Danc...
1	0	1	357	inf	NaN	Black Kids	Hit The Heartbrakes
2	0	2	349	inf	NaN	Black Kids	I've Underestimated My Charm (Again)
3	0	3	347	inf	NaN	Black Kids	Partie Traumatic
4	0	4	346	inf	NaN	Black Kids	I'm Making Eyes at You
:	:	:	:	:	:	:	:
3018202	3306	508	1	inf	NaN	Kings of Leon	Use Somebody
3018203	3306	10849	1	800.0	Saturday 23 Jan 2010, 3:01pm	Kings of Leon	Wasted Time
3018204	3306	8884	1	inf	NaN	Kiss	Love Gun
3018205	3306	170124	1	inf	NaN	Kittie	Charlotte
3018206	3306	230175	1	1224.0	Wednesday 6 Jan 2010, 9:22pm	Kittie	Cut Throat

Table 15: Cleaned full graph input data for PyG graph creation

6.1.3 Transformation Process

After the input data was initialized and song IDs were corrected, the data was further processed to conform to the PyG graph input structure. The output of the full PyG graph is displayed in Table 16. To this end, the necessary data structure for nodes and edge initialization was conducted as follows.

Nodes. Nodes were initialized with the function *torch.arange*, which created a tensor of a specified length. In the case of the full graph, the track nodes tensor had the form of *tensor([0, 1, 2, 3, ..., 252.012])* where the specified length was 252,013, which displayed all track ids in an ascending manner from the index 0 to the maximum ID.

Node features. Nodes were additionally enhanced with node features. Node features are certain attributes that can be assigned to nodes and can be used in the model definition and training process.

For song nodes the afore mentioned track tags and Spotify audio features were used. The textual tags and quantitative audio features had to be transformed into embeddings to be used by the model as node features. The textual track tags were hot-encoded by assigning each unique tag to a number. As an example, if there exist 3 songs with the following tags: Song 1: *[rock, pop, jazz]*, Song 2: *[jazz]*, Song 3: *[classical, instrumental]*, then their mapping would be: *rock* -> 0; *pop* -> 1, *jazz* -> 2, *classical* -> 3 and *instrumental* -> 4. If a song held fewer than 3 tags, the missing tag positions were filled in with -1. In this way, for each song a three dimensional mapping was implemented based on the track's top 3 tags. For the songs in this example, the embedding would have the following form:

$$x_tag = tensor([[0,1,2], [2,-1,-1], [3,4,-1]])$$

Audio features were in the form of numerical values of the eleven audio features types *energy*, *instrumentalness*, *liveness*, *speechyness*, *acousticness*, *danceability*, *valence*, *mode*, *key*, *loudness* and *tempo*. *Energy* and *instrumentalness* are values between 0-1, while *mode* for example refers to either 0 or 1 depending on if it is a major or minor mode. An in-depth explanation of all audio feature values can be found on the Spotify API¹⁷. They could, therefore, be directly transformed into a fitting song node feature embedding

¹⁷<https://developer.spotify.com/documentation/web-api/reference/get-audio-features>, last accessed 14.02.2025

tensor with 11 dimensions in the form of:

$$x_audio = tensor([[0.5707, 0.8297, -5.4162, ..., 0.7432, 135.0187, 4.8333]])$$

For artist nodes, artist tags were assigned as features by averaging the tag embeddings of their three most common tags over all their songs. For example, if the band "Linkin Park" had their three most common tags with *Rock*, *Alternative* and *Metal*, these three tags were assigned to the artist tag feature in their encoded embedding form.

This processes of creating node features should mirror the usage of the song tags, artist tags and Spotify audio features embeddings as used in the original MRecuri model as explained in Chapter 4. In this way, it can be tested if the usage of node features influences the results of the different GMR-ND models. More on the incorporation of the node features into the GMR-ND model and their influence on results can be found in the the GMR-ND architecture Chapter 6.2 and the results Chapter 8.

Edges. Edges in the heterogeneous PyG graph were initialized with index mappings in the form of a two-dimensional tensor. As an example, the edge index mapping of the full graph's (*users*, *listens_to*, *songs*) relationship is displayed below.

$$tensor([[0, 0, 0, ..., 3306, 3306, 3306], \quad (18)$$

$$[0, 1, 2, ..., 8884, 170124, 230175]]) \quad (19)$$

This tensor has the shape of *torch.Size([2, 3018207])*, which means it has 2 dimensions and in each dimension a total of 3,018,207 values. The first dimension corresponds to user IDs while the second dimension corresponds to song IDs. The number 3.018.207 denotes the total listening relationships between all users and songs. The index position of each dimension corresponds to the edge mapping relationship between the user and song nodes. In the above case, the user ID in dimension 0, is 0. Similarly the track dimension in index 0 is also 0. Therefore, this corresponds to a (*users*, *listens_to*, *songs*) relationship between the user with ID 0 and the song with the ID 0. When looking at next index positions, one can see that user 0 has also listened to the track with the ID 1 and 2. In this way, (*users*, *listens_to*, *songs*) are displayed and stored as an *edge_index* mapping in the heterogeneous graph object.

Edge Mappings. In total three different edge index mappings had to be derived from the input data that correspond to the three different edge types

- (*users, listens_to, songs*), (*users, is_friends_with, users*) and (*artists, makes, songs*) relations. Note that for the three different graphs (full, train and test) the creation process was slightly different due to the different input data, but functionally the processes were the same.

The (*users, listens_to, songs*) edge index mapping was created by extracting the user-to-song relationships from the final input graph as shown in Table 15. Each row displays one listening relationship that denotes the user ID and the corresponding song ID, plus additional information like scrobbles, artists and song names which are not relevant for the edge index creation.

The (*users, is_friends_with, users*) edge index mapping was derived from the social graph in the ORP package. Source and target nodes were directly extracted from the social NetworkX graph provided by the original MRecuri ORP. The social graph can be found in the *dataset_typed.pickle* file by accessing *dataset['users']* in the path *data/MRecuri_data/dataset_typed.pickle* in the ARP. Here dimension zero maps user ids to other users ids in dimension one. For example, user ID 0 is friends with user ID 763.

$$\text{tensor}([0, 0, 0, \dots, 3306, 3306, 3306], \quad (20)$$

$$[763, 1435, 122, \dots, 326, 926, 700]) \quad (21)$$

Thirdly, the (*artists, makes, songs*) edge index mappings were created based on the adapted artist mapping and holds the following form in the full graph. Here, dimension zero corresponds to artist ids that map to song IDs in the first dimension, as visible in the last index position the edge index, the artist ID 28,119 has created the song with ID 115,034.

$$\text{tensor}([0, 0, 0, \dots, 28117, 28118, 28119], \quad (22)$$

$$[0, 1, 2, \dots, 112259, 14295, 115034]) \quad (23)$$

6.1.4 Full, Train and Test Graph

With the subsections above, the most important steps of the graph creation process are covered. This section shows the final structure of the PyG graphs and highlights their individual differences that are crucial to consider for the training and testing part. Table 16 depicts the full, train and test graphs as heterogeneous PyG graph objects (the graph objects that PyG models work with), with their respective detailed node and edge characteristics. The most important aspect to consider in this regard is the *test graph*.

Since the PyG graph data is derived from the original ORP and the corresponding *NetworkX* graphs, their characteristics in terms of number

PyG Graph Objects	Attributes	Full Graph	Train Graph	Test Graph
Nodes	Users			
	node_id	[3307]	[3307]	[3279]
	Songs			
	node_id	[252013]	[252013]	[155848]
	x_audio	[252013, 11]	[252013, 11]	[155848, 11]
	x_tag	[252013, 3]	[252013, 3]	[155848, 3]
	Artists			
	node_id	[28120]	[28120]	[21943]
	x_tag	[28120, 3]	[28120, 3]	[21943, 3]
Edges	(users, listens_to, songs)			
	edge_index	[2, 3018207]	[2, 2564907]	[2, 509212521]
	edge_attr	[3018207]	[2564907]	–
	(users, is_friends_with, users)			
	edge_index	[2, 142919]	[2, 142919]	[2, 141145]
	(artists, makes, songs)			
	edge_index	[2, 252013]	[2, 252013]	[2, 155848]

Table 16: Comparison of PyG Graph Objects

of nodes and edges match. This is a positive sign that signals that the transformation process was successful and the data between the different graph representations does not deviate. This can be seen by comparing Figure 4 on page 40 and Figure 16 . As outlined in Chapter 4, the split between the train and test graph has been performed according to a temporal split for each users listening history. Therefore, the full graph holds all nodes and edges, the train graph holds around 70% of the listening data per user and the test graph covers the remaining 30% of users listening data.

With this, the most important data features of the PyG graph objects as depicted in Table 16 can be summarized as follows. The *full* and the *train* graph exhibit a total number of user 3,307 nodes with the corresponding feature of their *node_ids*. The test graph only covers 3,279 users. This decreased user number in the test set is due to the temporal split. The temporal split only selected tracks for the test set if they were listened to by more than one user. To this end 28 users had listened to tracks that no other user listened to, these users were excluded from the test set.

Song nodes exhibit a total of three features, namely *node_id*, *x_audio* and *x_tags*, where the later two refer to the tag and audio embeddings as

elaborated on in the sections above. The full and train graph show a total of 252,013 unique songs, with the audio features having 11 dimensions and the tags data having 3 dimensions in their respective embedding vectors. In contrast, the total amount of unique songs present in the test graph is 155,848. The test graph shows fewer tracks because of how the data was split. Only songs that at least two different users had listened to were included in the test set. Embedding sizes for the song node features remain the same for the test graph in comparison to the full and train graph.

Artist nodes display similar characteristics, whereby the full and train graph include all unique artists with 28,120 artists while the test graph holds 21,943 unique artists. Again, the reduced number is explained by the temporal split. Some artists were removed due to them having no tracks in the test set. Artist node features, in the form of the artist tag embeddings with 3 dimensions, are the same for all three graphs.

Edge data is comprised of the three different edge types (*users, listens_to, songs*), (*users, is_friends_with, users*) and (*artists, makes, songs*). The (*users, listens_to, songs*) relationship for the full graph holds all 3,018,207 recorded user-track interactions. Furthermore, the train graph covers a total of 2,564,907 user-song relations that are used for training the model. In contrast, the test graph is made of a 509,212,521 edges between user and song nodes. In order to comply with the test setup of the replication MRecuri study and to perform prediction over the same amount of edges, it is necessary to perform the testing predictions over the complete *User-Song Test-Edge Matrix*. The complete *User-Song Test-Edge Matrix* holds all possible edge combinations between the user and song nodes in the test graph. This means for $3,279 \times 155,848$ there are a total of 511,025,592 possible edges to predict over. This number is reduced by emitting the training edges that were already present during training for each user. This results in a total of possible 509,212,521 edges between users and songs in the *User-Song Test-Edge Matrix*. Therefore, this *User-Song Test-Edge Matrix* is the input for the PyG test graph edge index between user and songs. With this, the three necessary graphs for the GMR-ND model training and inferences have been initialized.

6.2 GNN Architecture

The Graph Neural Network architecture implemented in the GMR-ND model is designed to process and learn from the input heterogeneous graph data and is tailored to a music recommendation system with a focus on novelty and diversity. The architecture leverages graph-based learning through neighborhood aggregation in order to capture complex relationships between users,

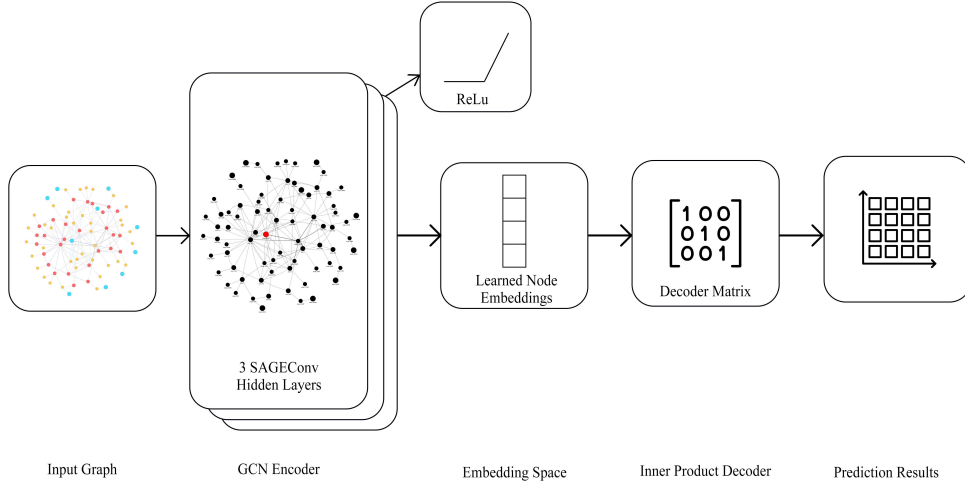


Figure 13: GMR-ND (GNN) architecture

songs, and artists. The basic GNN design choices were built on best practice methods from the PyGeometric documentation.

The models architecture is fundamentally structured as an encoder-decoder model that is able to learn from graph-based input and makes link predictions on user-track relations. The encoder component is realized by the learnable node embedding initialization and their subsequent processing by the graph convolutional layers. The graph convolutional layers are the main mechanism for capturing and propagating structural information across the graph. Specifically, GraphSAGE (Graph SAmple and aggreGatE) [25] convolutions are employed, which operate by sampling and aggregating features from a node’s local neighborhood in order to update the learnable embeddings. In the last step of the encoder part, the embedding features undergo a non-linear transformation through a ReLU activation function.

The decoder component is responsible for utilizing the encoded graph representations in their embedding form in order to make predictions about user-song interactions. In particular, an inner product decoder is used, which computes the likelihood of an edge (a user-song interaction) between user and song nodes, based on the user and song node learned embedding representation. The output is a scalar value and represents the model’s predicted probability that a user will listen to specific song. Additional details on the training process, loss functions and prediction process will be covered in the next section.

Model Variations While the basic encoder-decoder architecture of the GMR-ND remains the same, two different models have been set up: a *shallow model* and a *rich model*, each with its architectural characteristics. The *rich* and the *shallow* model refer to their dedicated usage of node features. While the *rich model* uses the specified song and artist node features of tags and audio embeddings, the *shallow* model uses only the graph structure without any node features.

The shallow model’s primary characteristic is its reliance on learned embeddings through message passing as the sole source of node information. It does not utilize any pre-existing node features or attributes, focusing instead on learning representations purely from the graph structure and edge information.

The rich model on the other hand, initializes its embeddings with the tags and audio node features. Note, however, that the message passing process during learning still updates the embeddings and learns the relationships between users, songs and artists. The node features, therefore, only serve as an additional learnable information that is combined with the learned embeddings from the graph structure.

In addition, the shallow and rich model have been differentiated further by implementing different kinds of loss functions. Specifically, each model was trained one time on a custom loss function and one time on a common binary-cross entropy loss function. The custom loss is equivalent to the loss function as defined by the original MRecuri paper in Chapter 4, and should increase the novelty and diversity scores to help users break free from filter bubbles. To this end, the impact of the proposed loss function by the MRecuri model can be studied with regards to the impact on the GMR-ND prediction results.

This ultimately results in a total of 4 different models based on shallow or rich node features and a binary-cross entropy (BCE), and a custom loss (CL) function with pre-computed distances. The four GMR-ND models are hereon forward notated as BCE Shallow, CL Shallow, BCE Rich and CL Rich, where *Shallow* and *Rich* conform to the node features architecture and *CL* refers to *Custom Loss* and *BCE* to *Binary Cross Entropy*.

6.3 Training and Predictions

The following section covers the training and prediction process in the GNN. The prediction task in the model is designated as a *link prediction task*, which predicts *links* (i.e. *edges*) between user and song nodes. The general training and prediction process will be explained and the specifics on loss function differentiation between binary-cross entropy and custom loss is given.

Generally, the training process for the GNN model is designed to optimize the network’s parameters through back-propagation, in order to make accurate music recommendations (edge predictions) on new graph-based input.

Input Data. The input training graph was split into a training and validation part in the ratio of 90/10. Note that this does not interfere with the 70/30 training test split of the original MRecuri model. The training graph still contains 70% of the data, just as the original MRecuri study, but since the original study did not employ a validation set, this study chose to take the validation set from the original training set. To this end, the GMR-ND training data held a total of 2,308,416 user-song edges while its validation counterpart had 256,491 user-song edges. Validation data was used to derive the validation loss during training. The test set, which is used for the final prediction, was made up of 509,212,521 edges between user and song nodes. Note, that even though three edge types exist in the heterogeneous graph, only the user-song edges were used for prediction, since we are only interested in predictions between users and songs. User-user and artist-song edge relations were used only during training and message aggregation to learn the graph structure and relationship between nodes.

Sampling. As a next step, the sampling technique *LinkNeighborLoader* from PyG was used which collects batches of graph data according to their edge relations. Sampling and splitting graph-based data is not a trivial process, and many different methods exist [25, 29, 71]. Samplers were responsible for loading training, validation and test data for training and prediction into the model. Most importantly, the samplers in PyG create and manage different edge types efficiently and create mini-batches that include a mix of message passing edges (for embedding aggregation) and supervision edges (for loss calculation), along with negative samples.

Model Training. The training process is an iterative process that feeds the training graph data, in the form of batches specified by the sampler, into the model. The specified epochs dictate how many times the training data is processed by the model. All four models were trained for 10 epochs. In the training process, the models forward pass function aggregated embeddings with the encoder and made predictions with the decoder. These predictions were then compared against the ground truth (the supervision edges) to compute the loss with the help of the loss function.

To this end, two different loss functions were used. First, a standardized binary-cross entropy loss that is commonly used in many machine learning tasks. Secondly, the custom loss function as defined by the original MRecuri model paper in Chapter 4.

After the loss had been calculated, the model computed the gradients and updated the model parameters with an optimizer. In this case, the "*Adam*"

optimizer was used with a learning rate of 0.001 in all four models.

Validation. In addition to the training loss, the validation loss was also computed in each epoch. The validation loss served the purpose of investigating the performance of the model on yet unseen data. The training and validation results will be elaborated on in the results Chapter 8 and 10.

Prediction and Evaluation. After the training process was completed, the model’s performance was evaluated on the held-out test set. This test set provided an unbiased estimate of the model’s generalization ability since the model had no previous interaction with this data. As specified in Chapter 6.1.4, the test set user-track relations were compromised of the full *User-Song Test-Edge Matrix* resulting in a big graph with 509,212,521 edge relations. The sampling size during testing was set to 40,000 samples to reduce testing time, as testing with a usual sample size of, e.g., 528 would have taken the tenfold amount of time. During testing, the test graph-data is passed through the model similarly as in training, but the back-propagation step is omitted since the test step should only make predictions rather than model parameter tuning. Furthermore, during testing no supervision edges were used to calculate a loss. The loss evaluation was not necessary since the final evaluation of the predictions made on the test set were carried out separately after the prediction process.

The testing function had to save results to the disk in order not to crash and run out of RAM memory. The output of the testing function yielded a parquet file with 3.6GB in size for each model. From the parquet file the top 10 song predictions for each user were trimmed to receive the final prediction *.csv* file that contained a total of 32,790 predicted songs (10 songs for each of the 3,279 users). The results of the GMR-ND model predictions are elaborated on in Chapter 8.

7 Evaluation

This section elaborates on the detailed setup of the evaluation design that leads to the results. This includes the specification of models that are tested in terms of their predictions performance and their according benchmark models. The setup of the benchmark models will be explained thoroughly. Lastly, the evaluation metrics that are used to measure and compare the results are indicated.

7.1 Model Setup

This thesis main aim is to provide a replication of the results of the original MRecuri study. To this end the study has been replicated as outlined in Chapter 5. Furthermore, several benchmark models that have been employed in the MRecuri paper are also replicated in this thesis project for results verification. In particular the benchmark models *ImplicitMF*, *Random model*, *Popularity model*, *Content-based model*, *GraphRec* and the *User-Track graph baseline* have been implemented and will be elaborated on in Chapter 7.2.

Additionally, as the second goal of this thesis, an extension effort has been made that implements the main concept of the MRecuri model in a newly proposed full Graph-Neural-Network approach, the GMR-ND. For hardware and software specifications used for the models, refer to Chapter 1.4. In the part below, the most important setup aspects of the replicated MRecuri model and the GMR-ND models are revised. Then, the benchmark models and their implementations are elaborated on.

MRecuri Model. The MRecuri model was replicated according to the steps taken in Chapter 5. Training of the model took approximately 2 hours while predictions took around 15 minutes. With a total of 2h15min, the execution time is considerably lower than the time of 6h15min in the original paper declaration. The model predictions were made over the full user-track matrix, but were trimmed down to the top 10 results for each user. The trimming was done in order to make the results comparable against the original MRecuri model results, since the original study only used the top 10 tracks for each users for the final results performance evaluation.

GMR-ND Models. As outlined in Chapter 6, the four different model BCE Shallow, CL Shallow, BCE Rich and CL Rich were implemented to test the impact of certain architectural choices. Training and validation for the shallow and rich models with binary-cross entropy loss took around 50 minutes each. Training for the models with custom loss functions took around 80 minutes each. This increase in training time can be explained by the additional pre-computed distances for the custom loss and was to

be expected. Predictions for each model took around one hour each. Model predictions were again made on the full user-track matrix, and trimmed down to the top 10 results for each user, to make the results comparable to the MRecuri replication.

7.2 Benchmark Models

Benchmark models were used in the MRecuri paper to compare the performances of the prediction models. Note, however, that these benchmark models were only described textually in the original paper and the ORP did not provide any code. This thesis aimed to rebuild all benchmark models, but due to the missing code of the original implementation, time constraints and implementation complexity, not all benchmark models were implemented. In an ideal scenario, all benchmark models used in the original study should be rebuilt and their results should be compared. Table 17 shows a summary of the replicated benchmark models and if they were able to produce prediction results.

Original Models	Replication Models	Prediction Results
MRecuri	MRecuri	Yes
Random	Random	Yes
Popularity	Popularity	Yes
Content	Content	Yes
ImplicitFM	ImplicitFM	Yes
GraphRec	GraphRec	No
MultVAE	/	/
Rank Aggregation	/	/
MMR-Inspired	/	/
VC	/	/

Table 17: Comparison of the models in the original and replication study. Column three shows if results could be replicated accurately

The random model and popularity model serve as non-personalized and trivial baselines. The content-based model, ImplicitFM and GraphRec represent state-of-the-art recommender system approaches.

Random model. The random model was implemented as a simple baseline for comparison with more sophisticated recommendation algorithms. For each user in the test set, the model randomly selected 10 unique tracks from the pool of available tracks. To simulate a ranking system, these randomly

selected tracks were assigned descending scores from 1 to 0.1. This scoring mechanism allowed the random recommendations to be evaluated using ranking-based metrics, despite their random nature.

Popularity model. The popularity model was implemented using a straightforward approach based on track popularity metrics derived from the user-track graph. The implementation was conducted by calculating two key popularity indicators for each track: the total number of scrobbles and the number of unique user appearances. To create a balanced popularity score, both metrics were normalized and then combined with equal weighting. This resulted in a single popularity score for each track, which accounted for both the frequency of plays and the breadth of user engagement. The tracks were then ranked based on their popularity score, and the top 10 most popular tracks were identified. The model’s recommendations were generated by assigning these top 10 tracks to every user in the test set. Unlike other models, this approach provided the same recommendations to all users.

Content-based model. The content-based model was implemented using a Word2Vec approach to create embeddings for tracks based on their associated tags. These track tags were then used to train a Word2Vec model, which created a 300-dimensional vector representations for each tag. For each track, an embedding was created by averaging the Word2Vec vectors of its associated tags. Similarly, user embeddings were generated by averaging the embeddings of all tracks the user had listened to in the training set. This approach allowed for a representation of both users and tracks in the same vector space. Recommendations were then generated by calculating the cosine similarity between each user’s embedding and all its track embeddings. The top 10 tracks with the highest similarity scores for each user were selected as recommendations.

Implicit FM. The ImplicitMF model is a collaborative filtering technique based on explicit feedback [28]. The model was implemented using the Alternating Least Squares (ALS) algorithm from the implicit GitHub repository¹⁸. The process began with data preparation, converting the training data into a sparse matrix format where rows represented users, columns represented tracks, and values indicated interaction strengths. A BM25 weighting scheme was then applied to this matrix to balance the influence of different user behaviors and item popularities. The core implementation involved initializing and training the ALS model with specific hyper parameters, including 64 latent factors, a regularization value of 0.05, and an alpha value of 2.0. These parameters were implemented according to the best practice

¹⁸<https://github.com/benfred/implicit>, last accessed 14.02.2025

methods in the documentation of the ImplicitFM model¹⁹. The model was trained for 50 iterations on CPU and took about 9 minutes to train. Once trained, it was used to generate top 10 track recommendations for each user in the test set.

GraphRec model. The GraphRec model is a GNN model for social recommendations [16] implemented according to the GitHub repository of the paper²⁰. The author of this thesis was able to replicate the model and adapt the code to the use case of a music recommender system but ultimately predictions could not be derived. While the training process was successful, it took around 4 hours per epoch, with a total of 40h training time on the train graph with the specified hardware. Even though, the training was carried out successfully and a model was saved, the predictions over the huge user-track matrix could not be executed due to size and memory issues. With an estimated time of over 360 hours for predictions, the prediction process would fail due to full GPU memory after about 30 minutes. To this end, no GraphRec prediction results were obtained.

User-track graph baseline. As in the original MRecuri paper, the user-track graph baseline should serve as a reference for novelty and diversity metrics regarding training and test set tracks. In this sense, the test set diversity indicates how *diverse* the users listening habits in the test set are. Therefore, a diversity score is calculated for each user in the test set over all tracks of that user. The final group diversity is then the mean diversity score over all test set users. On the other hand, novelty in the user-track graph baselines is calculated as the distance between training and test set. This was done in three steps. First, for each user their track structural and content-based embeddings were averaged once in the train set and once in the test set. This resulted in two user profiles per user, the train profile and the test profile. Then the Euclidean distance between this train profile and the test profile was computed which resulted in the respective structural and content-based novelty user scores. Finally, the individual user novelties were averaged over all users to derive the final group novelty scores.

7.3 Evaluation Metrics

This section elaborates on the four evaluation metrics that were used to compare the results of the introduced models. The two relevance-based metrics of precision and nDCG were used. In addition, novelty and diversity metrics were implemented to consider how well the models deal with filter bubble

¹⁹<https://benfred.github.io/implicit/>, last accessed 14.02.2025

²⁰<https://github.com/wenqifan03/GraphRec-WWW19>, last accessed 14.02.2025

avoidance. Note that a detailed implementation of novelty and diversity and their according structural and content-based embeddings has been outlined in Chapter 5.5. The definition in this chapter only summarizes these metrics and provides a short overview before the results section. All metrics are considered on the top@k recommendations, where $k = 10$.

Precision. Precision is a measure of the relevancy of the recommended items. It is defined as the number of relevant items (true positives) predicted by the recommendation system divided by the total number (true positives + false positives) of items predicted. In our context, the precision score is formulated in Equation 24 by the average precision across all users, where precision for each user is the proportion of recommended items that are relevant. Here, U is the set of all users, $R_u(k)$ is the set of top-k recommended items for user u and i is a track item. T_u is the set of relevant items for user u in the test set and k is the number of recommended items. As an example, if the model predicts 1 out of 10 songs correctly for user u , the precision@k score of u would be 0.1 .

$$\text{Precision@k} = \frac{1}{|U|} \sum_{u \in U} \frac{|\{i \in R_u(k) \cap T_u\}|}{k} \quad (24)$$

nDCG. The nDCG or *Normalized Discounted Cumulative Gain* is a relevance-based metric that takes into account both whether an item has been predicted correctly and the position of the recommendation in the ranked list. The nDCG score was implemented using SkLearn²¹. To illustrate how nDCG differs from precision, let's consider the following example. Precision only considers if a recommended track is actually a relevant track in the test set or not, regardless of its position in the recommendation list. The nDCG score, on the other hand, additionally considers the position of the relevant tracks in the top 10 recommendation list. If a correctly predicted track appears higher in the prediction list (i.e., has a higher prediction score), it contributes more to the nDCG score. This means that a recommendation list with relevant tracks ranked higher will achieve a better nDCG score than a list with the same relevant tracks ranked lower, even if the precision is identical.

Novelty. Novelty is a metric that assesses the model's ability to introduce users to new and potentially interesting items. This metric measures the dissimilarity between the recommended tracks and those tracks the user has previously interacted with.

²¹https://scikit-learn.org/stable/modules/generated/sklearn.metrics.ndcg_score.html, last accessed 14.02.2025

The novelty metrics are computed using both structural and content-based embeddings of the tracks. For each user, the process involves calculating the distances between the embeddings of the recommended tracks (top 10 predicted tracks) and those of the tracks in the user’s listening history (training set). Specifically, for each of the $k=10$ recommended tracks, pairwise Euclidean distances are computed between its embedding and the embeddings of all tracks in the user’s training set. The mean of these pairwise distances is then calculated to derive an average novelty score for each recommended track.

To obtain a single novelty score for a user, the mean of the novelty scores across all 10 recommended tracks is computed. This individual user-specific novelty score represents the average dissimilarity between the recommended tracks and the user’s historical preferences.

To assess the overall performance of the recommendation system in terms of novelty, the mean and standard deviation of these user-specific novelty scores are calculated across all users, providing a group-level novelty metric.

Diversity. The diversity metric assesses the variety among the recommended items, and evaluates if the model provides a broad range of options to users. In our context, diversity is calculated using both structural and content-based embeddings of tracks. The process begins by computing pairwise Euclidean distances between the embeddings of all recommended tracks for each user. This is done separately for structural and content-based embeddings. The diversity score for a single user is then calculated as the average of these pairwise distances, representing how different the recommended tracks are from each other.

To assess the model’s overall performance in terms of diversity, the mean and standard deviation of these user-specific diversity scores are calculated across all users. This process is performed twice: once using structural embeddings and once using content-based embeddings. This results in two sets of diversity metrics: structural diversity and content-based diversity. Higher diversity scores indicate that the recommendation system is providing a broader range of musical options, potentially exposing users to a more varied selection of tracks.

8 Results

This section presents the evaluation of results part of the thesis. All models, including the MRecuri replication model, the GMR-ND and all benchmark models had their predictions evaluated against the same test set on the following metrics: precision, nDCG, structural novelty and diversity and content-based novelty and diversity. As in the original MRecuri study, predictions of any models are considered *correct* if the predicted track appears in the test set. The four GMR-ND models are described in the results as BCE Shallow, CL Shallow, BCE Rich and CL Rich, where *Shallow* and *Rich* conform to the node features architecture and *CL* refers to *Custom Loss* and *BCE* to *Binary Cross Entropy*. The evaluations are furthermore carried out on only the top 10 predictions for each model. In this way, the replicated results are comparable to the original study results. Results are depicted in Figure 18.

Precision and nDCG. The evaluation of the recommendation models showed varied performance across the metrics. In terms of precision, the Popularity Model demonstrated the highest performance (0.038 ± 0.107), followed by MRecuri (0.014 ± 0.063) and the Content-based Model (0.009 ± 0.036). The best performing GMR-ND Model was the CL Shallow Model (0.008 ± 0.033) which followed on the fourth place. The Popularity Model also excelled in nDCG (0.101 ± 0.238), with MRecuri (0.037 ± 0.145) and the CL Shallow Model (0.037 ± 0.142) following closely.

Diversity. For content-based diversity, the Random Model showed superior diversity (6.293 ± 0.628), followed by CL Rich Model (6.261 ± 0.578) and BCE Rich Model (5.624 ± 0.474). The best performing GMR-ND model in this metric was CL Rich Model, while MRecuri (2.838 ± 0.075) scored notably lower. The CL Rich Model exhibited the highest structural diversity (0.917 ± 0.072), with the Random Model (0.884 ± 0.098) and BCE Rich Model (0.877 ± 0.080) following. The best GMR-ND model, CL Rich Model, performed competitively in this metric, while MRecuri (0.106 ± 0.007) showed room for enhancement in structural diversity.

Novelty. Regarding content-based novelty, the Random Model outperformed others (6.228 ± 0.536), followed by the CL Rich Model (6.155 ± 0.564) and ImplicitMF (6.073 ± 0.975). MRecuri (5.409 ± 0.976) performed reasonably well, ranking fifth in this metric. The best GMR-ND model, CL Rich Model, performed exceptionally well in content novelty. Finally,

for structural novelty, MRecuri demonstrated superior performance (0.862 ± 0.319), followed by CL Rich Model (0.830 ± 0.086) and BCE Rich Model (0.825 ± 0.094). The best performing GMR-ND model, CL Rich Model, showed competitive performance compared to MRecuri in this metric.

Model	Precision	nDCG	Content-based dissimilarities		Structural dissimilarities	
			Diversity	Novelty	Diversity	Novelty
MRecuri	0.014 $\pm$ 0.063	0.038 $\pm$ 0.146	2.838 $\pm$ 0.076	5.409 $\pm$ 0.977	0.107 $\pm$ 0.007	0.862 $\pm$ 0.319
ImplicitMF	0.005 $\pm$ 0.027	0.016 $\pm$ 0.088	4.265 $\pm$ 1.021	6.073 $\pm$ 0.975	0.291 $\pm$ 0.100	0.659 $\pm$ 0.210
Random	0.001 $\pm$ 0.008	0.003 $\pm$ 0.042	6.293 $\pm$ 0.628	6.228 $\pm$ 0.536	0.885 $\pm$ 0.098	0.825 $\pm$ 0.106
Popularity	0.039 $\pm$ 0.107	0.101 $\pm$ 0.239	2.199 $\pm$ 0.000	5.452 $\pm$ 1.048	0.119 $\pm$ 0.000	0.729 $\pm$ 0.254
Content-based	0.010 $\pm$ 0.037	0.036 $\pm$ 0.138	1.945 $\pm$ 0.685	4.149 $\pm$ 0.606	0.509 $\pm$ 0.206	0.526 $\pm$ 0.165
BCE Shallow	0.007 $\pm$ 0.027	0.029 $\pm$ 0.126	5.482 $\pm$ 0.636	5.878 $\pm$ 0.667	0.551 $\pm$ 0.101	0.672 $\pm$ 0.125
CL Shallow	0.009 $\pm$ 0.033	0.037 $\pm$ 0.143	5.372 $\pm$ 0.757	5.858 $\pm$ 0.697	0.537 $\pm$ 0.167	0.667 $\pm$ 0.137
BCE Rich	0.001 $\pm$ 0.007	0.002 $\pm$ 0.035	5.624 $\pm$ 0.474	5.928 $\pm$ 0.497	0.877 $\pm$ 0.080	0.826 $\pm$ 0.095
CL Rich	0.001 $\pm$ 0.008	0.003 $\pm$ 0.032	6.262 $\pm$ 0.579	6.156 $\pm$ 0.564	0.918 $\pm$ 0.072	0.830 $\pm$ 0.087
Original-User-Track-Graph	–	–	4.836 $\pm$ 0.782	1.225 $\pm$ 0.786	0.501 $\pm$ 0.162	0.160 $\pm$ 0.130

Table 18: Replicated recommendation model results. (Each metric’s top three results are indicated by first, second, third.)

8.1 Statistical Analysis of the Results

A statistical analysis was conducted to compare the results of the MRecuri model against the other models. Specifically, as in the original study, the MRecuri model is compared against all other models in a paired statistical analysis. This thesis used the Wilcoxon signed-rank test, results are depicted in Table 19, since the majority of the results data was not normally distributed. Each row in the table represents a different recommendation model being compared to the MRecuri model while the columns represent the evaluation metrics precision, nDCG, content-diversity, content-novelty, structural-diversity, and structural-novelty. The symbols in each row represent which model performed better, worse or if the performance did not differ significantly. The + indicates that MRecuri is significantly better than the compared model, – means that the compared model is significantly better than MRecuri and = indicates that no significant difference between MRecuri and the compared model exist.

Model	Precision	nDCG	Content-Diversity	Content-Novelty	Structural-Diversity	Structural-Novelty
ImplicitMF	+	+	–	–	–	+
Random	+	+	–	–	–	+
Popularity	–	–	+	–	–	+
Content	+	=	+	+	–	+
BCE Shallow	+	+	–	–	–	+
CL Shallow	+	=	–	–	–	+
BCE Rich	+	+	–	–	–	+
CL Rich	+	+	–	–	–	+

Note: + indicates significantly better than Mercury, – indicates significantly worse, and = indicates no significant difference ($p < 0.01$).

Table 19: Wilcoxon signed-rank Test Results Compared to Mercury

In precision and nDCG, the MRecuri model is outperformed by the popularity model significantly. On the other hand, MRecuri is significantly better than all other models, except the popularity model, in the precision score. In nDCG, the Content-based model and the Shallow CL GMR-ND model show no significant difference to MRecuri, while ImplicitMF, the random model, and the rich GMR-ND models perform significantly worse. The popularity model again performs significantly better than MRecuri in nDCG.

For content-based diversity, MRecuri has one of the worst performances, with almost all other models outperforming it in this metric, except for the Content-based model and the popularity model. The same pattern is observed for structural diversity, where MRecuri is significantly outperformed by all other models.

The content-based novelty score shows that MRecuri also performs significantly worse than all other models, except for the Content-based model.

In structural novelty, contrary to the content-based novelty, MRecuri outperforms every other model, making it the best model in this aspect.

In conclusion, while MRecuri maintains strong performance in precision and shows competitive results in nDCG, there is room for improvement in diversity metrics and content-based novelty. The GMR-ND models, while not outperforming MRecuri in precision and nDCG, are significantly better than the MRecuri model in all diversity and novelty scores, except for structural novelty.

8.2 Comparison of Replication Results and Original Results

In the previous section, the results of the replicated MRecuri model, the novel GMR-ND model and the replicated benchmark models are depicted. In contrast, this section compares the replicated results derived from the MRecuri replication model and the replicated benchmark models with the results of the original study that were outlined in Chapter 4. For this purpose, Table 20 shows a comparison of the original study results against the replicated models in each row. The highest performing model per metric in the original study is indicated in bold text, while the best replicated results are underlined. Note that a direct comparison is only possible for the relevance-based metrics precision and nDCG, while the novelty and diversity metrics in the original and replicated study differ in their magnitudes. This difference in magnitude between the novelty and diversity metrics in the replication study stems from the unclear textual definition of the novelty and diversity metrics in the original study. Therefore, the author had to replicate these embeddings and metrics, as explained in detail in Chapter 5.5. This replication based on only the textual description may have caused the replication to deviate in the creation of these metrics. In Chapter 10.1.1, Table 10.1.1, a compact overview about all deviations between the original and replicated study is given. To this end, novelty and diversity metrics can only be compared relatively to each other.

MRecuri Model. For the MRecuri model, the replication shows substantially lower performance in comparison to the original results in precision (0.014 ± 0.063 vs. 0.28 ± 0.198) and nDCG (0.037 ± 0.145 vs. 0.648 ± 0.227). While the absolute values for diversity and novelty metrics differ in magnitude, we can compare their relative performance to other models. In the original study, MRecuri outperformed ImplicitMF in content diversity, which is not the case in the replication. However, MRecuri’s structural diversity relative performance remains similar in both studies, being lower than

all other models. Content-based novelty is a strong feature of the original results which, however, is not observable in the replication, as the replicated MRecuri performs worse than the other models. Conversely, structural novelty is shows the same relative performance in the replication and original study, where in both cases MRecuri is the top performer.

Random Model. The Random model in the replication showed lower precision and nDCG. In terms of relative performance, the random model showed high diversity and novelty in both studies, often outperforming other models in these metrics.

Popularity Model. Apart from the relevance-based metrics discrepancy in precision and nDCG, the Popularity model’s relative performance in diversity and novelty remained consistent between the original and replicated studies, generally showing lower diversity but competitive novelty compared to other models.

Content-based Model. For the Content-based model, while Precision and nDCG were lower in the replication, its relative performance in diversity and novelty metrics remained largely consistent with the original study.

ImplicitMF. The ImplicitMF model showed the most consistent trend between the replicated and the original results in terms of relative performance. While its absolute values for Precision and nDCG were lower, its performance in diversity and novelty metrics relative to other models remained similar to the original study.

Model	Version	Precision	nDCG	Content-based dissimilarities		Structural dissimilarities	
				Diversity	Novelty	Diversity	Novelty
MRecuri	Original	0.28 $\pm$ 0.198	0.648 $\pm$ 0.227	0.471 $\pm$ 0.014	0.547 $\pm$ 0.031	0.128 $\pm$ 0.009	0.387 $\pm$ 0.092
	Replicated	0.014 $\pm$ 0.063	0.037 $\pm$ 0.145	2.838 $\pm$ 0.075	5.409 $\pm$ 0.976	0.106 $\pm$ 0.007	0.862 $\pm$ 0.319
Random	Original	0.101 $\pm$ 0.004	0.446 $\pm$ 0.201	0.496 $\pm$ 0.028	0.485 $\pm$ 0.023	0.224 $\pm$ 0.075	0.282 $\pm$ 0.045
	Replicated	0.0006 $\pm$ 0.007	0.002 $\pm$ 0.041	<u>6.293</u> $\pm$ <u>0.628</u>	<u>6.228</u> $\pm$ <u>0.536</u>	0.884 $\pm$ 0.098	0.824 $\pm$ 0.105
Popularity	Original	0.203 $\pm$ 0.133	0.581 $\pm$ 0.237	0.402 $\pm$ 0.017	0.46 $\pm$ 0.026	0.131 $\pm$ 0.015	0.279 $\pm$ 0.031
	Replicated	<u>0.038</u> $\pm$ <u>0.107</u>	<u>0.101</u> $\pm$ <u>0.238</u>	2.199 $\pm$ 0.000	5.452 $\pm$ 1.048	0.119 $\pm$ 0.000	0.728 $\pm$ 0.254
Content	Original	0.168 $\pm$ 0.124	0.522 $\pm$ 0.215	0.173 $\pm$ 0.062	0.388 $\pm$ 0.046	0.219 $\pm$ 0.061	0.247 $\pm$ 0.033
	Replicated	0.009 $\pm$ 0.036	0.036 $\pm$ 0.137	1.944 $\pm$ 0.685	4.148 $\pm$ 0.606	0.509 $\pm$ 0.205	0.526 $\pm$ 0.164
ImplicitMF	Original	0.366 $\pm$ 0.238	0.699 $\pm$ 0.226	0.376 $\pm$ 0.061	0.417 $\pm$ 0.042	0.146 $\pm$ 0.049	0.228 $\pm$ 0.036
	Replicated	0.004 $\pm$ 0.027	0.015 $\pm$ 0.088	4.264 $\pm$ 1.021	6.073 $\pm$ 0.975	0.290 $\pm$ 0.100	0.659 $\pm$ 0.209

Table 20: Comparison of original and replicated results for MRecuri and benchmark recommendation models. **Bold results** indicate the best result for the original models in each metric, while underlined results indicate the best replicated result per metric

8.3 GMR-ND Model Results

This subsection gives insights into the results of the GMR-ND model extensions implemented in PyG. In total four slightly different models based on shallow or rich node features and a binary-cross entropy (BCE) and a custom loss (CL) function with pre-computed distances were implemented as described in Chapter 6. Table 21 depicts the results of these four relevant models on the specified metrics. Bold values in the table depict the best results for each metric.

The CL Shallow Model shows the best results in relevance-based metrics: precision and nDCG. Both shallow models outperform the rich models in the relevance-based metrics. Conversely, in novelty and diversity metrics the CL Rich Model performs best, followed by the BCE Rich Model results. The BCE Shallow Model and CL Shallow Model perform worse in novelty and diversity than their rich counterparts. Explicitly the CL Rich Model shows the best performance over all four novelty and diversity metrics.

Statistical Analysis. To further validate the findings and extend the original papers analysis method of examining only the model performances against the MRecuri model, an additional statistical analysis comparing all models against each other, was conducted. First, the Kruskal-Wallis H-test²² was used to identify significant differences between models in each metric. Consequently, where the Kruskal-Wallis test indicated significant differences, we performed post-hoc pairwise Mann-Whitney U tests²³ to determine if there exist significance differences on a model-to-model basis. Figure 15 presents a comparison of all models across each metric, highlighting statistically significant differences between them.

The comparison shows a heat map figure for each metric: precision, nDCG, content diversity, content novelty, structural diversity and structural novelty. In each heat map, all models are compared against all other models. The symbols and colors indicate if a model has performed significantly worse or better, according to the mean values, than another model in the specified metric. The models are compared by row and column, where X indicates that the same model is compared. As an example, the graph can be read the following way: For the precision metrics, MRecuri in the first row is compared to the popularity model in column 4. The red minus quadrant indicates that MRecuri performed significantly worse than the popularity model in the precision score. Further on, the focus will be laid on analyzing

²²<https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.kruskal.html>, last accessed 14.02.2025

²³<https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.mannwhitneyu.html>, last accessed 14.02.2025

the results of the GMR-ND models against each other, the MRecuri model and the benchmark models.

For the precision metric it can be seen, that the shallow GMR-ND models do not perform significantly worse than the MRecuri model. The rich GMR-ND models, on the other hand, do perform significantly worse than almost all other models, except for the random model. It can also be observed that the rich models are significantly outperformed by their shallow GMR-ND counterparts. The results for the nDCG score is showing the same patterns, which are in line with the precision results.

For the content-based diversity metrics in the second row of heatmaps, there is a trend of the GMR-ND models outperforming the other models. Only the random model consistently stands out and performs significantly better, or in the case of the CL Rich GMR-ND, the same as the GMR-ND models.

In the content-based novelty metrics, the GMR-ND models again show a tendency to significantly outperform the other models, except for the Random and ImplicitFM model. In the last row, the structural diversity metrics are showing significant results in favor of the GMR-ND models as well, with the random model being the only benchmark to compete.

Finally, in the structural novelty metric, MRecuri significantly outperforms all other models including the GMR-ND models. However, GMR-ND models still show a strong performance compared to all other benchmarks.

Overall, the statistical analysis results align with the results from Chapter 8, showing that GMR-ND show promising results in novelty and diversity metrics. Even in the relevance-based metrics of precision and nDCG, the shallow models do not perform significantly worse than other benchmarks in most cases and show that they are not significantly worse than the MRecuri model in terms of precision and nDCG. However, especially the rich GMR-ND models, lack performance in relevance-based performance.

Training and Validation loss. Furthermore, Figure 14 gives insight into the training and validation loss for the four different models. Note that the magnitude of the scales differs for the custom loss and the BCE loss. On the left hand side the rich models are displayed, whereas the right side shows the shallow models. The upper two figures represent the BCE loss, while the figures at the bottom show the custom loss models. The training loss is specified with the blue line and the validation loss with the orange line. Generally, a large gap between the training loss and the validation loss is visible in all four cases.

When observing the training losses of the models, it is notable that the BCE loss models decrease their training losses only minimally throughout the training process of 10 epochs. The BCE Shallow Model starts with a training

loss of 0.205 and ends with a training loss of 0.185 in epoch 10. Similarly, the BCE Rich Model start with 0.607 and ends with 0.606. Conversely, the CL models decrease their training losses more notably. The CL Rich Model starts with a training loss of 112.866 and ends with 109.87. The CL Shallow Model start 67.55 and goes down to 32.35 showing the biggest relative training loss over all models.

On the other hand, the validation losses show a different behavior. Apart from showing a wide gap between validation and training loss, the validation loss furthermore does not decrease. The only exception is the CL Rich Model where a general trend of a decreasing validation loss is visibly. Most notably, for both shallow models, the validation loss increases starting from epoch 1.

Model	Precision	nDCG
BCE Shallow	0.007 ± 0.027	0.029 ± 0.126
CL Shallow	0.009 ± 0.033	0.037 ± 0.143
BCE Rich	0.001 ± 0.007	0.002 ± 0.035
CL Rich	0.001 ± 0.008	0.003 ± 0.032

Table 21: Results of GMR-ND model variations - Accuracy metrics. **Bold results** indicate the best performing model per metric

Model	Content Diversity	Content Novelty	Structural Diversity	Structural Novelty
BCE Shallow	5.482 ± 0.636	5.878 ± 0.667	0.551 ± 0.101	0.672 ± 0.125
CL Shallow	5.372 ± 0.757	5.858 ± 0.697	0.537 ± 0.167	0.667 ± 0.137
BCE Rich	5.624 ± 0.474	5.928 ± 0.497	0.877 ± 0.080	0.826 ± 0.095
CL Rich	6.262 ± 0.579	6.156 ± 0.564	0.918 ± 0.072	0.830 ± 0.087

Table 21: (continued) Results of GMR-ND model variations - Beyond-accuracy metrics. **Bold results** indicate the best performing model per metric

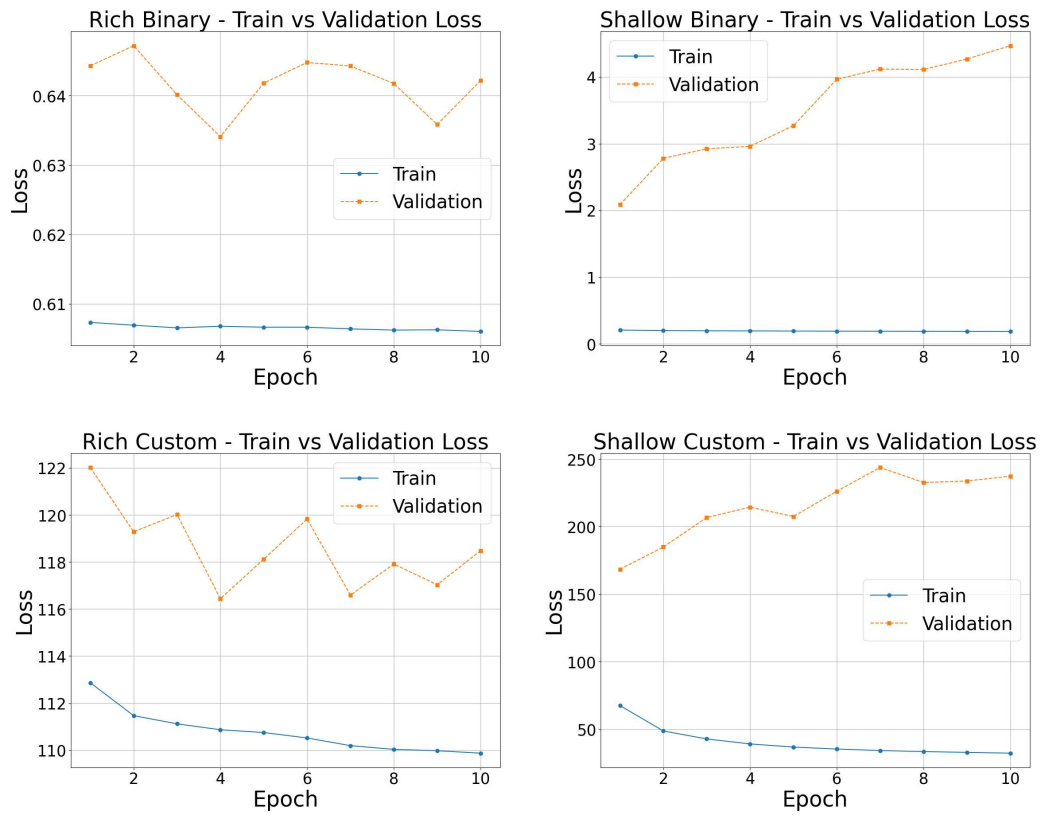


Figure 14: Training and validation loss of the shallow and rich GMR-ND models.

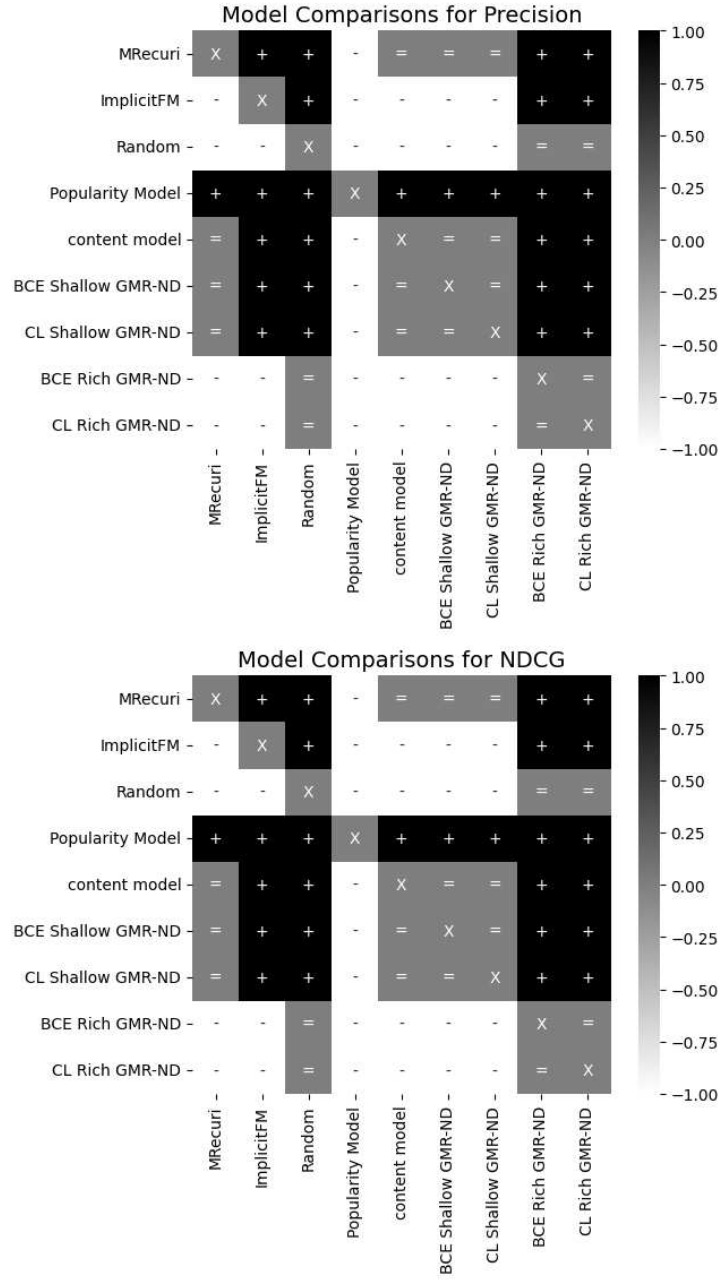


Figure 15: Statistical analysis of the performance of each model against all other models for each metric. A plus (black background) indicates significantly better performance than another model, - (white background) indicates significantly worse performance, = (grey background) indicates the models are not significantly different and X (grey background) indicates the model itself. This page shows precision and nDCG for all models, other metrics follow on the next pages

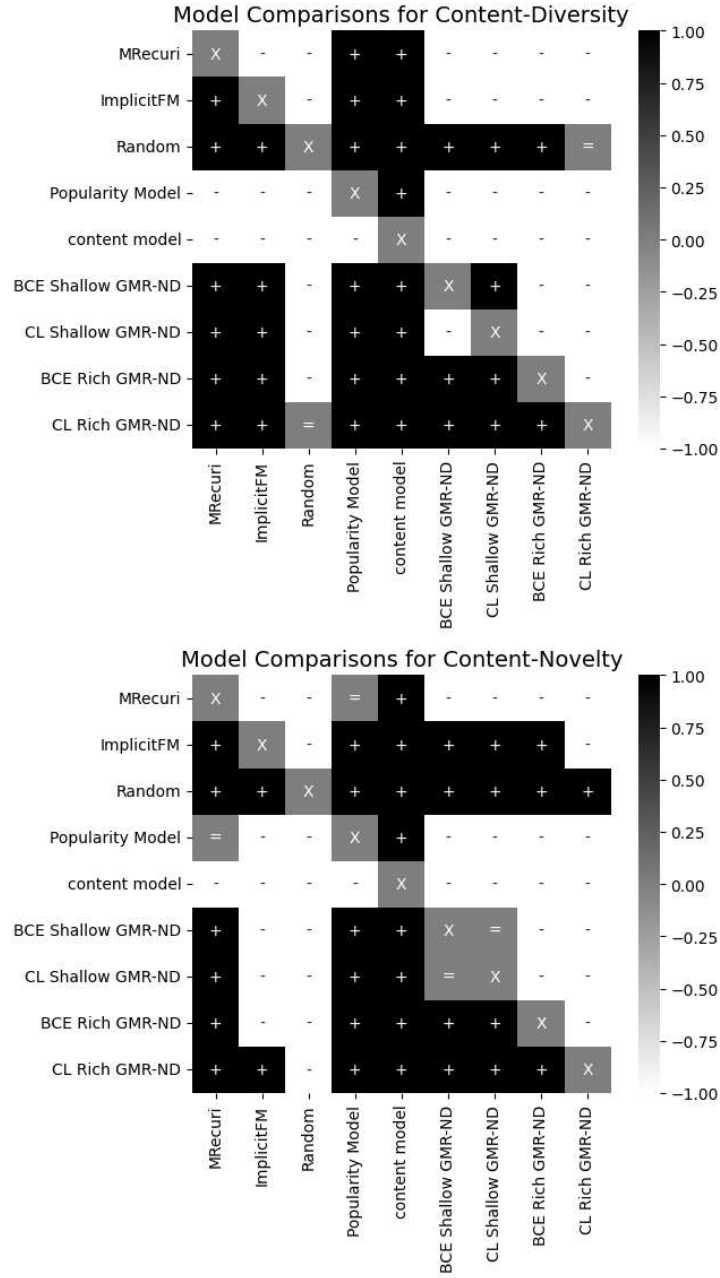


Figure 15: (continued) Statistical analysis of content-based diversity and novelty comparison for all models

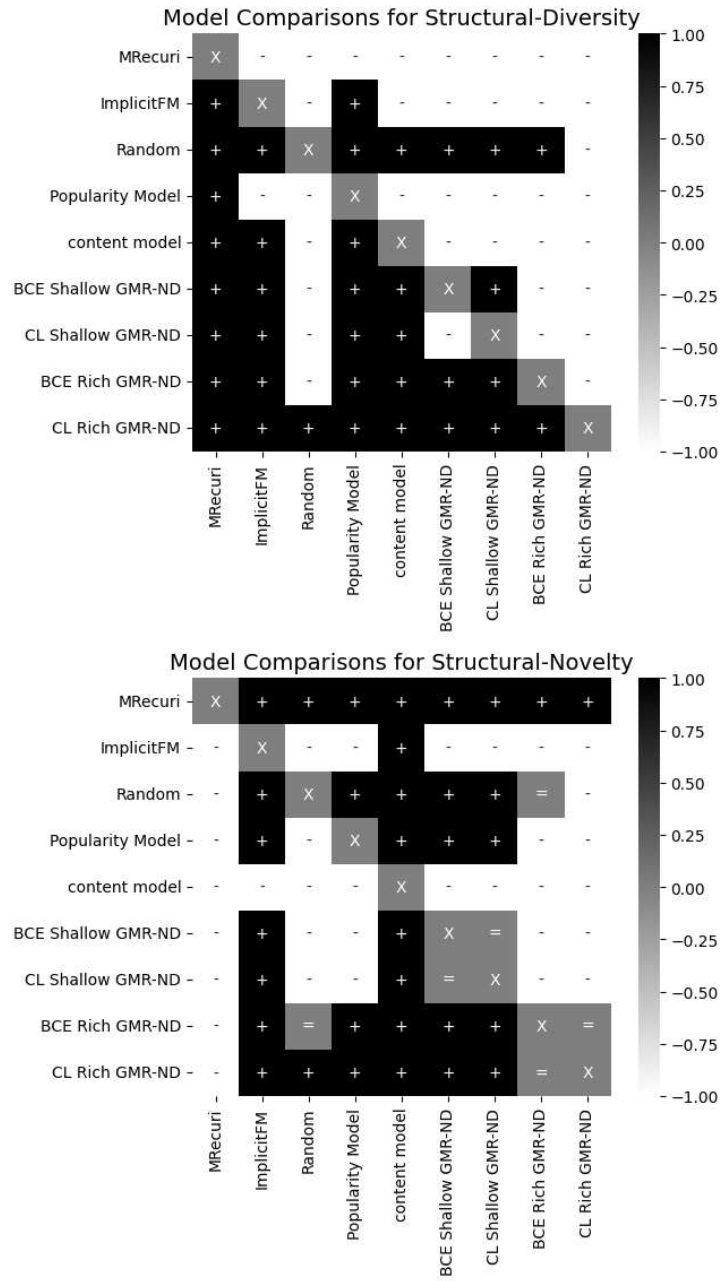


Figure 15: (continued) Statistical analysis of structural diversity and novelty comparison for all models

9 Threats to Validity

The following section presents threats to the validity of the presented results. Internal validity, external validity and construct validity are discussed and their implications on the findings and possible mitigation's are provided.

9.1 Internal Validity

The following subsection discusses threats to internal validity. The internal validity threats indicate that following aspects may compromise the validity of the original and the replicated results. Internal validity threats that were observed in the original study were mitigated in the replication model and novel GMR-ND approach when possible.

9.1.1 Data Quality

The replication of the original study uncovered several issues with the provided data, as detailed in Chapter 5.1.2. These problems have potential implications for the reliability and validity of the model's performance and results.

One key issue identified concerned the temporal split between the training and test sets. While the original intention was to divide the data based on listening time attributes, this approach was compromised by a substantial amount of missing temporal information. As a consequence, the majority of data lacking time attributes was defaulted to the training set. This could lead to a situation where, for some users, the actual listening order was not preserved, potentially resulting in a random split between training and test sets rather than a true temporal division.

Another concern emerged regarding the presence of duplicate artists in the graph data. This data-integrity problem could potentially distort the model training process and subsequent predictions, as the model might overemphasize certain artists due to their unintended repetition in the dataset.

These data quality issues could have consequences for the model's performance evaluation. The compromised temporal split, in particular, might have allowed the model to train on data that, in reality, should have been part of the test set. This scenario could lead to wrong performance metrics and reduce the model's ability to generalize to truly unseen data.

While the appearance of multiple artists could be pre-processed and mitigated for the replication MRecuri model and GMR-ND model, the temporal split problems remained and could not be circumvented.

9.1.2 Popularity Model

Furthermore, the popularity model shows odd behavior in their diversity metrics. The popularity model selects the same 10 tracks based on a popularity score to all users. Given the definition of the diversity metric in Equation 9 on page 70, the diversity is computed by first taking the pairwise distance between all 10 recommended tracks and then averaging these 10 pairwise distances to receive the individual diversity score for each user. In a popularity-based model, where all 10 recommendations are the same for all users, these user diversities should therefore all be the same. Hence, taking their mean over all users does not change the results and the standard deviation should be equal to 0. This expected behavior is correctly depicted in the replicated results in Table 18 on page 101 with content diversity ($2.199 \pm \mathbf{0.000}$) and structural diversity ($0.119 \pm \mathbf{0.000}$). However, it is not the case in the results of the original study as seen in Table 5 with content diversity mean and standard deviation being equal to ($0.402 \pm \mathbf{0.017}$) and a structural diversity of ($0.131 \pm \mathbf{0.015}$). This further puts the validity of the original results into question.

9.1.3 Novelty and Diversity Embeddings

The potential deviation from how the original study implemented the structural and content-based embeddings poses a threat to the derivation of the novelty and diversity metrics. If the embedding creation process deviates slightly, this could lead to different metric magnitudes that are likely not comparable. To this end, due to the unavailability of the evaluation metrics in the original study, the derivation of the replicated evaluation metrics, specifically the novelty and diversity metrics and their associated embeddings, likely differ from the originally employed metrics based on the results as seen in Table 18. The magnitude between content-based embeddings of the original study and the replication, as seen in the results, supports this assumption. This issue stems from the missing and ambiguous definition of these metrics in the original study. To effectively compare the results from the original study and the replicated study, ideally the same implementation of the evaluation metrics should be used.

9.1.4 Benchmark Models

In a similar fashion, the implementations of the benchmark models are crucial to compare the results of the models performances in the original and replicated study. Deviations of the exact implementations of the benchmark models can lead to different results when replicating them. The benchmark

models that are used to compare against the MRecuri results are not available in the ORP. This, again, may lead to a deviation in the ARP code and may influence the validity of the results. To this end, the replicated benchmark models also differ in their relative results compared to the original results.

9.1.5 FRediECH Model Definitions

In the original study, the main architectural concept and many features of the model like pre-computed distances and the customized loss function stem from the FRediECH paper [59], by the same authors as in the original MRecuri study [60]. As outlined, some concepts used in FRediECH are often implemented in the exact same manner in the MRecuri model. For example the loss function and the definition of the GCN model part. Furthermore, the metrics used to measure results are also the same with precision, nDCG, novelty and diversity.

As outlined in Chapter 5.5, the evaluation metrics and their defined embeddings had to be replicated based on the textual description of the MRecuri paper. The original MRecuri description of the *user embeddings* that are used to derive novelty and diversity metrics is as laid in Chapter 5.5 was not possible, conversely, *track embeddings* are needed. We speculate, that this doubtful original definition of needed user embeddings is due to the fact that this definition was copied from the FRediECH paper. In the FRediECH paper, those user embeddings are correct since they refer to the items, but in the MRecuri context, items refer to tracks.

Regardless of the reason for this potential definition mix-up, the MRecuri paper should specify the implementation of the evaluation metrics more precisely with less ambiguous text and precise equations. Furthermore, to mitigate such unclear definitions, the evaluation metrics set up should also be included in the ORP. Lastly, it should be noted, that the FRediECH paper also had no traces of evaluation metrics set up in their replication package.

9.2 External Validity

The external validity refers to the generalizability of the study’s findings. Threats on the External Validity for the original model, the replicated model and the GMR-ND model are, therefore, the same. For one, the recommender system is focused only on music recommendations for registered users at LastFM. These users potentially have a high interest in music and may therefore not be comparable to the usual music streaming consumer. Furthermore, the cultural background of the users is not transparent, potentially limiting the results to certain user demographics. In addition, the studies only per-

ceive a relatively small set of users and tracks. Whereas, in a real-world scenario millions of both users and songs exist. This limits first of all the significance of the results on diversity and novelty and furthermore undermines the usability of these models in terms of scaling to a real-world scenario.

In comparison to the related works on more diversity and novelty in Chapter 3.2, this research is only conducted on one music dataset, while the related work of [56, 57] compare the performance of their models on three different datasets each. Similarly, their datasets contain user-item interactions in the millions rather, while in this thesis only a two hundred thousand interactions are researched. Note, that these works do not focus specifically on music recommendations.

The related works which are focusing specifically on music recommender systems with GNNs in Chapter 3.1 also often work with at least two different datasets and with a higher number of user-song interactions. This shows that the focus of this thesis might be too narrow and that the results should be verified on more and bigger datasets.

9.3 Construct Validity

In scientific works not all concepts can be measured easily by their own definition. A concept that can not be measured directly can be made measurable by employing certain indicators at an operational level. This process of using indicators to measure these concepts, is called a *construct*. Subsequently, the construct validity defines how well a used construct actually measures the concept it represents [51].

Sjøberg et al. [51] identify three primary threats to construct validity: inadequate definition of a concept, construct underrepresentation, and construct-representation bias. Construct underrepresentation occurs when the indicators are too narrow or imbalanced to capture all important aspects of a concept. Construct-representation bias happens when indicators either fall outside the concept’s scope or systematically misrepresent the concept through measurement errors.

The threats on the construct validity regarding the three concepts *music metadata*, *user interaction* and *user evaluation* in the context of the proposed music recommender models will be discussed and categorized according to the definition above. Since the original model, the replicated model and the extension model use the same data and evaluation methods, these threats apply to all of them.

Music metadata. While the general concepts of music metadata are defined at a detailed level with data on song names, song tags, song audio features and artist names, these concepts could be defined in even more detail

to ensure their validity. One could for example include more elaborate data on the relations between artists. In the case that one song has multiple artists, the collaboration relations between those artists could be depicted in the graph data. In the construct of songs, more information on the song itself could be included by adding information on lyrics, instrumentation or land of origin. By having only limited information on music metadata, some relations between artists and songs may be lost and can potentially impact the quality of results. Therefore, this threat can be categorized as construct underrepresentation. The addition of such auxiliary information could potentially improve the model, while keeping the basic architecture in place.

User interaction. The construct that represents the concept of user interactions is currently only defined by the number of times a user has listened to a song. Hence, the construct could be more refined with more explicit indicators like save rates (if a user has saved this track to his library), playlist adds (if the user has put this song into a playlist) and skip rates (if a song is skipped very early on into the song). The lacking of such information may limit how well the model can learn how users interacted with songs in real life, since only basic interactions are displayed. For example, it is possible that a user listened to a song by accident and skipped it after only a few seconds. In the model however, this song will have the same weight as any other song the user has listened to. By using only a listening count, the construct may not accurately represent user interaction and can hence be categorized as construct-representation bias.

User evaluation. Lastly, the evaluation methods of the studies are limited to an offline setting without real user feedback on the recommendations. The assumption that users would listen to all recommended songs for the evaluation is unlikely for a real-world scenario. This limits the meaningfulness of the results and the metric scores. Especially, novelty and diversity are heavily simplified in this context, as they have not been validated in any user-centric evaluation. There is no evidence, that a good novelty or diversity metric in the model also translates to an increase of novelty and diversity in a real-life recommendation setting. Therefore, this construct threat can be categorized into the threat of construct underrepresentation. To this end, in the future more refined evaluation steps on the effectiveness of recommended songs in terms of relevance, novelty, and diversity scores should be considered.

10 Discussion

10.1 Limitations

10.1.1 Deviations from the Original Study

In face of the limited information that could be derived from the original MRecuri study, some deviations in the replication efforts could not be mitigated. To this extent, especially the missing access to parts of the original input data of the MRecuri model and the missing implementation details of the evaluation metrics and benchmark models were the biggest limitation in deriving the original results as close as possible.

Table 22 summarizes the above issues and shows a summary of the missing code and data artifacts from the original MRecuri study, which in turn may have lead to deviations in the replication process. The column *Category* describes the kind of missing information, *Missing Artifact (Original)* shows what exactly is not provided by the original study and *Deviation Implementation (Replication)* explains how this problem was circumvented and consequently how it was implemented in the replication model. The third column also includes a reference to the part of this thesis that deals with the implementation of this specific issue in depth. The following paragraphs describe the overview of the table in more detail.

First, the provided data did not include the relevant audio tags and audio features which are needed for processing by the MRecuri model. Since they were vital for the model however, they had to be re-collected, as explained in Chapter 5.1.3. It is possible, even likely, that the re-collected audio feature data does not completely match the data used by the original authors and therefore may be a factor for results deviation.

Furthermore, the original data was missing a majority of the time / position data, which was relevant in order to perform a meaningful time split into training and test data. This fact could not be changed, and it was not possible to retrieve the time data for the relevant users. Furthermore, it also would not have been meaningful to recollect or impute time data in this context, since it would have changed the distribution of the data into training and test data and would have caused a discrepancy between original and replicated splits. Therefore, the same split of training and test data as in the original study was used. Note that the time attribute was only used in this context of the split and did not contribute to the model in another way.

The input data which should have been used to run the MRecuri model in the ORP notebooks 3 and 4 was connected to a database that we had no access to. Instead of the database data the provided pickle files combined

with the newly collected audio tags and feature attributes were used to train and run the model in the replication. This led to several adaptations in the original source code due to uncertainty of their originally used database data structure. This fact may have contributed to a deviation in results. For more details on the replicated implementation of training and predictions, refer to Chapter 5.4.2.

The originally provided notebook 4, which is the prediction part of the model, was not executed visibly and therefore no conclusions about the results of that code part could be derived. Notably, the author did expect to find information on how the predictions were made and how the final structure of the predictions data looked like before the evaluation. This would have helped to ensure a similar or matching data structure before the evaluation.

Finally, the creation process of the diversity and novelty metrics and the content-based and structural embeddings was not provided as a code artifact. Only a textual description about them was given in the original paper. Similarly, the benchmark models, while described textually in the original paper, were not included in their source code. Therefore, the creation of diversity and novelty metrics, the evaluation itself and the benchmark models were implemented on a best-effort approach using the textual description of the original paper. For the replication of the novelty and diversity metrics refer to Chapter 5.5, for the evaluation process to Chapter 7.3 and for the replication of the benchmark models to Chapter 7.2.

All of the above mentioned adaptations may have led to deviations from the original implementation and may be the cause for the difference in results between the original and replicated model, which will be discussed more in Chapter 10. Overall, the replication process was conducted on a best-effort basis, where intermediate results and code steps were continuously compared against the original work to ensure the closest possible adherence to the original implementation while taking into account the drawbacks of missing information.

Category	Missing Artifact (Original)	Deviation Implementation (Replication)
Data features	Missing audio tag data	Re-collecting of audio tag data, see 5.1.3
Data features	Missing audio features data	Re-collecting of audio features data, see 5.1.3
Data features	Missing time data attributes	No workaround, see 5.1.2
Input Data	Missing access to model input data	Code Adaptation, see 5.4.2
MRecuri Model	Notebook 4 did not show any print outs	/
Evaluation Metrics	No implementation code given	Implementation based on textual description, see 5.5
Evaluation Results	No implementation code given	Implementation based on textual description, see 7.3
Benchmark Models	No implementation code given	Implementation based on textual description, see 7.2

Table 22: Deviations between original and replicated implementation due to missing artifacts

10.1.2 Scraping and Imputation Process

During the collection process of the missing tag and audio features data from LastFM and the Spotify API, certain data could not be collected. As laid out clearly in Chapter 5, for certain tracks tag and audio features could not be derived and hence their values were imputed based on the described methods in the respective section. These imputation methods could potentially alter the training and prediction process. Even though their impact is assumed to be minimal since the number of imputed values is rather low, they could account for some deviations between the replicated and original study results.

10.1.3 Limited Hardware Resources

Most models that were used in this thesis project did run and produce results within reasonable time given the specified hardware in Chapter 7. One benchmark model however, the GraphRec model, could not produce the final benchmark predictions due to long execution time. Even though, the author assumes this problem to be accountable to the GraphRec design, hardware upgrades could potentially also solve this problem.

10.1.4 Communication with the Original Authors

As laid out in Chapter 5, a reach-out process to the original authors was conducted to receive supplement information on the original study. In an ideal scenario a close collaboration between original and replication authors would be suitable to replicate and verify the original results. However, the communication was not possible and no further information from the original authors could be gained.

10.2 Discussion of Results

The following section discusses how the obtained results in Chapter 8 can be interpreted and what conclusions can be made based on them. To this end, first, the replicated results will be discussed. Second, the original and the replicated results are compared and lastly the novel GMR-ND extension results are discussed. The original study results are not repeated here and can be found in Chapter 4.

As indicated in the related work Chapter 3.3, several studies [6, 17] have established that there exist problems in the replication process of recommender systems. The replication process in this thesis supports these findings of difficulties in the reproduction process, especially concerning the lack of availability and quality of replication material such as source code and

evaluation documentation. Furthermore, also in line with the findings of the related works, the reach out process to the original authors for collaboration on the replication process remains a challenge.

10.2.1 Replication Results

In the replication results, as a general observation, while the Popularity Model excelled in precision and nDCG, other models like the Random Model and CL Rich Model showed strengths in diversity and novelty metrics. The MRecuri model demonstrated competitive performance across most metrics, particularly excelling in precision and structural novelty. The GMR-ND models, particularly the CL Rich Model, showed promising results in the diversity and novelty metrics ranking in the top 3 in all four metrics. On the other hand the GMR-ND models, especially the rich model variations, lagged behind in precision and nDCG scores, suggesting potential for further optimization and improvement in their architecture for relevancy-based metrics.

10.2.2 Original and Replicated Results

By comparing the original study results with the replication, several notable differences emerged across the various recommendation models and evaluation metrics. Most notably, a clear trend is visible in relevance-based metrics which show that the replicated model performances are generally at least one to two decimal places worse than the original model. These discrepancies highlight a need for careful consideration of the original results and may provide insights into the robustness and generalizability of the original findings. These significant decreases in accuracy metrics may suggest potential differences in the original and the replicated model implementation or differences in dataset characteristics like tags or audio features data, as shown in the overview Table 22 in Chapter 10.1.1.

In addition, Table 23 shows a comparison of the original and replication study. Specifically, it shows which models were replicated and if the original results could be confirmed by the replication study. The MRecuri model and the benchmark models Random, Popularity, Content and ImplicitFM were successfully replicated, with modifications necessary as stated in Chapter 10.1.1 and were able to deliver predictions. The GraphRec model was implemented, but did not deliver predictions due to a huge computational cost, refer to Chapter 7.2 for more detail. The benchmark models MultVAE, Rank Aggregation, MMR-Inspirec and VC were not implemented since their implementation effort were out of scope for this master thesis. While the replicated models were able to produce predictions, non of the predictions

could be replicated close to the results in the original study. Refer to the results in Chapter 8 for a detailed comparison of the the original and replicated models and their metrics.

Original Models	Replication Models	Matching Results
MRecuri	MRecuri	No
Random	Random	No
Popularity	Popularity	No
Content	Content	No
ImplicitFM	ImplicitFM	No
GraphRec	GraphRec	/
MultVAE	/	/
Rank Aggregation	/	/
MMR-Inspired	/	/
VC	/	/

Table 23: Comparison of the models in the original and replication study. Column three shows if results could be replicated accurately

10.2.3 Original and Replication Result Deviations

This subsection discusses how the discrepancies in the original study results and the replicated results may be explained. To this end, specifically the big difference in accuracy metrics between the original, the replicated and the benchmark models show a big deviation. Across all models, the original precision and nDCG metrics show values between 0.1 and 0.36, while the values in the replicated range from 0.0006 to 0.038. This would indicate that the worst model in the original study still performs around three times better than the best replicated model. This discrepancy in value magnitude may be traced back to the original model and their results. Even arguing that the replication MRecuri model implementation was flawed and its results are worse than the original model, it is hard to argue that also all other benchmark models, plus the novel GMR-ND approach, would be so erroneous as to deviate from the original results in such a magnitude.

Furthermore, to argue against the possibility that the replicated MRecuri model was implemented incorrectly by the author of this thesis, is the fact that the model training in the original and replicated code packages ORP and ARP exhibit very similar behavior and output. Specifically, the output of the training loss during the training process is available as printout in

the ORP and the ARP and shows very similar numbers. Hence, the overall model weights should exhibit similar parameters and the models should perform equally well, if the prediction and evaluation process stay the same. Unfortunately, since the prediction process and the evaluation process is not available in the ORP, it could not be ensured that these replicated processes match the original ones. In conclusion, even though different data features and input data was used in the replication study, the deviation of the original code remained as small as possible. This makes implementation mistakes on the replication code improbable to cause the discrepancies between original and replicated results.

10.2.4 Replicated MRecuri Predictions

The replication results show a strange behavior of the replicated MRecuri model as well. In particular, the predictions only cover a very narrow variety of predicted tracks. Over the test prediction set of 155,849 tracks, only a total of 59 different tracks were predicted over all 32,790 tracks (10 tracks x 3,279 users) that were recommended to the users. This limits the personalization impact of MRecuri and is counter-intuitive to its explained behavior to break individual users out of the filter bubbles. Filter bubbles and user consumption patterns would be expected to have rather different patterns, and hence personalized recommendations would be needed.

In comparison to the benchmark model and some GMR-ND models, the replicated MRecuri model shows a significantly lower number of unique tracks over their recommended sets as depicted in Figure 16. The specified 59 make the recommendations seem un-personalized, similar to the recommendations of the popularity model. This suggests either an implementation error of the replicated model or an erroneous model architecture design in the original study.

10.2.5 GMR-ND Results

Loss Function Comparison. The following part will discuss the impact of the custom loss function (CL) on the metrics in comparison to the BCE loss function. In this way, the goal is to analyze if the claimed attribute of the CL being advantageous for escaping the filter bubble can be supported in the GMR-ND models.

When comparing the performance of the loss function variations of GMR-ND models against each other, the CL Shallow Model shows the best results in relevance-based metrics precision and nDCG. Therefore, it outperforms its BCE Shallow Model counterpart. Similarly, the CL Rich Model performs

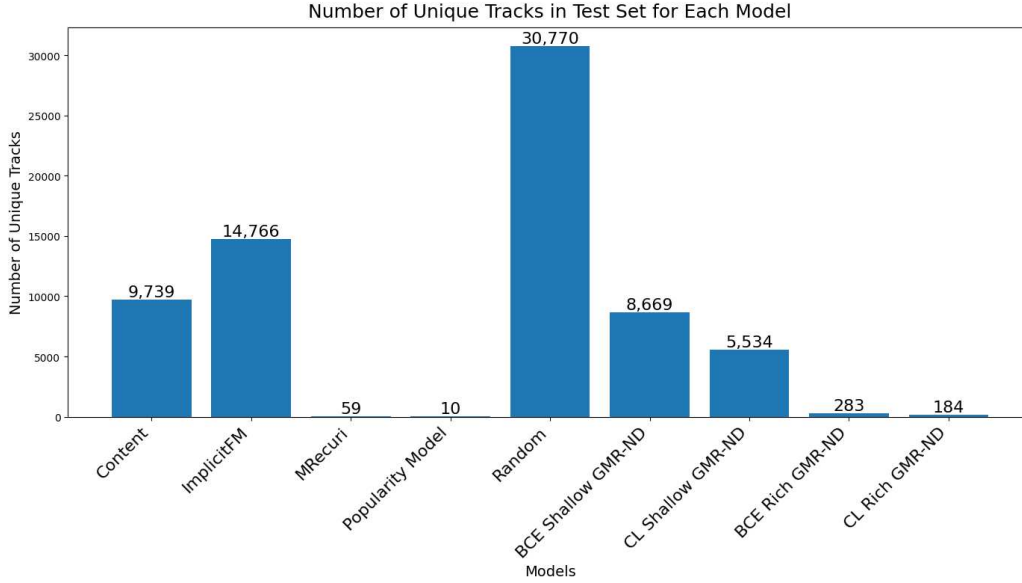


Figure 16: Number of unique tracks in the prediction set per model.

slightly better than its counterpart, the BCE Rich Model in the relevance-based metrics.

However, looking at the significance analysis from Table 15, it is observable, that these differences in precision and nDCG in the respective BCE and CL model variations is not significant. This suggests, that even though the CL function results were slightly higher in the relevancy-based metrics, they are not significantly better. Hence the custom loss function seems to have negligible impact on the relevance metric performance.

Furthermore, when comparing the novelty and diversity metrics between custom vs non-custom loss function, the picture is less clear. While the CL Rich Model performs better on all four novelty and diversity metrics than its BCE Rich Model counterpart, results between CL Shallow Model and BCE Shallow Model differ only very slightly, with no clear model outperforming the other. Even though, the statistical analysis in Figure 15 shows that the CL Rich Model performs significantly better than the BCE Rich model, the CL Shallow Model is not significantly better than the BCE in the novelty and diversity metrics. To this end, the claimed attribution to the custom loss function of reducing filter bubble impact, that is made by the original MRecuri study, can not be verified in the GMR-ND model variations.

Node Features. This part will discuss whether the rich vs shallow node features in the architecture design had significant impact on the results. The shallow models show higher scores in precision and nDCG than the

rich models, Table 21, which are also significant according to the statistical analysis in Figure 15.

On the other hand, the novelty and diversity metrics of the rich models are performing better and are also significantly different from the shallow models. However, looking at the significance in relevance-based metric results of the rich models, their performance is similar and not significantly different from the performance of the random model. This suggests that while their potential to recommend novel and diverse songs is high, their quality of recommendations in terms of precision is so low, that they can be compared to a random model. This limits their usability in a real life scenario and highlights the need for more elaborate architectures in the rich node GMR-ND models.

The shallow GMR-ND models, on the other hand, while not showing quite the same performance on novelty and diversity as the rich models, still outperform all other MRecuri and benchmark models significantly in novelty and diversity metrics. Furthermore, they also do not significantly lack in relevancy-based metrics. Hence, it can be stated that the shallow GMR-ND models strike a good balance of recommending relevant songs, while increasing the novelty and diversity aspect compared to state of the art benchmark models.

Training and Validation Loss. As indicated in Chapter 8.3, the large gap between training and validation loss may indicate over-fitting. When looking at the rather poorly performing precision metric results of the GMR-ND models, the argument for over-fitting can be supported. Note, however, that the validation set and the test set / prediction set differ significantly. While the training set and the validation set consist of 2.3 million and 256 thousand edges, respectively, the prediction set consists of over 500 million edges. Hence, even though over-fitting on the training data is indicated in the validation loss, the relation to the prediction (i.e. test) set must be considered with care.

Furthermore, as aforementioned in Chapter 8.3, the unusual behavior of the validation losses highlights the need for more refinement of the GMR-ND architecture.

11 Conclusion

The main goals of this thesis were to replicate the study of Tommasel et al. [60] and extend it with a new full graph-based model. In this regard, the research questions as defined in Chapter 1 can be answered as follows. Research question one asks if the original results can be replicated accurately. As outlined in this thesis, the original results are not replicable to the same extent as in the original study. The replications process was conducted on a best effort basis, but due to missing implementation details, unclear definitions and missing code parts in the original study, assumptions and modifications had to be made in the replication process. Therefore, potential deviations in the results of the replicated and original study are possible.

On the other hand, research question two can be answered positively, by showing that an original model adaption and extension in a fully graph-based GNN model, implemented with PyTorch Geometric, was possible. Further, the obtained results by the extension model show promising potential of full GNN-based models and the PyGeometrics library. Especially the shallow GMR-ND model variations are able to compete with benchmark models and MRecuri in relevance-based metrics and improve performance on diversity and novelty metrics.

In conclusion, the results show that a replication of the original model was only possible with modifications to the originally provided code and data material and that replicated results deviate from the original results. The deviation of the results between replication and original study raise questions about the validity of the methods used in the original study and about the credibility of the original results. Further, it shows the need for future studies, especially in the music recommendation field, to make results and replication material more transparent and accessible.

Furthermore, the testing of the impact of certain design choices, that was investigated with the implementation and comparison of the four different GMR-ND models, shows that more auxiliary information in node features, does not necessarily produce better results. In fact, the results on the relevancy-based metrics suggest the contrary, making simpler GNN models an important future research perspective. Conversely, the implementation of a custom loss function, as suggested in the original MRecuri model, that aims to prevent the creation of filter bubbles and yields more diverse and novel results, shows no clear improvement when implemented in the GMR-ND model.

Finally, in terms of results comparison on the replicated MRecuri, benchmark and GNN models, the results show that the MRecuri model and the extension GMR-ND model can both improve in various aspects. While MRecuri

performed well in relevancy-based metrics, the GMR-ND model performed better on novelty and diversity metrics. In this regard, the GMR-ND model can be improved from its relatively basic implementation in this thesis and shows promising results for novelty and diversity for future research in the field. It is evident, that while GNN models show promising aspects, that more research and optimization needs to be done in order to come up with recommendation models that are producing optimal results in both relevancy and diversity-based metrics.

Future research in the field of music recommender systems and specifically work building on this thesis could focus on improving communication with original authors, improving the GMR-ND architecture, make standardization efforts for novelty and diversity metrics, employ user-centric evaluation methods and use different and bigger datasets for results verification.

First and foremost, to verify results of the original MREcuri study more clearly and with less ambiguity, further communications with the original MREcuri authors on their methodology is needed.

In terms of improving the extension GMR-ND model in a PyGeometrics-based environment, several concrete adaptations are possible. Specifically, the impact of several modular components of the model could be investigated. For example, the impact of different sampling methods could be investigated. Currently, the *LinkNeighborLoader* is used to sample batches for training, but other sampling methods like *HGTSampler* or *GraphSAINTSampler* could be used. Another example could be the integration of a GCN model to detect communities instead of employing the Louvain algorithm. Furthermore, metapath-based embeddings could be used as in the work [19] to derive more accurate embeddings. Generally, the impact of shallow and rich node features in a GNN model should be studied to a greater extent to conclude if more information on the nodes necessarily yields a better prediction outcome or not. Basic foundations of this have been laid out in this thesis by providing the four different GMR-ND models. However, these results should be verified in different recommendation settings in the future.

Apart from replication and extension model improvements, a general standardization for diversity and novelty-based metrics would benefit the comparison process between various recommendation models in the field. Concretely, in part due to the fact that such a standardized definition of novelty and diversity is missing, the deviations as outlined in Chapter 5 were documented. Since novelty and diversity are acknowledged as important metrics in the field of recommender systems [47] a clear and standardized definition would support the verification, replication and results comparison process.

Furthermore, the actual evaluation of music recommendations is of high

relevance, especially since music is a subjective preference of people. In the original and replication study many assumptions are taken to simplify and facilitate the evaluation process. These include the assumption that users would listen to all recommendations or assuming that evaluating recommended songs against a test set can be equated with songs the user actually listened to. These factors highlight the need for a more user-centric evaluation method and user experimentation on such music recommendation results.

Finally, future studies should test their models on multiple datasets and with a higher number of interactions between users and songs to make the models more robust and to increase their generalizability.

References

- [1] G. Adomavicius and A. Tuzhilin. 2005. Toward the next Generation of Recommender Systems: A Survey of the State-of-the-Art and Possible Extensions. *IEEE Transactions on Knowledge and Data Engineering* 17, 6 (June 2005), 734–749. <https://doi.org/10.1109/TKDE.2005.99>
- [2] Yassine Afoudi, Mohamed Lazaar, and Safae Hmaidi. 2023. An Enhanced Recommender System Based on Heterogeneous Graph Link Prediction. *Engineering Applications of Artificial Intelligence* 124 (Sept. 2023). <https://doi.org/10.1016/j.engappai.2023.106553>
- [3] Vivek Anand, Matthew Yang, and Zhazhan Zhao. 2022. Mitigating Filter Bubbles within Deep Recommender Systems. *arXiv preprint arXiv:2209.08180* (2022). <https://doi.org/10.48550/arXiv.2209.08180> [cs.LG]
- [4] Lorena A. Barba. 2018. Terminologies for Reproducible Research. *arXiv preprint arXiv:1802.03311* (2018). <https://doi.org/10.48550/arXiv.1802.03311> [cs.DL]
- [5] Joeran Beel, Bela Gipp, Stefan Langer, and Corinna Breiteringer. 2016. Research-Paper Recommender Systems: A Literature Survey. *International Journal on Digital Libraries* 17, 4 (Nov. 2016), 305–338. <https://doi.org/10.1007/s00799-015-0156-0>
- [6] Alejandro Bellogín and Alan Said. 2021. Improving accountability in recommender systems research through reproducibility. *User Modeling and User-Adapted Interaction* 31, 5 (2021), 941–977. <https://doi.org/10.1007/S11257-021-09302-X>

- [7] Ron Berman and Zsolt Katona. 2020. Curation Algorithms and Filter Bubbles in Social Networks. *Marketing Science* 39, 2 (2020), 296–316. <https://doi.org/10.1287/mksc.2019.1208>
- [8] Vincent D Blondel, Jean-Loup Guillaume, Renaud Lambiotte, and Etienne Lefebvre. 2008. Fast unfolding of communities in large networks. *Journal of statistical mechanics: Theory and experiment* 2008, 10 (2008), P10008. <https://doi.org/10.1088/1742-5468/2008/10/P10008>
- [9] Poonam B.Thorat, R. M. Goudar, and Sunita Barve. 2015. Survey on Collaborative Filtering, Content-based Filtering and Hybrid Recommendation System. *International Journal of Computer Applications* 110, 4 (Jan. 2015), 31–36. <https://doi.org/10.5120/19308-0760>
- [10] Donald T Campbell and Julian C Stanley. 2015. *Campbell*. Ravenio Books.
- [11] Jaime Carbonell and Jade Goldstein. 1998. The use of MMR, diversity-based reranking for reordering documents and producing summaries. In *Proceedings of the 21st annual international ACM SIGIR conference on Research and development in information retrieval*. ACM, 335–336.
- [12] Allison J. B. Chaney, Brandon M. Stewart, and Barbara E. Engelhardt. 2018. How algorithmic confounding in recommendation systems increases homogeneity and decreases utility. In *Proceedings of the 12th ACM Conference on Recommender Systems (Vancouver, British Columbia, Canada) (RecSys '18)*. Association for Computing Machinery, 224–232. <https://doi.org/10.1145/3240323.3240370>
- [13] Yashar Deldjoo, Markus Schedl, and Peter Knees. 2024. Content-Driven Music Recommendation: Evolution, State of the Art, and Challenges. *Computer Science Review* 51 (Feb. 2024). <https://doi.org/10.1016/j.cosrev.2024.100618>
- [14] Tommaso Di Noia, Vito Claudio Ostuni, Jessica Rosati, Paolo Tomeo, and Eugenio Di Sciascio. 2014. An analysis of users’ propensity toward diversity in recommendations. In *Proceedings of the 8th ACM Conference on Recommender Systems (RecSys '14)*. Association for Computing Machinery, 285–288. <https://doi.org/10.1145/2645710.2645774>
- [15] Tommaso Di Noia, Vito Claudio Ostuni, Jessica Rosati, Paolo Tomeo, and Eugenio Di Sciascio. 2014. An analysis of users’ propensity toward

- diversity in recommendations. In *Proceedings of the 8th ACM Conference on Recommender Systems*. ACM, 285–288. <https://doi.org/10.1145/2645710.2645774>
- [16] Wenqi Fan, Yao Ma, Qing Li, Yuan He, Eric Zhao, Jiliang Tang, and Dawei Yin. 2019. Graph Neural Networks for Social Recommendation. In *Proceedings of the World Wide Web Conference*. ACM, 417–426. <https://doi.org/10.1145/3308558.3313488>
- [17] Maurizio Ferrari Dacrema, Simone Boglio, Paolo Cremonesi, and Dietmar Jannach. [n. d.]. A Troubling Analysis of Reproducibility and Progress in Recommender Systems Research. 39, 2 ([n. d.]), 20:1–20:49. <https://doi.org/10.1145/3434185>
- [18] Sarah Frost, Manu Mathew Thomas, and Angus G Forbes. 2019. Art I don’t like: An anti-recommender system for visual art. In *Proceedings of Museums and the Web*, Vol. 17.
- [19] Xinyu Fu, Jiani Zhang, Ziqiao Meng, and Irwin King. 2020. MAGNN: Metapath Aggregated Graph Neural Network for Heterogeneous Graph Embedding. In *Proceedings of The Web Conference 2020 (WWW ’20)*. ACM, 2331–2341. <https://doi.org/10.1145/3366423.3380297>
- [20] Rumi Ghosh and Kristina Lerman. 2009. Structure of Heterogeneous Networks. In *2009 International Conference on Computational Science and Engineering*, Vol. 4. 98–105. <https://doi.org/10.1109/CSE.2009.142>
- [21] David Goldberg, David Nichols, Brian M. Oki, and Douglas Terry. 1992. Using Collaborative Filtering to Weave an Information Tapestry. *Commun. ACM* 35, 12 (Dec. 1992), 61–70. <https://doi.org/10.1145/138859.138867>
- [22] Omar S Gómez, Natalia Juristo, and Sira Vegas. 2014. Understanding replication of experiments in software engineering: A classification. *Information and Software Technology* 56, 8 (2014), 1033–1048. <https://doi.org/10.1016/J.INFSOF.2014.04.004>
- [23] Quentin Grossetti, Cédric Du Mouza, and Nicolas Travers. 2019. Community-based recommendations on Twitter: Avoiding the filter bubble. In *Web Information Systems Engineering–WISE 2019: 20th International Conference, Hong Kong, China, January 19–22, 2020, Proceedings 20*. Springer, 212–227. https://doi.org/10.1007/978-3-030-34223-4_14

- [24] Aric Hagberg, Pieter J. Swart, and Daniel A. Schult. [n. d.]. Exploring network structure, dynamics, and function using NetworkX. ([n. d.]). <https://www.osti.gov/biblio/960616>
- [25] Will Hamilton, Zhitao Ying, and Jure Leskovec. 2017. Inductive Representation Learning on Large Graphs. In *Advances in Neural Information Processing Systems*, I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett (Eds.), Vol. 30. Curran Associates, Inc., 1–11.
- [26] Aaron Christian Hansel, Adrianus, Lunardi Pradana, Abba Suganda, Girsang, and Ariadi Nugroho. 2022. Optimized LightGCN for Music Recommendation Satisfaction. In *2022 6th International Conference on Information Technology, Information Systems and Electrical Engineering (ICITISEE)*. IEEE, 449–454. <https://doi.org/10.1109/ICITISEE57756.2022.10057831>
- [27] Christian Hansen, Rishabh Mehrotra, Casper Hansen, Brian Brost, Lucas Maystre, and Mounia Lalmas. 2021. Shifting Consumption towards Diverse Content on Music Streaming Platforms. In *Proceedings of the 14th ACM International Conference on Web Search and Data Mining (WSDM '21)*. Association for Computing Machinery, 238–246. <https://doi.org/10.1145/3437963.3441775>
- [28] Yifan Hu, Yehuda Koren, and Chris Volinsky. 2008. Collaborative filtering for implicit feedback datasets. In *2008 Eighth IEEE international conference on data mining*. Ieee, IEEE, 263–272. <https://doi.org/10.1109/ICDM.2008.22>
- [29] Ziniu Hu, Yuxiao Dong, Kuansan Wang, and Yizhou Sun. 2020. Heterogeneous Graph Transformer. In *Proceedings of The Web Conference 2020 (Taipei, Taiwan) (WWW '20)*. Association for Computing Machinery, 2704–2710. <https://doi.org/10.1145/3366423.3380027>
- [30] Gourav Jain, Tripti Mahara, and Kuldeep Narayan Tripathi. 2020. A Survey of Similarity Measures for Collaborative Filtering-Based Recommender System. In *Soft Computing: Theories and Applications*, Millie Pant, Tarun K. Sharma, Om Prakash Verma, Rajesh Singla, and Afzal Sikander (Eds.). Vol. 1053. Springer Singapore, 343–352. https://doi.org/10.1007/978-981-15-0751-9_32
- [31] Ray Jiang, Silvia Chiappa, Tor Lattimore, András György, and Pushmeet Kohli. 2019. Degenerate Feedback Loops in Recommender Systems. In *Proceedings of the 2019 AAAI/ACM Conference on AI, Ethics,*

- and Society* (Honolulu, HI, USA) (*AIES '19*). Association for Computing Machinery, 383–390. <https://doi.org/10.1145/3306618.3314288>
- [32] Thomas N. Kipf and Max Welling. 2017. Semi-Supervised Classification with Graph Convolutional Networks. *arXiv preprint arXiv:1609.02907* arXiv:1609.02907 (Feb. 2017).
 - [33] Jure Leskovec. 2024. CS224W | Home — web.stanford.edu. <http://web.stanford.edu/class/cs224w>. [Accessed 17-12-2024].
 - [34] Chen Li, Yang Cao, Ye Zhu, Debo Cheng, Chengyuan Li, and Yasuhiko Morimoto. 2024. Ripple Knowledge Graph Convolutional Networks for Recommendation Systems. *Machine Intelligence Research* 21, 3 (June 2024), 481–494. <https://doi.org/10.1007/s11633-023-1440-x>
 - [35] Dawen Liang, Rahul G Krishnan, Matthew D Hoffman, and Tony Jebara. 2018. Variational autoencoders for collaborative filtering. In *Proceedings of the 2018 World Wide Web Conference*. ACM, 689–698. <https://doi.org/10.1145/3178876.3186150>
 - [36] Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg S Corrado, and Jeff Dean. 2013. Distributed Representations of Words and Phrases and their Compositionality. In *Advances in Neural Information Processing Systems*, Vol. 26. Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2013/file/9aa42b31882ec039965f3c4923ce901b-Paper.pdf
 - [37] Ramal Moonesinghe, Muin J Khoury, and A Cecile J W Janssens. 2007. Most published research findings are false—but a little replication goes a long way. *PLoS medicine* 4, 2 (2007), e28. <https://doi.org/10.1371/journal.pmed.0040028>
 - [38] Ruihui Mu. 2018. A Survey of Recommender Systems Based on Deep Learning. *IEEE Access* (2018). <https://doi.org/10.1109/ACCESS.2018.2880197>
 - [39] Eli Pariser. 2011. *The Filter Bubble: What the Internet is hiding from you*. Penguin UK.
 - [40] Alwin Poulose, Chinthala Sreya Reddy, Shatarupa Dash, and Bharat Jyoti Ranjan Sahu. 2022. Music recommender system via deep learning.

- Journal of information and optimization sciences* 43, 5 (2022), 1081–1088. <https://doi.org/10.1080/02522667.2022.2094546>
- [41] Radim Řehůřek and Petr Sojka. 2010. Software Framework for Topic Modelling with Large Corpora. In *Proceedings of the LREC 2010 Workshop on New Challenges for NLP Frameworks*. ELRA, Valletta, Malta, 45–50.
<http://is.muni.cz/publication/884893/en>.
 - [42] Paul Resnick, Neophytos Iacovou, Mitesh Suchak, Peter Bergstrom, and John Riedl. 1994. GroupLens: An Open Architecture for Collaborative Filtering of Netnews. In *Proceedings of the 1994 ACM Conference on Computer Supported Cooperative Work - CSCW '94*. ACM Press, 175–186. <https://doi.org/10.1145/192844.192905>
 - [43] Deepjyoti Roy and Mala Dutta. 2022. A Systematic Review and Research Perspective on Recommender Systems. *Journal of Big Data* 9, 1 (May 2022), 59. <https://doi.org/10.1186/s40537-022-00592-5>
 - [44] Dimitris Sacharidis. 2019. Diversity and novelty in social-based collaborative filtering. In *Proceedings of the 27th ACM Conference on User Modeling, Adaptation and Personalization*. ACM, 139–143. <https://doi.org/10.1145/3320435.3320479>
 - [45] Franco Scarselli, Marco Gori, Ah Chung Tsoi, Markus Hagenbuchner, and Gabriele Monfardini. 2009. The Graph Neural Network Model. *IEEE Transactions on Neural Networks* 20, 1 (Jan. 2009), 61–80. <https://doi.org/10.1109/TNN.2008.2005605>
 - [46] Markus Schedl, Hamed Zamani, Ching-Wei Chen, Yashar Deldjoo, and Mehdi Elahi. 2018. Current challenges and visions in music recommender systems research. *International Journal of Multimedia Information Retrieval* 7 (2018), 95–116. <https://doi.org/10.1007/s13735-018-0154-2>
 - [47] Elena Shakirova. 2017. Collaborative Filtering for Music Recommender System. In *2017 IEEE Conference of Russian Young Researchers in Electrical and Electronic Engineering (EIConRus)* (St. Petersburg and Moscow, Russia). IEEE, 548–550. <https://doi.org/10.1109/EIConRus.2017.7910613>
 - [48] Meenakshi Sharma and Sandeep Mann. 2013. A survey of recommender systems: approaches and limitations. *International Journal of Innovations in Engineering and Technology* 2, 2 (2013), 8–14.

- [49] Ritu Sharma, Dinesh Gopalani, and Yogesh Meena. 2017. Collaborative Filtering-Based Recommender System: Approaches and Research Challenges. In *2017 3rd International Conference on Computational Intelligence & Communication Technology (CICT)*. IEEE, 1–6. <https://doi.org/10.1109/CICT.2017.7977363>
- [50] Jagendra Singh. 2020. Collaborative Filtering Based Hybrid Music Recommendation System. In *2020 3rd International Conference on Intelligent Sustainable Systems (ICISS)*. IEEE, 186–190. <https://doi.org/10.1109/ICISS49785.2020.9315913>
- [51] Dag I. K. Sjøberg and Gunnar Rye Bergersen. 2023. Construct Validity in Software Engineering. *IEEE Transactions on Software Engineering* 49, 3 (March 2023), 1374–1396. <https://doi.org/10.1109/TSE.2022.3176725>
- [52] Yading Song, Simon Dixon, and Marcus Pearce. 2012. A Survey of Music Recommendation Systems and Future Perspectives. *9th International Symposium on Computer Music Modelling and Retrieval (CMMR 2012)* (2012).
- [53] A. Sperduti and A. Starita. 1997. Supervised Neural Networks for the Classification of Structures. *IEEE Transactions on Neural Networks* 8, 3 (May 1997), 714–735. <https://doi.org/10.1109/72.572108>
- [54] M Sridevi, R Rajeshwara Rao, and M Varaprasad Rao. 2016. A survey on recommender system. *International Journal of Computer Science and Information Security* 14, 5 (2016), 265.
- [55] Xiaoyuan Su and Taghi Khoshgoftaar. 2009. A Survey of Collaborative Filtering Techniques. *Advances in Artificial Intelligence* 2009 (2009), 19. <https://doi.org/10.1155/2009/421425>
- [56] Jianing Sun, Wei Guo, Dengcheng Zhang, Yingxue Zhang, Florence Regol, Yaochen Hu, Huifeng Guo, Ruiming Tang, Han Yuan, Xiuqiang He, et al. 2020. A framework for recommending accurate and diverse items using bayesian graph convolutional neural networks. In *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*. 2030–2039.
- [57] Jianing Sun, Wei Guo, Dengcheng Zhang, Yingxue Zhang, Florence Regol, Yaochen Hu, Huifeng Guo, Ruiming Tang, Han Yuan, Xiuqiang He, and Mark Coates. 2020. A Framework for Recommending Accurate

- and Diverse Items Using Bayesian Graph Convolutional Neural Networks. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining* (Virtual Event, CA, USA) (*KDD '20*). Association for Computing Machinery, 2030–2039. <https://doi.org/10.1145/3394486.3403254>
- [58] Maria Taramigkou, Efthimios Bothos, Konstantinos Christidis, Dimitris Apostolou, and Gregoris Mentzas. 2013. Escape the bubble: guided exploration of music preferences for serendipity and novelty. In *Proceedings of the 7th ACM Conference on Recommender Systems* (Hong Kong, China) (*RecSys '13*). Association for Computing Machinery, 335–338. <https://doi.org/10.1145/2507157.2507223>
- [59] Antonela Tommasel, Juan Manuel Rodriguez, and Daniela Godoy. 2021. I Want to Break Free! Recommending Friends from Outside the Echo Chamber. In *Proceedings of the 15th ACM Conference on Recommender Systems* (Amsterdam, Netherlands) (*RecSys '21*). Association for Computing Machinery, 23–33. <https://doi.org/10.1145/3460231.3474270>
- [60] Antonela Tommasel, Juan Manuel Rodriguez, and Daniela Godoy. 2022. Haven't I just Listened to This?: Exploring Diversity in Music Recommendations. In *Adjunct Proceedings of the 30th ACM Conference on User Modeling, Adaptation and Personalization* (Barcelona, Spain) (*UMAP '22 Adjunct*). Association for Computing Machinery, 35–40. <https://doi.org/10.1145/3511047.3536409>
- [61] Saúl Vargas and Pablo Castells. 2013. Exploiting the diversity of user preferences for recommendation. In *Proceedings of the 10th Conference on Open Research Areas in Information Retrieval*. ACM, 129–136. <https://doi.org/10.5555/2491748.2491776>
- [62] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. 2018. Graph Attention Networks. *arXiv preprint arXiv:1710.10903* arXiv:1710.10903 (Feb. 2018). <https://doi.org/10.48550/arXiv.1710.10903> [cs, stat]
- [63] Dongjing Wang, Guandong Xu, and Shuiguang Deng. 2017. Music Recommendation via Heterogeneous Information Graph Embedding. In *2017 International Joint Conference on Neural Networks (IJCNN)*. IEEE, 596–603. <https://doi.org/10.1109/IJCNN.2017.7965907>

- [64] Dongjing Wang, Xin Zhang, Yuyu Yin, Dongjin Yu, Guandong Xu, and Shuiguang Deng. 2023. Multi-View Enhanced Graph Attention Network for Session-Based Music Recommendation. *ACM Trans. Inf. Syst.* 42, 1 (Aug. 2023), 16:1–16:30. <https://doi.org/10.1145/3592853>
- [65] Jun Wang, Arjen P. De Vries, and Marcel J. T. Reinders. 2006. Unifying User-Based and Item-Based Collaborative Filtering Approaches by Similarity Fusion. In *Proceedings of the 29th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*. ACM, 501–508. <https://doi.org/10.1145/1148170.1148257>
- [66] Wenjie Wang, Fuli Feng, Liqiang Nie, and Tat-Seng Chua. 2022. User-controllable Recommendation Against Filter Bubbles. In *Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval (Madrid, Spain) (SIGIR '22)*. Association for Computing Machinery, 1251–1261. <https://doi.org/10.1145/3477495.3532075>
- [67] He Weng, Jianjiang Chen, Dongjing Wang, Xin Zhang, and Dongjin Yu. 2022. Graph-Based Attentive Sequential Model With Metadata for Music Recommendation. *IEEE Access* 10 (2022), 108226–108240. <https://doi.org/10.1109/ACCESS.2022.3213812>
- [68] Shiwen Wu, Fei Sun, Wentao Zhang, Xu Xie, and Bin Cui. 2023. Graph Neural Networks in Recommender Systems: A Survey. *ACM Computing Surveys* 55, 5 (May 2023), 1–37. <https://doi.org/10.1145/3535101>
- [69] Zonghan Wu, Shirui Pan, Fengwen Chen, Guodong Long, Chengqi Zhang, and Philip S. Yu. 2021. A Comprehensive Survey on Graph Neural Networks. *IEEE Transactions on Neural Networks and Learning Systems* 32, 1 (Jan. 2021), 4–24. <https://doi.org/10.1109/TNNLS.2020.2978386>
- [70] Han Xiao, Dezhi Sun, and Fucheng You. 2023. Knowledge Aggregation of Importance Sampling Based on GNN for Music Recommendation. In *2023 7th International Conference on Electrical, Mechanical and Computer Engineering (ICEMCE)*. IEEE, 858–861. <https://doi.org/10.1109/ICEMCE60359.2023.10490603>
- [71] Hanqing Zeng, Hongkuan Zhou, Ajitesh Srivastava, Rajgopal Kannan, and Viktor K. Prasanna. 2020. GraphSAINT: Graph Sampling Based Inductive Learning Method. In *8th International Conference on Learning Representations, ICLR 2020, Addis Ababa, Ethiopia, April 26-30, 2020*. arXiv, 1–19. <https://doi.org/10.48550/arXiv.1907.04931>

- [72] Yutao Zhang, Jie Tang, Zhilin Yang, Jian Pei, and Philip S Yu. 2015. Cosnet: Connecting heterogeneous social networks with local and global consistency. In *Proceedings of the 21th ACM SIGKDD international conference on knowledge discovery and data mining*. ACM, 1485–1494. <https://doi.org/10.1145/2783258.2783268>

Appendices

A Communication with Tommasel et al.

05.07.24, 13:33

GMX - Meeting Request to Discuss Outstanding Questions

GMX FreeMail

Meeting Request to Discuss Outstanding Questions

Von: "Benedikt Badke" <benediktbadke@gmx.de>
An: "Antonela Tommasel" <antonela.tommasel@isistan.unicen.edu.ar>
CC: "Timo Rodinger" <timo.rodinger@s.wu.ac.at>
Datum: 17.04.2024 11:10:40

Hi Antonela,

I hope this message finds you well!

I wanted to follow up on the various questions we've been discussing. Given the complexity and number of these questions, I believe it would be more effective to address them in a meeting.

Could we schedule some time to meet next week?

Thank you in advance for your time. Looking forward to your reply.

Best regards, Benedikt

Am 27.03.2024 um 10:20 schrieb Benedikt Badke <benediktbadke@gmx.de>:

Hi Antonela,

Thank you very much for your answer and the provided insights.

Regarding the selection process described, am I correct in understanding that you chose a quarter of the top 20% users with the most social interactions, based on the amount of scrobbles they had, to achieve the 5% mentioned in the study? Or is this a misunderstanding?

Also, I've included a colleague in this conversation (CC:) who is collaborating with me on a study that expands upon your original work. We are seeking to comprehend the various decisions you made during your research. There are certain aspects we are still trying to grasp completely, thus we have compiled a document with questions. These inquiries will cover the following topics:

1. Scripts for scraping the data
2. Tag data available on GitHub
3. Test/Train split based on scrobbling date

We highly appreciate your help on these aspects to better understand your study and decision points. We can also schedule a meeting to cover all the questions in person, as this might be more time efficient for you.

If you have any question related to one of our inquiries, feel free to contact us anytime. Thanks in advance for your help & best regards
Benedikt

<Questions to the Original Study.pdf>

Am 25.03.2024 um 18:52 schrieb Antonela Tommasel <antonela.tommasel@isistan.unicen.edu.ar>:

Dear Benedikt,

Apologies for my late reply and thanks for your interest in our research.

There are many music-based datasets in the literature, but none of them included all the data we wanted (e.g., some of them had the anonymized listening history but no social interactions, others consisted of anonymized social interactions but no listening history). So, we needed to create our own dataset. The only related dataset we found that didn't anonymize user names was the one from Zhang et al. [1], which only included the social interactions of over 136k users. As it would've been impossible to scrape all data from [Last.fm](#), we started by selecting approx 20% of users (we might have included a few more users) with the largest number of social interactions. Not all of those users were still available in [last.fm](#). For those who were, we collected the scrobble information. Finally, we kept those users with the largest listening history.

file:///Users/elvira/Downloads/Meeting Request to Discuss Outstanding Questions.html

1/2

05.07.24, 13:33

GMX - Meeting Request to Discuss Outstanding Questions

[1] Yutao Zhang, Jie Tang, Zhilin Yang, Jian Pei, and Philip S Yu. Cosnet: Connecting heterogeneous social networks with local and global consistency. In Proceedings of the 21th ACM SIGKDD international conference on knowledge discovery and data mining, pages 1485–1494, 2015.

Hope this answers your question.

Best regards,
Antonela

On Sat, Mar 16, 2024 at 5:32 AM Benedikt Badke <benediktbadke@gmx.de> wrote:

Dear Mrs. Antonela Tommasel,

My name is Benedikt Badke, and I am a Master Thesis Student at the University of Economics and Business in Vienna. Currently, I am writing my master thesis in the field of diversity in music recommender systems, and I am focusing on the MRecur Algorithm you proposed in the paper "Haven't I just Listened to This?: Exploring Diversity in Music Recommendations". I think your field of research is super interesting and, thus I decided to focus on this topic in my master thesis. Based on the thesis, I am planning so scrape a new dataset from the Last.FM API, and validate the results with a new dataset.

Therefore, I am currently preparing all the relevant scripts for scraping the data. One question arose during reading your paper, related to the user selection: You wrote in your study "we selected the top 5% of users with the highest number of social connections and scrobbles". How did you balance the number of connections and scrobbles for the selection procedure? Did you simply add them or did you took the top 5% of connections and scrobbles individually?

Thank you in advance for your help and support. Best regards
Benedikt Badke

--

T: +54 249 4385650 interno 2301
antonela.tommasel@isistan.unicen.edu.ar
<https://tommantonela.github.io>
www.isistan.unicen.edu.ar
Campus Universitario, Paraje Arroyo Seco.
Tandil, Buenos Aires, Argentina
 [Facebook icon](#)  [Twitter icon](#)

B First Set of Questions

Questions to the original Study

1. Question: Data Extraction Files
2. Question: Tag Data available on GitHub
3. Question: Test/Train Split based on Listening Date

Context of our Study:

We are in the process of completing our Master's Thesis at the University of Economics and Business in Vienna, and our research focuses on the area of Music Recommender Systems. Your study, titled "Haven't I Heard This Before?: Investigating Diversity in Music Recommendations," is of particular interest to us. Our project involves both a replication of and an expansion on your original research. To enhance our understanding and strengthen our study's methodology, we seek clarification on a few points that were not fully explained in the GitHub repository or in your study itself. We want to highlight that our inquiries are intended purely for academic insight and are not critiques.

Topic 1 - Data Extraction Files:

Currently, we are scraping a dataset via the Last.FM API to validate the algorithm and to see how it generalizes on a new dataset. For this purpose, we want to scrape a dataset which closely aligns with the original dataset you used in your study. Thus, we had the following question:

Could you provide us with the scripts used to extract the data from the Last.FM API?

Topic 2 – Tag Data available on GitHub:

One key element of the intended study is to replicate parts of your study with the original data in a full graph neural network context. So leaving all users, songs in the graph. Unfortunately, the tags that are provided in the GitHub repository are not mapped correctly, so it is not possible to identify which song/artist has which tag/tags. Currently, the mapping that can be accessed is in the screenshot below (SC1) and shows both TagID and ArtistID just counting up. Therefore, it currently misses a concrete mapping to retrieve information about the matched artists and tags. Your source code does not provide evidence to this aspect, and we are currently missing access to the original MangoDB Database. Thus, we want to address the following questions:

Would you be able to provide us with the original Tag data for Artists and Songs?

Or alternatively, can you point out what method you used to collect your artist/ song tags?

Topic 3 - Test/Train Split based on Listening Date:

Furthermore, we have seen that a large portion of the "Pos" (Position) / "Date", so any kind of time related data (order data), to when a user last listened to a specific song, is missing in the data on GitHub. About 82% of the data has no such information. As a result, and according to how you split your data into Training/Test in the source code, it turns out that many of the users have a huge amount of training data vs little test

data. So, the temporal split of 70/30 only holds true in about 30% of the cases. Therefore, we came to the following questions:

Since it was not explicitly mentioned in the limitations, we wanted to know why this was the case? Was it no possible to scrape this kind of data and how do you think it impacts the trustfulness of the model?

Since many of the tracks in the training data could be songs that in reality would only have been heard by the user at a later stage, in our context, maybe even only in after the temporal test/train cut. Therefore, in the training data the model should not have knowledge about these songs.

Thanks in advance for your help and collaboration. Best regards,
Timo Rödinger & Benedikt Badke

SC1:

```
[34]: dataset["tag_id"]

[34]: {'indie pop': 1,
      'indie rock': 2,
      'indie': 3,
      'black kids': 4,
      'rock': 5,
      'alternative': 6,
      'alternative rock': 7,
      'pop': 8,
      'indie disco': 9,
      'electropop': 10,
      'love at first listen': 11,
      'summer': 12,
      'happy': 13,
      'catchy': 14,
      'electronic': 15,
      'freakin the night': 16,
      ...}

[12]: dataset["artist_id"]

[12]: {'robin schulz': 0,
      'darya dadvar': 1,
      'colle der fomento': 2,
      'charlie parker quartet': 3,
      'anastasija': 4,
      'nik turner': 5,
      'sounds of blackness': 6,
      'linda perhacs': 7,
      ...}
```


C Second Set of Questions

Questions Part 2

Context Explanation: I am rebuilding your model with a few additional features such as a full knowledge graph and GCN in the Pygeometrics framework. The idea was to benchmark if the usage of a full GCN integration was feasible given relatively weak computational power (also GTX3060) and if it was performing better than your hybrid approach of GCN + CNN.

In this context it is of course important for me to re-use as much of the data that your original study used to have comparative benchmarks. Unfortunately as said, the Tags data is not usable, and the Spotify Audio features are not included in the GitHub repository. For this reason I scraped the data myself to the extent possible and will re-run your model as the baseline with my new Tag and Audio Features data.

This brings me to my next question of how exactly your evaluation Metrics of "Diversity and Novelty" or rather how the "Structural" and the "content-based" approach metrics were actually calculated. They are explained in theory in the paper but unfortunately these metrics and how you derive them are not included in the source code.

Therefore, I aim to replicate these metrics of "structural and content-based" diversity as well as possible too. Unfortunately there are still a few things unclear to me that I would be grateful to get have your input on:

1. Co-Listening Graph: on the structural dissimilarities you define the "Co-Listening Graph" as "two tracks are connected if they were listened by the same user". Which I tried to replicate but realized such a graph with 3307 users and 252013 tracks is insanely big in terms of size. I first of all was not able to compute it due to it being over at least over 64GB in size and it didn't fit my RAM (64GB), and even if I tried to compute it in batches and save it on disk I assume a community algorithm like Louvain would take ages to compute. Please clarify if this is really the final implementation of how you computed the co-listening graph or if it ended up as something different. For example if we just take the normal "user-track graph", Louvain detects 8-11 communities which seems reasonable.
2. I assumed the user-track graph as the "Co-Listening Graph" and computed structural and content-based dissimilarity based on the description of your paper and the paper you cited. Mostly the works from Prof. Castells seem to be relevant for the "Novelty and Diversity". But here unfortunately I realized that the definitions for Novelty and Diversity always need "Track Embeddings" to be comparable. Eg. Diversity by comparing all recommended tracks to each other. But the Structural and Content based dissimilarities are actually described in your paper as "user embeddings". Hence I am having a hard time to understand how to explicitly use the user embeddings in the context of Diversity and Novelty. I would be grateful for an explanation or providence of your code for the evaluation metrics.